

lytlnyblOS

Learning Manual

Build a mental model of a small 32-bit operating system

An original, source-led introduction for a programmer starting from zero.
Read it beside the final implementation in `lytlnybl/`, experiment in
QEMU, and make small changes safely.

September 2026

How to use this manual

This book explains the operating system that is present in this repository. It is not a line-by-line copy of the existing guide. Its job is to give a new learner a reliable map: what each component does, why it must exist, which source file owns it, and how to test an idea before moving on.

The project is a deliberately small 32-bit x86 operating system. It boots through a BIOS-compatible virtual machine, switches from 16-bit real mode to 32-bit protected mode, then runs a C kernel. The finished system includes text output, interrupts, a timer, a keyboard, page-based memory management, a heap, cooperative pieces of process management, user mode, system calls, a tiny filesystem, a program loader, and a shell.

You do not need to memorize every x86 detail. For each chapter, first understand the promise made by a subsystem. Then open the named files and trace the path of one request through them. Finally, perform the checkpoint. Small operating systems are unforgiving: one wrong address or one missing interrupt acknowledgement can stop everything. That is exactly why short experiments are more valuable than large edits.

What you should know first

You should be comfortable with variables, functions, arrays, pointers, and compiling a small C program. Some basic hexadecimal notation is enough: `0x10` is sixteen, and an address is a number that names a byte in memory. Assembly is introduced only where the CPU interface requires it.

Safety and scope

Run the system in QEMU, not on a physical disk. The build process creates a disposable raw disk image under `lytlnybl/build/`. This implementation is a teaching kernel, not a secure or complete operating system. Several choices trade robustness for clarity; the final chapter names useful improvements.

Contents

How to use this manual	i
1 The whole system at a glance	1
1.1 Build products	1
2 Booting: from firmware to your first instruction	2
2.1 The boot contract	2
2.2 Real mode and segmented addresses	2
2.3 The memory map	2
3 Entering 32-bit protected mode	4
3.1 Why change CPU mode?	4
3.2 The Global Descriptor Table	4
3.3 Linking fixes addresses	4
4 The first C kernel and text output	5
4.1 Kernel initialization order	5
4.2 VGA text mode	5
5 Interrupts: events that pause the current code	6
5.1 Three kinds of events	6
5.2 The IDT and assembly stubs	6
5.3 PIC remapping and acknowledgement	6
5.4 Faults are diagnostics	6
6 Timer and keyboard drivers	7
6.1 The timer creates time	7
6.2 Keyboard input is asynchronous	7
6.3 Polling versus interrupts	7
7 Physical memory: allocating 4 KiB frames	8
7.1 Physical and virtual are different questions	8
8 Paging and the kernel heap	9
8.1 Two-level paging	9
8.2 A simple kernel allocator	9
9 Processes and context switching	10
9.1 What a process owns here	10
9.2 Saving a paused CPU	10
9.3 Round-robin scheduling	10

10 User mode and system calls	11
10.1 Privilege is a hardware rule	11
10.2 The syscall boundary	11
10.3 The tiny user library	11
11 Disk I/O and the filesystem	12
11.1 ATA sectors	12
11.2 On-disk layout	12
11.3 Host tool versus kernel code	12
12 Programs and the shell	13
12.1 Loading a flat user binary	13
12.2 Reading the shell as a complete vertical slice	13
I Detailed guided study	14
13 Before changing code: the machine model	15
13.1 Bits, bytes, words, and hexadecimal	15
13.2 Registers and the stack	15
13.3 Three address meanings	16
13.4 The C runtime you do not get	16
14 A precise boot-to-C walkthrough	17
14.1 Stage 1: BIOS gives the boot sector control	17
14.2 Stage 2: preserve information about RAM	17
14.3 Stage 3: read the kernel	17
14.4 Stage 4: the second assembly entry	18
14.5 Linking is part of booting	18
15 Descriptor tables and interrupt frames in depth	19
15.1 GDT selectors and privilege levels	19
15.2 What the CPU puts on a stack	19
15.3 Exceptions, IRQs, and system calls use different policies	19
15.4 Page faults as a worked example	20
16 Device drivers as state machines	21
16.1 Port I/O is not memory I/O	21
16.2 PIT timer calculation	21
16.3 Keyboard scan codes and lines of input	21
16.4 ATA PIO disk operations	21
17 Memory management laboratory	23
17.1 Read the PMM initialization from end to end	23
17.2 Page flags and protection	23
17.3 Walk one translation manually	23
17.4 Heap metadata and fragmentation	23
18 Virtual memory from zero: a slow, visual explanation	25
18.1 The child-friendly picture: street names and houses	25
18.2 Three labels for one byte	25
18.3 Why pages are 4096 bytes	26
18.4 One big address split into three pieces	26

18.5	Do a complete translation with the project's formulas	26
18.6	What a page-table entry stores	27
18.7	Read <code>init_vmm</code> like a recipe	27
18.8	Read <code>map_page</code> one line at a time	27
18.9	Why the TLB exists, and why it must be invalidated	28
18.10	The kernel heap is a useful real mapping	28
18.11	User address spaces in this exact system	28
18.12	What happens during a page fault	28
18.13	Common beginner confusions, answered carefully	29
18.14	A safe virtual-memory practice sequence	29
19	Process, scheduler, and privilege-transition laboratory	31
19.1	Lifecycle of a kernel process	31
19.2	Lifecycle of a user process	31
19.3	Scheduling states tell the story	31
19.4	What context switching does not solve	32
20	Filesystem and userspace laboratory	33
20.1	Calculate the on-disk structures	33
20.2	Formatting creates invariants	33
20.3	Follow an open/read/write request	33
20.4	From a host file to a running program	34
20.5	Shell commands are an API exercise	34
21	Build, test, and debug like a kernel developer	35
21.1	Make clean experiments	35
21.2	Use staged evidence	35
21.3	GDB and QEMU	35
21.4	A review checklist for every extension	35
21.5	Capstone projects	36
22	A practical learning path	37
22.1	A debugging routine	37
23	What to improve next	38
II	Guided source reference	39
A	How to read this reference	40
B	Build contract and boot path	41
B.1	<code>../lytlnybl/Makefile</code>	41
B.2	<code>../lytlnybl/boot/bootloader.asm</code>	45
B.3	<code>../lytlnybl/kernel/linker.ld</code>	47
B.4	<code>../lytlnybl/kernel/kernel_intro.asm</code>	48
B.5	<code>../lytlnybl/kernel/kernel_main.c</code>	51
C	Interrupt architecture and basic kernel services	54
C.1	<code>../lytlnybl/include/kernel/interrupts.h</code>	54
C.2	<code>../lytlnybl/kernel/interrupts.asm</code>	58
C.3	<code>../lytlnybl/kernel/interrupts.c</code>	63
C.4	<code>../lytlnybl/include/kernel/kernel_utils.h</code>	70

C.5	../lytlnybl/kernel/kernel_utils.c	71
C.6	../lytlnybl/include/kernel/mappings.h	72
D	Screen, timer, keyboard, and disk drivers	73
D.1	../lytlnybl/include/kernel/drivers/vga_text.h	73
D.2	../lytlnybl/kernel/drivers/vga_text.c	76
D.3	../lytlnybl/include/kernel/drivers/timer.h	79
D.4	../lytlnybl/kernel/drivers/timer.c	80
D.5	../lytlnybl/include/kernel/drivers/keyboard.h	82
D.6	../lytlnybl/kernel/drivers/keyboard.c	83
D.7	../lytlnybl/include/kernel/drivers/ata.h	88
D.8	../lytlnybl/kernel/drivers/ata.c	89
E	Physical frames, virtual pages, and the heap	91
E.1	../lytlnybl/include/memory/pmm.h	91
E.2	../lytlnybl/memory/pmm.c	92
E.3	../lytlnybl/include/memory/vmm.h	96
E.4	../lytlnybl/memory/vmm.asm	97
E.5	../lytlnybl/memory/vmm.c	98
E.6	../lytlnybl/include/memory/heap.h	101
E.7	../lytlnybl/memory/heap.c	102
F	Processes, context, scheduler, and loading	105
F.1	../lytlnybl/include/tasks/procman.h	105
F.2	../lytlnybl/tasks/procman.asm	108
F.3	../lytlnybl/tasks/procman.c	109
F.4	../lytlnybl/include/tasks/context.h	113
F.5	../lytlnybl/tasks/context.c	114
F.6	../lytlnybl/include/tasks/scheduler.h	116
F.7	../lytlnybl/tasks/scheduler.c	117
F.8	../lytlnybl/include/tasks/loader.h	118
F.9	../lytlnybl/tasks/loader.c	119
G	Filesystem layout, core, and manager	120
G.1	../lytlnybl/include/filesystem/fs_layout.h	120
G.2	../lytlnybl/include/filesystem/fs.h	122
G.3	../lytlnybl/filesystem/fs.c	123
G.4	../lytlnybl/include/filesystem/fs_manager.h	134
G.5	../lytlnybl/filesystem/fs_manager.c	135
G.6	../lytlnybl/tools/mkfs.c	141
H	User library, binaries, and shell	151
H.1	../lytlnybl/user/user.ld	151
H.2	../lytlnybl/include/user/libc/syscalls.h	152
H.3	../lytlnybl/user/libc/syscalls.asm	153
H.4	../lytlnybl/user/libc/syscalls.c	154
H.5	../lytlnybl/include/user/libc/memory.h	155
H.6	../lytlnybl/user/libc/memory.c	156
H.7	../lytlnybl/include/user/libc/string.h	160
H.8	../lytlnybl/user/libc/string.c	161
H.9	../lytlnybl/include/user/libc/chars.h	163
H.10	../lytlnybl/user/libc/chars.c	164

H.11	../lytlnybl/include/user/libc/output.h	165
H.12	../lytlnybl/user/libc/output.c	166
H.13	../lytlnybl/include/user/programs/shell.h	168
H.14	../lytlnybl/user/programs/shell.c	169
H.15	../lytlnybl/user/programs/test.c	173
I	Reference study projects	175
	Quick reference	176

Chapter 1

The whole system at a glance

An operating system is the program that takes control after firmware and provides controlled access to hardware. This one is a chain of hand-offs. Each link depends on the one before it.

```
BIOS -> boot sector -> kernel entry assembly -> kernel_main()
      -> interrupt and memory services -> processes -> user shell
      -> filesystem and disk driver
```

The most important folders are listed below. Treat headers in `lytlnybl/include/` as the contracts between modules and `.c/.asm` files as their implementations.

Location	Responsibility
<code>boot/</code>	The 512-byte boot sector that BIOS can load.
<code>kernel/</code>	Protected-mode entry point, interrupts, utility code, and hardware drivers.
<code>memory/</code>	Physical frames, page tables, and the kernel heap.
<code>tasks/</code>	Processes, saved CPU context, scheduling, and loading user code.
<code>filesystem/</code>	On-disk structures and higher-level file operations.
<code>user/</code>	Tiny user library and programs, including the shell.
<code>tools/mkfs.c</code>	A host-side program that puts <code>rootfs/</code> files in the image.
<code>Makefile</code>	The build recipe that connects all these pieces.

1.1 Build products

The Makefile uses an `i686-elf` cross compiler for code that will run in the guest and the host compiler for `mkfs`. Cross compilation matters because the kernel must not silently depend on the host operating system or its startup files.

```
bootloader.asm -> bootloader.bin
kernel C + assembly -> kernel.elf -> kernel.bin
user C + libc -> program ELF -> flat program binary
bootloader.bin + kernel.bin -> kernel.img
rootfs/ + kernel.img -> mkfs -> formatted filesystem image
```

From `lytlnybl/`, run `make run`. It builds `build/kernel.img` and gives that image to `qemu-system-i386`. If tools are missing, install NASM, QEMU, Make, a host C compiler, and an `i686-elf` GCC/binutils cross toolchain. Do not replace the cross compiler with normal GCC unless you understand the consequences of its host assumptions.

Checkpoint. Before reading further, locate the `run` target in `lytlnybl/Makefile`. Explain why `mkfs` is compiled with `gcc` while the kernel uses `i686-elf-gcc`.

Chapter 2

Booting: from firmware to your first instruction

2.1 The boot contract

On a BIOS-style machine, firmware searches for a bootable disk, reads its first 512-byte sector at physical address 0x7C00, and checks for the two-byte signature 0xAA55. If the signature is present, it begins executing the bytes it just loaded. There is no C runtime, heap, or operating system service yet.

The file `boot/bootloader.asm` meets this contract. Its final two lines pad the sector to 510 bytes and append the signature. This fixed size is an external rule imposed by BIOS, so a bootloader that grows past it stops being bootable.

2.2 Real mode and segmented addresses

The processor begins in 16-bit real mode. An address is commonly written as `segment:offset`; its physical value is approximately `segment * 16 + offset`. The bootloader initializes DS with 0x07C0, making labels in its loaded sector accessible through the data segment. It prints messages through BIOS interrupt 0x10, gathers the E820 memory map through `int 0x15`, and reads kernel sectors through `int 0x13`.

The disk read loads 60 sectors beginning at disk sector 2 into 0x9000. The boot sector itself occupies the first sector. The far jump `jmp 0900h:0000` transfers execution to the loaded kernel entry.

```
mov ax, 0900h          ; destination segment: physical 0x9000
mov es, ax
mov ah, 02h           ; BIOS disk read service
mov al, 60            ; number of 512-byte sectors
mov cl, 02h          ; start at sector 2
int 13h
jmp 0900h:0000
```

2.3 The memory map

The bootloader stores E820 entries at 0x5000 and their count at 0x4FFC. Later, `memory/pmm.c` reads that data to decide which physical pages can be allocated. This is a useful early lesson: a boot-time data structure is only useful if both producer and consumer agree on its exact address and layout.

Try it. Change only a bootloader greeting, rebuild, and confirm it appears before the kernel greeting. Then restore it. If NASM reports that the padding count is negative, the boot sector is too large.

Chapter 3

Entering 32-bit protected mode

3.1 Why change CPU mode?

Real mode offers BIOS services but has no useful isolation or modern paging. Protected mode gives 32-bit registers, privilege levels, descriptor tables, and the paging machinery used later by this project. Entering it is a precise sequence, implemented in `kernel/kernel_intro.asm`.

3.2 The Global Descriptor Table

The Global Descriptor Table (GDT) describes segments. This project chooses “flat” code and data segments: their base is zero and their limit covers the 32-bit address space. Segmentation therefore does not divide the application into separate regions; paging will do most address management later. The GDT still supplies selectors and ring permissions.

The entry assembly loads the GDT with `lgdt`, sets bit 0 (PE) in control register `CR0`, then performs a far jump. The far jump is essential because it loads `CS` from the new protected-mode descriptor and flushes the old instruction interpretation. It then loads `DS`, `ES`, `FS`, `GS`, and `SS`, creates a kernel stack, and calls `C`.

```
cli
lgdt [gdtr]
mov eax, cr0
or eax, 1
mov cr0, eax
jmp CODE_SELECTOR:protected_mode_entry
```

3.3 Linking fixes addresses

The linker script `kernel/linker.ld` places the kernel at `0x9000`, matching the bootloader destination. The linker resolves symbol references as if the first kernel byte lives there. If the load address and linker address disagree, jumps and data references point somewhere else.

Checkpoint. Trace the three occurrences of `0x9000`: the bootloader destination, the linker script, and the kernel’s intended start. They form one contract.

Chapter 4

The first C kernel and text output

4.1 Kernel initialization order

`kernel/kernel_main.c` is the first normal C function. It initializes components in dependency order: screen output first, then the interrupt table, physical memory, virtual memory, heap, process manager, drivers, TSS, and filesystem manager. That order is not cosmetic. For example, the heap needs pages; processes need a heap and paging; disk-based files need the ATA driver.

4.2 VGA text mode

The familiar console is not a terminal emulator at this stage. VGA text mode exposes a memory buffer at `0xB8000`. Each screen cell uses two bytes: an ASCII character and a colour attribute. `kernel/drivers/vga_text.c` wraps that raw memory in a `vga_text` structure that tracks a cursor, colors, line wrapping, scrolling, and number formatting.

Writing directly is enough to prove the kernel runs:

```
volatile char *vga = (volatile char *)0xB8000;
vga[0] = 'C';
vga[1] = 0x02;    /* green foreground on black */
```

`volatile` tells the compiler that the store is observable hardware I/O and must not be optimized away. The driver is preferable for real messages because it owns cursor state and scrolling.

Try it. Add one `vga_text_writeline` after `init_vmm()` to record successful paging initialization. A message before that call tests a different part of the startup path.

Chapter 5

Interrupts: events that pause the current code

5.1 Three kinds of events

An exception is raised by the CPU while executing an instruction, such as divide-by-zero or a page fault. A hardware interrupt is raised by a device, such as the keyboard or timer. A software interrupt is an explicit instruction; this kernel uses `int 0x80` for system calls. All arrive through the Interrupt Descriptor Table (IDT).

5.2 The IDT and assembly stubs

`kernel/interrupts.c` builds 256 IDT entries and loads the IDT register. Each gate names a handler address, a code selector, and flags. The assembly stubs in `kernel/interrupts.asm` save registers, arrange a `registers_t` frame, call a C handler, restore state, and finish with `iret`. The exact stack layout must agree with `include/kernel/interrupts.h`; this is an ABI between assembly and C.

5.3 PIC remapping and acknowledgement

Old PC hardware routes IRQs through the Programmable Interrupt Controller. IRQ 0 and IRQ 1 would otherwise overlap CPU exception vectors, so `pic_remap(32, 40)` moves them to IDT vectors 32–47. After an IRQ is handled, `pic_send_eoi()` sends End Of Interrupt. Without it, the PIC may refuse to deliver further interrupts.

5.4 Faults are diagnostics

The page-fault handler reads `CR2`, which contains the address that caused the fault, and decodes the error code. This is a model debugging feature: report the faulting address, access type, privilege level, and instruction pointer before changing code blindly.

Checkpoint. Press a key in QEMU and trace the route: keyboard hardware, IRQ 1, remapped vector 33, `irq_handler`, then `keyboard_handler`.

Chapter 6

Timer and keyboard drivers

6.1 The timer creates time

`kernel/drivers/timer.c` programs the programmable interval timer and counts ticks in its IRQ handler. At 100 Hz, an interrupt arrives roughly every 10 milliseconds. The handler wakes processes whose `wake_tick` has passed and calls the scheduler. A timer is what turns a kernel from a straight-line program into a system that can revisit tasks.

6.2 Keyboard input is asynchronous

`kernel/drivers/keyboard.c` reads scan codes from I/O port `0x60`, translates selected codes to characters, and updates the process currently blocked on standard input. Enter marks the read complete; backspace changes both the display and the buffered input. This separation matters: a keystroke can arrive while unrelated kernel code is running.

6.3 Polling versus interrupts

Polling repeatedly asks a device whether it has work. Interrupt-driven input lets the CPU do other work and be notified later. This system still has simple shared state and no locking, which is acceptable for a small single-CPU teaching kernel but becomes unsafe as drivers and CPUs multiply.

Try it. Find the timer frequency argument in `timer_init(100)`. Change it to a noticeably lower value, run the system, and observe how this changes scheduler responsiveness. Restore 100 after the experiment.

Chapter 7

Physical memory: allocating 4 KiB frames

7.1 Physical and virtual are different questions

Physical memory is the actual RAM exposed by the machine. Virtual memory is the address space seen by code. The Physical Memory Manager (PMM) answers “which 4 KiB physical frame is free?” The Virtual Memory Manager (VMM) answers “which frame should this virtual address use?” Keeping those questions separate makes the design understandable.

`memory/pmm.c` uses a bitmap. One bit represents one 4096-byte frame. A set bit means reserved or used; a clear bit means available. The BIOS E820 map tells the PMM which ranges may be cleared. The bitmap itself, the loaded kernel, VGA area, and other critical ranges must remain marked used.

For a physical address P , its frame index is $P/4096$. For frame index F , its address is $F*4096$. `alloc_frame()` finds a clear bit, sets it, and returns the frame address. `free_frame()` reverses that operation.

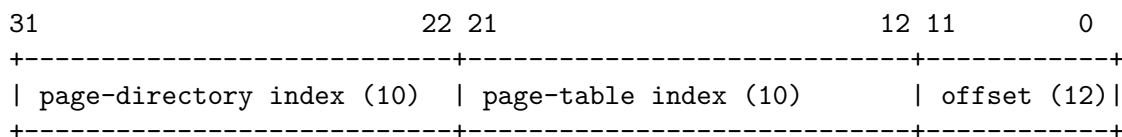
Checkpoint. Use `print_bitmap_summary()` temporarily after `init_pmm()`. Relate the printed free/reserved ranges to the BIOS map stored by the bootloader.

Chapter 8

Paging and the kernel heap

8.1 Two-level paging

In 32-bit x86 paging, a virtual address splits into a directory index, a table index, and a page offset:



`memory/vmm.c` creates a page directory and an identity-mapped first page table, then loads its address into CR3. Setting bit 31 (PG) in CR0 enables paging. Identity mapping means virtual address `x` initially refers to physical address `x`; it lets existing kernel code continue running during the transition.

`map_page()` extracts the two indexes, creates a table if needed, stores the physical frame and flags, and invalidates the single TLB entry when editing the active address space. The TLB is the processor's cache of recent translations; flushing it after a mapping edit prevents stale answers.

8.2 A simple kernel allocator

`memory/heap.c` maps an initial page at `HEAP_START` (0xC0000000) and keeps a linked list of block headers. `kmalloc` uses first fit, splits a large free block, and grows the heap by mapping more frames. `kfree` marks a block free and merges adjacent free blocks.

This is excellent learning code but not a production allocator. It is linear-time, has no alignment policy or corruption checks, and does not return frames to the PMM. Those limitations are opportunities for later work.

Try it. Draw the linked list after allocating 20 bytes, then 30 bytes, then freeing the first allocation. Explain why freeing only marks metadata; it does not erase the old payload.

Chapter 9

Processes and context switching

9.1 What a process owns here

A process is represented by `process_t` in `include/tasks/procman.h`. It stores a PID, state, saved registers, page directory, kernel stack, user stack information, and links to the next process. The possible states are running, ready, blocked, sleeping, and terminated.

Creating a kernel process gives it the shared kernel address space and a kernel code selector. Creating a user process also creates a new page directory, copies the kernel mappings, allocates user stack and heap pages, and uses ring-3 code/data selectors. The code itself is mapped later by the loader.

9.2 Saving a paused CPU

The CPU does not know the high-level idea of a process. A context switch makes that illusion by saving the interrupted process's registers into its `process_t`, selecting another process, switching CR3 if necessary, restoring the new registers, and returning from the interrupt frame. `tasks/context.c` performs the C side; the interrupt return path finishes the hand-off.

9.3 Round-robin scheduling

`tasks/scheduler.c` chooses the next ready process by walking the linked list in a circle. Every `DEFAULT_TIME_SLICE` timer ticks it requests a switch. This is round-robin scheduling: easy to understand and fair among ready processes, but unaware of priority, deadlines, I/O cost, or multicore concerns.

Checkpoint. In `kernel/kernel\_main.c`, find the demonstration `test_process`. Explain why the timer is needed before the main kernel can return to it.

Chapter 10

User mode and system calls

10.1 Privilege is a hardware rule

The processor distinguishes kernel privilege (ring 0) from user privilege (ring 3). User code should not run arbitrary I/O instructions or change page tables. When ring-3 code performs `int 0x80`, the IDT gate transfers execution to the kernel. The Task State Segment (TSS) supplies the kernel stack pointer the CPU must use while entering from user mode.

10.2 The syscall boundary

The user wrapper in `user/libc/syscalls.asm` follows a small register convention: `EAX` is the call number; `EBX`, `ECX`, and `EDX` are arguments; `EAX` returns the result. `kernel/interrupts.c` dispatches that number in `syscall_handler`. Both header files define the same numeric constants, so changes must be made consistently.

```
mov eax, SYSCALL_WRITE
mov ebx, 1           ; standard output
mov ecx, buffer
mov edx, count
int 0x80
```

The implemented calls cover `exit`, `PID`, `yield`, `sleep`, `read/write`, heap growth through `sbrk`, filesystem operations, `open`, and `close`. Standard input blocks the current process until keyboard input completes. The kernel wakes the parent process when a child exits.

10.3 The tiny user library

`user/libc/` supplies enough C facilities for programs: strings, character tests, formatting, memory allocation, and syscall wrappers. The user linker script fixes program code at `0x00400000`; `tasks/loader.c` maps file bytes there page by page.

Try it. Add a syscall only on paper first. Choose its number, user prototype, register arguments, kernel validation, return convention, and failure value. This design work prevents mismatched interfaces.

Chapter 11

Disk I/O and the filesystem

11.1 ATA sectors

The ATA driver in `kernel/drivers/ata.c` reads and writes 512-byte sectors using legacy I/O ports. The driver waits for the device status to indicate readiness, chooses a sector, commands a read or write, and transfers words through the data port. The rest of the filesystem deliberately treats its block size as one sector, so one filesystem block is 512 bytes.

11.2 On-disk layout

The filesystem layout types and constants are in `include/filesystem/fs\_layout.h`. Its core structures are a superblock, allocation bitmap, inode table, data blocks, and fixed-size directory entries. The superblock records where each region begins. An inode records type, byte size, and a small array of data-block numbers. A directory is a file whose data blocks contain name-to-inode records.

```
filesystem region:  superblock | bitmap | inode table | data blocks
file lookup:        /bin/shell -> root entry -> bin inode -> shell inode -> data blocks
```

`filesystem/fs.c` implements the low-level rules: format the volume, allocate/free blocks and inodes, read/write inodes, add directory entries, and read/write file bytes. `filesystem/fs\_manager.c` is the convenience layer: resolve a slash-separated path, maintain open-file records and offsets, and expose operations such as `fs_touch`, `fs_mkdir`, `fs_rm`, `fs_read`, and `fs_write`.

11.3 Host tool versus kernel code

`mkfs` is a normal host program. It opens `build/kernel.img`, formats the same layout, then walks `rootfs/` and copies host files into the image. It shares layout definitions with the kernel so both sides agree on the disk format. It must not share kernel driver code because the host already has file APIs.

Checkpoint. Run the shell and use `ls /bin`. Then relate the visible `shell` file to the Makefile rule that copies its flat binary into `rootfs/bin/` before `mkfs` runs.

Chapter 12

Programs and the shell

12.1 Loading a flat user binary

`load_program(path)` opens a file, reads its bytes, creates a user process, allocates physical frames, copies file chunks into them, and maps them at `USER_CODE_BASE`. The user linker script uses exactly that base. This loader handles a simple flat binary; it does not parse ELF headers, perform relocations, or load shared libraries.

12.2 Reading the shell as a complete vertical slice

`user/programs/shell.c` is a REPL: print a prompt, read a line, tokenize it, choose a command, invoke library functions, and repeat. It reaches almost every operating-system layer:

Shell action	Path through the system
<code>ls /</code>	tokenizer → <code>fs_ops</code> syscall → kernel path resolver → directory blocks → VGA output.
<code>cat /notes.txt</code>	<code>open</code> → file descriptor → repeated <code>read</code> syscalls → <code>write</code> to standard output.
<code>run /bin/user_test</code>	filesystem operation → loader → new user process → parent shell blocks until child exit.

When learning a large code base, choose a vertical slice like this instead of reading folders in isolation. You will see why each boundary exists.

Try it. Create a small user program modeled on `user/programs/test.c`. Add a Makefile rule, copy the binary into `rootfs/bin/`, rebuild, and run it from the shell. Start with one `printf`.

Part I

Detailed guided study

Chapter 13

Before changing code: the machine model

This chapter slows down and establishes the vocabulary that makes the rest of the source readable. A CPU is not a program that understands C functions or files. It repeatedly fetches bytes from the address named by its instruction pointer, decodes one instruction, executes it, and advances to the next instruction. A compiler translates C into those bytes; a linker chooses where groups of bytes will live and connects references between them.

13.1 Bits, bytes, words, and hexadecimal

A bit is either zero or one. Eight bits make a byte. The system uses 32-bit registers for most kernel work, so a natural machine word is four bytes. Hexadecimal is useful because one hex digit represents exactly four bits. For example, `0xB8000` is an address, not a magic string: it identifies the VGA text buffer. The suffix `h` used in NASM, such as `07C0h`, means the same thing.

Do calculations explicitly while learning. A 4 KiB page is $4096 = 2^{12}$ bytes, so its address has a 12-bit offset. Dividing an address by 4096 finds its page number; masking with `0xFFF` finds the byte offset within that page. When a function turns a page index back into an address, it multiplies by 4096. These are not arbitrary tricks: powers of two let the CPU use shifts and masks.

13.2 Registers and the stack

Registers are tiny storage locations inside the CPU. `EAX`, `EBX`, `ECX`, and `EDX` are general-purpose registers. `EIP` names the next instruction. `ESP` names the top of the current stack. `EBP` is commonly used as a stable reference inside a function. `CR0`, `CR2`, and `CR3` are control registers used for protected mode and paging.

The stack is a region of memory used for return addresses, parameters, saved registers, and local variables. On 32-bit x86 it grows toward lower addresses. `push eax` subtracts four from `ESP` and stores `EAX`; `pop eax` loads from `ESP` and adds four. A `call` also pushes a return address. A `ret` pops that address into `EIP`. If code corrupts the stack, returning from a function can jump into arbitrary bytes.

The C compiler and assembly agree on a calling convention. In this project, small assembly helpers receive their first C argument at `[esp + 4]` when entered. See `memory/vmm.asm`: `set_cr3` reads its argument from there, writes it to `CR3`, and returns. The system-call trampoline uses the same stack positions to move four C arguments into registers before executing `int 0x80`.

13.3 Three address meanings

At different points in boot, a number can mean three different things.

1. A *file offset* is a position inside the disk image. The boot sector is at offset zero.
2. A *physical address* names real RAM supplied by QEMU. BIOS loads the boot sector near `0x7C00`; this project loads the kernel at `0x9000`.
3. A *virtual address* is the address a running instruction uses after paging is enabled. It must be translated through page tables before RAM is reached.

At startup, the kernel identity maps low memory, so physical and virtual addresses happen to have the same numeric value. Do not confuse equality in this early mapping with identity of the concepts. A user program at virtual `0x00400000` may be stored in any allocated physical frame.

13.4 The C runtime you do not get

Freestanding kernel C differs from a normal hosted C application. There is no operating system to call before yours starts, no default `main`, no standard I/O, and no automatic C library. This is why the build uses `-ffreestanding`, why the entry assembly calls `kernel_main` directly, and why the repository implements its own text output and memory routines. Compiler-generated requirements still exist: C code needs a valid stack, correctly linked symbols, and sensible ABI rules.

Try it. In a notebook, calculate the physical address of `0x0900:0x0000` in real mode. Then calculate the offset of `0x00401234` within its 4 KiB page. Check both answers before continuing.

Chapter 14

A precise boot-to-C walkthrough

This chapter follows the actual first instructions in order. Keep `boot/bootloader.asm`, `kernel/kernel\_intro` and `kernel/linker.ld` open beside it.

14.1 Stage 1: BIOS gives the boot sector control

BIOS reads exactly one sector before it knows anything about your design. The source ends with `times 510-($-$$) db 0` and `dw 0xAA55`. The first directive fills unused space; the second puts the recognizable signature at bytes 510 and 511. The code begins by placing `0x07C0` in `DS`. Since $0x07C0 * 16 = 0x7C00$, labels in this segment refer to the sector BIOS loaded.

`print_string` demonstrates a common assembly loop. `SI` points at a zero-terminated byte string. `lodsb` loads the next byte into `AL` and advances `SI`. A zero ends the loop; otherwise BIOS service `int 0x10` function `0x0E` prints the character. The routine returns to its caller with `ret`. At this point BIOS provides the output service; after protected mode, this interrupt interface is no longer the system's normal display API.

14.2 Stage 2: preserve information about RAM

`store_memory_map` invokes BIOS `E820` repeatedly. `EBX` is a continuation value: zero starts the query, and BIOS changes it until it becomes zero at the last entry. Each returned record is 24 bytes. The bootloader puts records beginning at `0x5000` and writes the number of records at `0x4FFC`. It temporarily sets `DS` to zero so these absolute low-memory addresses are not shifted by the bootloader segment.

This code assumes those low-memory locations are safe during this small boot sequence. A real boot protocol would define a fuller hand-off structure and carefully reserve it. The educational value is that memory discovery happens before a physical allocator exists; firmware is the only source of this information.

14.3 Stage 3: read the kernel

`load_kernel_from_disk` asks BIOS disk service `0x13` to read 60 sectors. It sets `ES:BX` to `0x0900:0`, which is physical `0x9000`. It starts at CHS sector 2 because sector 1 contains the bootloader. The exact CHS values assume a simple QEMU geometry and a kernel that fits in the fixed read. The carry flag reports failure; the code branches to an error message if it is set.

The limitation is important. The image eventually contains more than a kernel: it contains a filesystem beginning at a later sector. The bootloader does not understand it. It merely copies a fixed early region into RAM. Enlarging the kernel beyond 60 sectors can overwrite assumptions

or leave part of the kernel on disk. Measure the size of `build/kernel.bin` before experimenting with this constant.

14.4 Stage 4: the second assembly entry

The far jump enters `kernel/kernel\_intro.asm`. The source is still 16-bit at its beginning. It again establishes a data segment, displays a last BIOS message, and then disables maskable interrupts with `cli`. Interrupts must remain off until an IDT exists; an interrupt at the wrong moment would use uninitialized descriptors.

The GDT contains a null descriptor, kernel code/data descriptors, user code/data descriptors, and space for a TSS descriptor. `lgdt` loads the address and size of this table. Setting PE in CR0 enables protected-mode semantics, but the far jump makes the processor reload CS from the new GDT. At `p_mode_main`, the assembler is told `[bits 32]`; the code loads data selectors, assigns `ESP = kernel_stack_top`, writes a proof character to VGA memory, and calls `kernel_main`.

14.5 Linking is part of booting

The kernel linker script starts sections at 0x9000. It defines `_kernel_start` and `_kernel_end`, then places `.text`, `.data`, and `.bss`. The binary produced by `objcopy -O binary` has no ELF headers; it is raw bytes intended to be copied exactly where the linker expected them. That is why a mismatch here normally produces a crash long before a useful error message.

Checkpoint. Use `objdump -h build/kernel.elf` after a build. Locate the virtual memory address of `.text`; it should agree with the link base. Then use `stat -c%s build/kernel.bin` and divide by 512 to check the fixed sector budget.

Chapter 15

Descriptor tables and interrupt frames in depth

15.1 GDT selectors and privilege levels

A selector is not a raw address. It indexes a descriptor table and includes privilege bits. The project uses kernel code selector `0x08` and kernel data selector `0x10`; user code and data selectors are based on `0x18` and `0x20`, with the low two bits set to 3 for ring 3. The descriptor's access byte determines whether code can execute or data can be written. The descriptors are flat, so base zero plus a large limit covers the ordinary address range.

The TSS descriptor is populated later by `init_tss`. The CPU does not switch tasks through this TSS; the kernel switches tasks in software. It uses the TSS for one critical job: when an interrupt transfers from ring 3 to ring 0, the CPU needs a trusted kernel stack. `tss.esp0` is updated when a user process becomes current.

15.2 What the CPU puts on a stack

When an interrupt or exception occurs, the processor saves enough state to resume: at least EIP, CS, and EFLAGS; an exception with an error code adds one. A privilege transition also saves the old user ESP and SS after switching to the kernel stack. The assembly stubs save general registers and a data segment value around these CPU-provided fields. The resulting memory order is represented by `registers_t`.

This explains why changing that C structure without changing assembly is dangerous. A handler reading `regs->eax` is reading a fixed stack offset. If the two sides disagree, it may mistake a return address for an interrupt number. Always treat assembly and its matching C structure as one unit.

15.3 Exceptions, IRQs, and system calls use different policies

`isr_handler` handles CPU exceptions. It prints a diagnosis; for a fault from user mode it terminates that process and chooses another. A kernel-mode fault enters an infinite loop because the kernel cannot safely continue after corrupted privileged state. `irq_handler` dispatches hardware events and then acknowledges the PIC. `syscall_handler` interprets a deliberate request from user code and returns a result in the saved EAX field.

The distinction is a design tool. An exception means an instruction could not complete. An IRQ means an external device needs service. A syscall means code intentionally requested a defined kernel service. Combining them into one opaque “interrupt” idea makes error handling confusing.

15.4 Page faults as a worked example

Vector 14 is a page fault. The CPU writes the virtual address it tried to access into **CR2**; it also pushes an error code. `memory/vmm.c` tests the code's present, write, user, reserved, and instruction-fetch bits. A non-present read can indicate an unmapped page; a present write can mean a protection violation. The instruction pointer identifies the code that made the access, while **CR2** identifies the address it wanted.

Try it. Temporarily access an unmapped address from kernel code only after you have verified screen output. Record the fault address and EIP. Do not use this test with an address that changes disk data or mapped allocator metadata.

Chapter 16

Device drivers as state machines

16.1 Port I/O is not memory I/O

The instructions `in` and `out` communicate with I/O ports, a separate x86 address space. Assembly helpers in `kernel/interrupts.asm` make these instructions callable from C: `outb`, `inb`, `outw`, `inw`, and `io_wait`. A port number such as `0x60` does not point into normal RAM. The value of `volatile` in VGA code has a similar motivation: device interactions must happen in the requested order.

16.2 PIT timer calculation

The PIT base oscillator is 1,193,182 Hz. To request a frequency f , `timer_init` calculates divisor $d = 1193182/f$, writes a command byte to port `0x43`, then writes low and high divisor bytes to channel 0 at `0x40`. The timer IRQ increments a tick counter. `timer_wait_ms` converts milliseconds into a target tick count and waits until it is reached.

This is approximate time. Integer division rounds the divisor, scheduling costs time, and the implementation does not calibrate against a real clock. It is sufficient to explain periodic interrupts and sleeping processes. Do not use it for precise timing claims.

16.3 Keyboard scan codes and lines of input

A PS/2 keyboard sends scan codes, not ASCII characters. The driver reads one code at a time from port `0x60`. It recognizes release codes, shift, extended prefixes, and a small keymap. Translation is a policy layer: the hardware describes keys; the OS decides what character or action they mean.

If `current_process` is blocked in a standard-input read, the keyboard handler appends characters to that process's `reading_state.buffer`. On Enter it completes the read and marks the process ready. This exposes an important boundary: code requesting input does not spin in a loop; it changes process state and lets another process run. The driver later causes progress.

The current implementation is intentionally narrow. It has no keyboard layout configuration, no queues for multiple readers, no modifiers beyond its small mapping, and no terminal discipline. These are improvements, not evidence that the basic design is wrong.

16.4 ATA PIO disk operations

The ATA driver uses programmed I/O (PIO): the CPU waits for device status, then transfers sector words itself. `ata_read_sector` selects an LBA sector through ports `0x1F2`–`0x1F6`, issues `ATA_CMD_READ_PIO`, waits for DRQ, and reads 256 16-bit words from `ATA_DATA`. A write mirrors

the sequence and flushes the disk cache. Status flags distinguish busy, data request, device fault, and error.

This is synchronous I/O. While waiting, the kernel does not provide a nonblocking disk completion mechanism. The simple filesystem can tolerate that, but a faster system would use interrupt-driven I/O, request queues, and recovery policies.

Checkpoint. Find every place that uses `outb` or `inb`. For each port, write down whether it belongs to PIC, PIT, keyboard, or ATA. This turns apparently mysterious numbers into a hardware map.

Chapter 17

Memory management laboratory

17.1 Read the PMM initialization from end to end

The PMM starts with all bitmap bits reserved. It reads E820 records placed by the bootloader and clears bits only for usable ranges. It then re-reserves areas the kernel depends on: low memory, the loaded kernel, VGA memory, and the bitmap location. This conservative order is safer than assuming all memory is free and trying to remember every exclusion later.

The bitmap is placed at `BITMAP_BASE`, `0x100000`. That choice itself requires care: this memory must be reserved before the allocator can hand it out. `alloc_frame` scans bits linearly. It is simple but has a cost proportional to the number of frames examined. On a small QEMU configuration that is acceptable; on a large machine it would be expensive.

17.2 Page flags and protection

Each page table entry contains a page-aligned physical frame address plus flags. `PAGE_PRESENT` says translation exists. `PAGE_WRITABLE` allows writes. `PAGE_USER` permits ring-3 access. An address without `PAGE_USER` may still be mapped for kernel use but fault when user code touches it. That one bit is the beginning of isolation between applications and the kernel.

`create_uprocess` copies the kernel directory entries into a fresh user directory. It maps a user stack below `USER_STACK_TOP` and an initial user heap page at `USER_HEAP_START`. The loader maps program code at `USER_CODE_BASE`. This kernel's copied kernel mappings make interrupt handling possible while a user address space is active, but a mature design would carefully restrict which kernel pages are exposed and set permissions precisely.

17.3 Walk one translation manually

For virtual address `0x00401234`, the directory index is bits 31–22, the table index is bits 21–12, and the offset is `0x234`. In C, `map_page` computes these using shifts and masks. The directory entry locates a page table; the selected table entry locates a physical frame. Add the offset to the frame base to get the physical byte. Use this exercise whenever page-table code looks abstract.

17.4 Heap metadata and fragmentation

The kernel heap begins with one free block. A block header lives immediately before its usable data and contains size, free flag, and a link. If `kmalloc(24)` chooses a much larger free block, `split_block` creates a second header after 24 payload bytes. `kfree` obtains the header by

subtracting `sizeof(heap_header_t)` from the user pointer. Adjacent free blocks merge only in the forward direction from the block being freed.

Fragmentation occurs when free space is divided into pieces too small for a later request even if their total size is large. First-fit allocation is easy to explain but can amplify this effect. A useful experiment alternates differently sized allocations, frees every other one, and tries one large allocation. Never run such a test in an unbounded loop: this teaching heap has no protection against exhaustion.

Try it. Add a debug function that walks the heap list and prints each header's address, payload size, and free flag. Call it after a controlled series of allocations. Remove it before progressing to unrelated debugging, because verbose output in low-level code can hide timing problems.

Chapter 18

Virtual memory from zero: a slow, visual explanation

This is the chapter to reread until it feels ordinary. Virtual memory often seems difficult because one word, “address,” is used for several jobs. We will use separate names and move only one step at a time. Nothing in this chapter relies on magic. It is a lookup table built from very regular pieces.

18.1 The child-friendly picture: street names and houses

Imagine that every program writes letters using street names. A program may say, “put this letter at 0x00400000.” That is a *virtual address*: a name inside the program’s own map. RAM contains the actual houses, which have *physical addresses*. The operating system owns a large map that tells the CPU which virtual street leads to which physical house.

Two children can both use the street name “0x00400000” in their own drawings. They do not collide if their maps point to different physical houses. A child cannot visit the castle (the kernel) merely by writing its name if the map marks the road private. This is the purpose of virtual memory: convenient names, separation, and controlled access.

The CPU performs the lookup automatically for every instruction fetch, read, and write after paging is enabled. The kernel does not translate every byte by hand. It builds the map once, tells the CPU where the map begins, and the CPU uses it at speed.

18.2 Three labels for one byte

Suppose a user program executes this C statement:

```
buffer[0] = 'A';
```

The compiler produces an instruction containing or computing a virtual address. It may be 0x00A00000, the beginning of the user heap in this project. The CPU consults the current process’s page tables and might find that it corresponds to physical address 0x00345000. It stores the byte at that physical RAM location.

Write the relationship like this:

```
program says:           virtual 0x00A00000
page tables translate:   -> physical 0x00345000
RAM receives:           byte 'A' at 0x00345000
```

The virtual address is meaningful only together with a page directory. If the scheduler switches to another process and loads another directory into CR3, the same virtual number can

lead somewhere else. This is why a pointer saved from one process cannot automatically be trusted as a pointer in another process.

18.3 Why pages are 4096 bytes

The x86 paging mode used here divides memory into fixed-size pieces called pages. One page is 4096 bytes, also written 4 KiB or 0x1000. Instead of mapping every individual byte, the kernel maps a whole page at once. The first byte in a page has offset 0; the last has offset 4095 (0xFFF).

Think of an apartment building with exactly 4096 rooms. The page number tells you which building; the offset tells you which room in that building. If the virtual address is 0x00401234, then the last three hexadecimal digits, 234, are the offset. The page begins at 0x00401000. The same offset 0x234 is used after translation, so a mapping that chooses physical page 0x00123000 reaches physical byte 0x00123234.

18.4 One big address split into three pieces

An address in 32-bit paging is 32 bits wide. The low 12 bits are the page offset because $2^{12} = 4096$. The remaining 20 bits choose the mapping. x86 divides those 20 bits into two groups of 10:

Directory index 10 bits	Table index 10 bits	Offset 12 bits
bits 31–22	bits 21–12	bits 11–0

A page directory has 1024 entries because $2^{10} = 1024$. Each page table also has 1024 entries. One table therefore describes $1024 * 4096$ bytes, or 4 MiB. One directory can choose 1024 tables, describing 4 GiB in total. The hierarchy prevents the kernel from allocating a giant flat mapping array when most of the address space is unused.

The source declares these exact shapes:

```
typedef uint32_t page_directory_t[1024];
typedef uint32_t page_table_t[1024];
```

Each entry is four bytes, so a complete directory or table is 4096 bytes: exactly one physical frame. This pleasing fit is not an accident. It is why `alloc_frame()` is a natural way to allocate either one.

18.5 Do a complete translation with the project's formulas

We will translate virtual address 0x00401234. Use the formulas in `memory/vmm.c`:

```
directory_index = virtual_address >> 22
table_index     = (virtual_address >> 12) & 0x3FF
offset         = virtual_address & 0xFFF
```

For this address, the directory index is 1, because 0x00400000 is 4 MiB and each directory entry covers 4 MiB. The table index is 1, because the address is one 4 KiB page past 0x00400000. The offset is 0x234.

Now imagine directory entry 1 contains 0x001AB007. The low bits are flags; masking with 0xFFFFF000 leaves table physical address 0x001AB000. Entry 1 in that table might contain 0x00345007. Masking gives physical page 0x00345000. Finally add the untouched offset: physical byte 0x00345234.

The lookup is therefore not “search through all memory.” It is two indexed array reads and one addition. The CPU has hardware designed to perform this quickly.

18.6 What a page-table entry stores

An entry is a 32-bit number. Its high bits hold a page-aligned physical address because every frame begins at a multiple of 0x1000. Its low bits hold rules. This project defines three rules:

Name	Bit	Meaning
PAGE_PRESENT	0	The CPU may use this entry. If clear, an access causes a page fault.
PAGE_WRITABLE	1	Writes are allowed. If clear, a write is a protection fault.
PAGE_USER	2	Ring-3 code may access this mapping. If clear, it is kernel-only.

For instance, 0x00345007 means physical page 0x00345000 plus flags 1, 2, and 4: present, writable, and user-accessible. The code combines these with bitwise OR:

```
entry = physical_address | PAGE_PRESENT | PAGE_WRITABLE | PAGE_USER;
```

The physical address must already be page-aligned; otherwise its low address bits would collide with flag bits. `alloc_frame()` returns aligned addresses for this reason.

18.7 Read `init_vmm` like a recipe

Open `memory/vmm.c`. `init_vmm()` first asks the PMM for one frame for `kernel_directory` and one for `tbl_ptr`. It clears both frames with `memset`. A cleared table means every mapping starts absent; this is a safe empty map.

Next, a loop fills all 1024 entries of the first table:

```
(*tbl_ptr)[i] = (i * 0x1000) | PAGE_PRESENT | PAGE_WRITABLE;
```

For `i = 0`, virtual page 0 maps physical page 0. For `i = 1`, virtual 0x1000 maps physical 0x1000. For the final index, virtual 0x003FFF000 maps the same physical page. This is called an *identity map*. It covers the first 4 MiB.

Then directory entry 0 is set to point at that first table. Entry zero is chosen because virtual addresses from 0x00000000 through 0x003FFFFFFF have directory index zero. Finally, the code sets `current_directory`, loads its address into `CR3`, and sets the PG bit in `CR0`. From that instruction onward, the CPU translates addresses. The currently executing kernel is safe because it was loaded below 4 MiB and its old addresses still refer to the same RAM through the identity map.

18.8 Read `map_page` one line at a time

`map_page(directory, virtual, physical, flags)` is the central operation. It does five jobs.

1. It calculates directory and table indexes from the virtual address.
2. It looks in the directory for the selected table.
3. If no present table exists, it allocates and clears one with `create_page_table`, then records it in the directory.
4. It writes the physical frame plus flags into the selected table entry.
5. If this directory is currently active, it calls `invlpg` for that virtual page.

The third step is easy to miss. Mapping address `0xC0000000`, for example, has directory index 768. The first identity table is directory index 0, so it cannot describe that address. `map_page` creates a new table for directory entry 768 automatically. This is why the heap initialization can request a mapping at `HEAP_START` without first manually constructing a high-address table.

18.9 Why the TLB exists, and why it must be invalidated

Page walking would require two memory reads before every ordinary memory access. The Translation Lookaside Buffer (TLB) is a small CPU cache remembering recent virtual-page to physical-page answers. It makes virtual memory fast in normal use.

The cache introduces one rule: after changing a page table, the old answer may still be cached. `flush_tlb_page` in `memory/vmm.asm` executes `invlpg [eax]`, which invalidates the cache entry for one virtual page. `map_page` uses it only if it modified `current_directory`; an inactive future process has no active TLB entry on this CPU yet. `flush_tlb` reloads CR3 to invalidate the entire cache, which is simpler but more expensive.

18.10 The kernel heap is a useful real mapping

`init_heap` requests a physical frame, maps it at virtual `0xC0000000`, and places its first allocation header at that virtual address. Notice the two different values: the PMM chooses the physical frame, while the VMM gives it a stable high virtual name. The heap allocator never needs to know which RAM frame backs its blocks. When the heap grows, `expand_heap` repeats the pattern for `heap_end`, then increments `heap_end` by one page.

This is an important division of responsibility. The heap allocator manages variable-size pieces inside mapped virtual pages. The VMM manages whole-page mappings. The PMM manages whole physical frames. If you mix the layers, bugs become very hard to reason about.

18.11 User address spaces in this exact system

When `create_uprocess` runs, it allocates a fresh page directory and clears it. It copies every entry from `kernel_directory`. Therefore the user directory begins with the kernel's existing mappings. It then maps one user stack page at `USER_STACK_TOP - 4096` and one user heap page at `USER_HEAP_START`; both have present, writable, and user flags.

The loader later maps program pages at `USER_CODE_BASE`, `0x00400000`, also with user and writable flags. The program's link script starts its sections at that same address. This three-part agreement is essential:

```
user/user.ld           expects code at 0x00400000
include/kernel/mappings.h names USER_CODE_BASE as 0x00400000
tasks/loader.c         maps file pages at USER_CODE_BASE
```

The code defines `USER_VGA`, but the current process setup does not map it. Do not assume that a named constant proves a feature exists. Search for calls to `map_page` to discover what the final implementation actually maps.

18.12 What happens during a page fault

If code touches a virtual page without a usable entry, the CPU stops the attempted instruction and enters IDT vector 14. It saves state, puts the troublesome virtual address in CR2, and supplies error bits. `page_fault_handler` prints the address and decodes the reason. It

does not allocate missing pages on demand; it is a diagnostic handler. After a user-originated fault, `isr_handler` terminates the user process. After a kernel fault, it loops forever because continuing could damage the system.

The phrase “page fault” does not always mean a software bug. Modern operating systems intentionally leave pages absent and use faults to load files or allocate memory lazily. In this project, it normally means that a mapping or permission rule is wrong, so it is a very useful error report.

18.13 Common beginner confusions, answered carefully

“Is a page a physical thing or a virtual thing?” Both names are used. A *virtual page* is a 4 KiB range in an address space; a *physical frame* is a 4 KiB range in RAM. This book uses “frame” for physical and “page” for virtual whenever possible.

“Does creating a process copy all program bytes?” No. `create_uprocess` creates its bookkeeping, page directory, initial stack, and heap. `load_program` reads a specific file and maps code frames. A separate process can therefore exist before a file is loaded, depending on the design.

“Why does the kernel use virtual addresses if it has all privileges?” Paging gives the kernel flexible placement, controlled mappings, and the same useful page-granularity tools. The current kernel starts with a simple identity map for clarity, then maps its heap high at `0xC0000000`.

“Why can the code cast a page-table address to a pointer?” Early kernel page tables are allocated from identity-mapped physical frames. While that identity mapping exists, the physical frame number can also be used as a kernel virtual address. A higher-half kernel would need a deliberate conversion or mapping instead.

“Does `get_physical_address` return the complete byte address?” In the present source it masks and returns the page frame base. It does not add the original virtual offset. That is adequate for its current use only if callers expect a page base. A general translation helper should add `virtual_address & 0xFFF`.

18.14 A safe virtual-memory practice sequence

Do these steps in order, rebuilding after each one.

1. Add a message immediately before and immediately after `init_vmm`. If the second never appears, do not proceed to heap work.
2. Print the addresses returned by the first two `alloc_frame` calls used for directory and table. Confirm both end in three hexadecimal zeros.
3. In a temporary function, call `map_page(kernel_directory, 0xC0100000, frame, PAGE_PRESENT | PAGE_WRITABLE)` and write a recognizable value through `(uint32_t *)0xC0100000`. Do not choose an address used by existing mappings.
4. Read the same value through that virtual address. This confirms that allocation, mapping, TLB invalidation, and ordinary access all agree.
5. Call `unmap_page` for the test address and then deliberately read it only if you are prepared for the diagnostic page-fault screen.
6. Restore the experiment before working on processes. Keep the written notes; the code should return to a clean baseline.

Checkpoint. Explain this sentence in your own words: “CR3 chooses a map; the directory chooses a table; the table chooses a frame; the offset chooses a byte.” If any noun is unclear, repeat the matching section before moving on.

Chapter 19

Process, scheduler, and privilege-transition laboratory

19.1 Lifecycle of a kernel process

`init_procman` creates PID 1 for the running kernel context. `create_process` allocates a `process_t`, assigns a PID and state, allocates a kernel stack, sets an instruction pointer, then chooses `create_kprocess` or `create_uprocess`. New processes enter the linked list as ready. The scheduler never creates CPU state out of nowhere; it only selects a record that already contains a saved or initial register image.

For a new kernel process, EIP is the task function and ESP starts at the top of that process's kernel stack. Code/data selectors are ring 0. The initial EFLAGS value 0x202 sets the always-reserved bit and enables interrupts when returned to. The process shares `kernel_directory`.

19.2 Lifecycle of a user process

For a user process, code starts at `USER_CODE_BASE`; the loader makes it true by mapping file data at that virtual address. CS, DS, and SS receive ring-3 selectors. The process gets its own page directory and user stack. When the context layer chooses it, it sets `return_to_user`, updates `tss.esp0`, loads its CR3, and restores the saved interrupt frame so that the eventual `iret` crosses down to ring 3.

The TSS's `esp0` must point at the selected user's kernel stack. Otherwise the CPU would have nowhere trusted to save a user-to-kernel interrupt frame. This is one of the most common conceptual gaps in first operating-system projects: a user stack is untrusted and cannot be reused as the kernel's interrupt stack.

19.3 Scheduling states tell the story

The scheduler considers ready processes. A running process becomes ready during a switch unless some other code already marked it blocked, sleeping, or terminated. `SYSCALL_SLEEP` records a future tick and makes the caller sleeping. `SYSCALL_READ` on standard input makes it blocked. The timer and keyboard handlers later move them back to ready. `SYSCALL_EXIT` marks a child terminated and wakes a parent that is waiting for that PID.

This is a state machine. Draw arrows rather than memorizing cases:

READY -> RUNNING -> READY	(time slice / yield)
RUNNING -> SLEEPING -> READY	(sleep / timer deadline)
RUNNING -> BLOCKED -> READY	(stdin or child wait / event)
RUNNING -> TERMINATED	(exit or user fault)

19.4 What context switching does not solve

The scheduler is round robin over one linked list. It has no priority, idle task, locks, SMP support, signal handling, or robust resource cleanup. `destroy_process` also demonstrates why process teardown is harder than creation: every mapped page, file record, and parent relationship needs a clear owner. Treat this code as a working learning base; inspect ownership before adding a new resource to `process_t`.

Checkpoint. Set a breakpoint on `context_switch` in GDB and record old PID, new PID, old state, new state, and CR3 before/after. Repeating this for a timer switch and a syscall-driven sleep makes the state machine concrete.

Chapter 20

Filesystem and userspace laboratory

20.1 Calculate the on-disk structures

The filesystem starts at disk sector 70, leaving the bootloader and fixed kernel load area earlier in the image. A filesystem block is one 512-byte ATA sector. The superblock occupies filesystem block 0; the bitmap is block 1; inode storage begins at block 2. There are 128 inodes. Because each inode has a size, type, and ten 32-bit block numbers, calculate `sizeof(fs_inode_t)` with the host/compiler layout rather than guessing. The formatter computes how many 512-byte blocks the inode table requires, then names the first data block.

The root inode number is zero. A free inode has type zero. Directory entries contain an inode number and a fixed 32-character name. Inode zero cannot be used as an ordinary empty-entry marker in a root directory, so the implementation's conventions deserve close reading when modifying directory code. Fixed-size entries simplify scanning but impose filename and capacity limits.

20.2 Formatting creates invariants

`fs_format` writes a superblock, marks metadata blocks used in the bitmap, reserves a root data block, clears inode-table blocks, writes the root inode as a directory, and clears its first directory block. After formatting, every function assumes these invariants. For example, allocation expects metadata bits to be set; path resolution expects root inode to exist; a directory reader expects blocks named by its inode to contain entry records.

When a filesystem function fails halfway through several sector writes, it can leave metadata inconsistent. This teaching implementation has limited failure recovery. A production filesystem would use transactions, a journal, copy-on-write, or fsck-like repair. The small code is still useful because it shows why those advanced ideas are needed.

20.3 Follow an open/read/write request

`fs_open` resolves an absolute path, finds an unused record in the global `open_files` array, stores its inode and offset, then returns an external descriptor offset by `FS_FD_OFFSET`. `fs_read` converts that descriptor back to an array index, asks the filesystem core to copy bytes beginning at the stored offset, and increments the offset by the result. `fs_write` follows the same pattern.

This gives a concrete meaning to a file descriptor: it is not the file itself. It identifies a live record containing the current position. The current table is global, so two processes can interfere with descriptor allocation and offsets. Making it per-process is an excellent later extension.

20.4 From a host file to a running program

The Makefile compiles a user C file with freestanding flags, links it at 0x00400000, converts the ELF into a flat binary, and copies that binary into `rootfs/bin/`. `mkfs` traverses `rootfs` using the host's `opendir`, `readdir`, and `fread` APIs and writes matching entries into the raw image. At boot, `load_program` reverses the final step: open path, read bytes, allocate frames, copy chunks, map them at the link address, and create a user process.

The loader uses a variable-length stack buffer sized to the file. That is simple but potentially dangerous for a large binary because kernel stack space is limited. A stronger loader would stream one page at a time or allocate a controlled buffer. It also maps code writable and does not parse executable metadata; record those facts before treating it as a general program loader.

20.5 Shell commands are an API exercise

The shell expects exactly two words for every command in its present form. `ls`, `mkdir`, `touch`, `rm`, and `run` call `fs_ops`. `cat` uses `open`, a read loop, and `write`. `write` dynamically grows a user-space buffer, receives a line from standard input, writes it to an open descriptor, and closes it. Each command is small because its work is divided cleanly between parser, libc wrapper, syscall dispatcher, manager, and filesystem core.

Try it. Implement a `pwd` command only after deciding what data is necessary. The present filesystem has no parent inode or process working directory, so a true `pwd` requires a design change. A useful first version can explicitly report that paths are absolute.

Chapter 21

Build, test, and debug like a kernel developer

21.1 Make clean experiments

Use a short branch or copy for each experiment. Build from `lytlnybl/` with `make clean` followed by `make run` when you need to rule out stale objects. The output image is generated, so do not edit it by hand unless your task is specifically to inspect sectors. Keep a one-line change log: changed file, observation, and next hypothesis. Kernel debugging becomes much faster when you know exactly which assumption a test was meant to check.

21.2 Use staged evidence

The bootloader has BIOS text, the protected-mode entry writes a direct VGA character, and `kernel_main` prints through the driver. These are three independent checkpoints. Add similarly small markers around risky initialization. If a marker disappears after a change, the last visible marker is more useful than a large speculative patch.

For disk and filesystem work, inspect both levels: use shell commands for functional evidence and host tools such as `hexdump -C build/kernel.img` for raw evidence. For paging work, print CR2 and EIP on a fault. For scheduler work, print process IDs sparingly; excessive printing inside interrupt paths changes timing and can hide the bug.

21.3 GDB and QEMU

The Makefile contains a commented `-s -S`. With it enabled, QEMU opens a GDB server on port 1234 and stops before execution. In another terminal, start `i686-elf-gdb build/kernel.elf`, then use `target remote :1234`. Break at symbols such as `kernel_main`, `idt_init`, or `context_switch`. Use `info registers`, `x/16i $eip`, and `x/32wx $esp` to inspect state.

Be aware that the bootloader is a raw binary while the kernel ELF retains symbols. GDB can understand the latter naturally. To debug early boot, you need to set the architecture and load symbols at the correct address, or use a disassembly listing generated by NASM. Start with kernel C debugging; it yields more information with less setup.

21.4 A review checklist for every extension

1. Name the public interface: header, function signature, constants, and return errors.
2. Identify ownership: who allocates each resource and who frees it on success and failure.

3. Identify contexts: may the code run in interrupt context, user context, or only normal kernel context?
4. Identify address spaces: is each pointer physical, kernel virtual, or user virtual?
5. Add one observable success case and one observable failure case.
6. Rebuild from clean state and repeat the existing shell smoke test.

21.5 Capstone projects

Choose one project and write a two-page design before coding. Good choices are a serial logger, a per-process file descriptor table, append mode for files, a safer command parser, a page-table dump command, or a simple ELF loader. Each project teaches more when it respects existing interfaces and includes a test plan than when it adds a large amount of untested code.

Chapter 22

A practical learning path

Work through these stages in order. Do not begin with a filesystem change if you cannot yet tell whether the kernel reached C.

1. Build and run the untouched image. Record each startup message.
2. Change a boot greeting; verify it appears before protected mode.
3. Add a VGA line in `kernel_main`; verify the C hand-off.
4. Trace one timer tick and one keyboard character through the IDT.
5. Print PMM summary data, then allocate and free a frame in a controlled test.
6. Map one new kernel page and observe a deliberate page fault with a clear diagnostic.
7. Create two kernel processes that print distinct messages and watch round-robin progress.
8. Add a harmless user program that calls `get_pid()` and `write()`.
9. Create, write, list, and read a file through the shell.
10. Make one extension below and document its interface before coding it.

22.1 A debugging routine

First identify the latest guaranteed checkpoint. If boot text appears but C text does not, inspect the kernel load address, linker script, GDT, protected-mode jump, and stack. If the timer never fires, inspect IDT initialization, PIC remapping, and EOI. If a user program fails, verify its binary is in `rootfs`, its link base matches the loader mapping, and its page permissions include `PAGE_USER`. Use QEMU's GDB mode (remove the comment on `-s -S` in the Makefile) when screen messages cannot narrow the failure enough.

Chapter 23

What to improve next

The best next project is one that preserves an existing contract while strengthening it.

- Make the bootloader load a kernel larger than the current fixed 60 sectors, ideally from filesystem metadata.
- Validate user pointers before dereferencing syscall buffers in the kernel.
- Replace the flat loader with a minimal ELF loader and separate read-only code from writable data.
- Give each process its own open-file table instead of the current global table.
- Improve the allocator with alignment, error checks, and frame reclamation.
- Add nested directories fully, command parsing with arguments, and filesystem consistency checks.
- Add a serial debug driver so messages remain available when VGA code is suspect.
- Move toward higher-half kernel mappings only after you can explain every existing identity map.

Final perspective

You have now seen the minimum connected story behind an operating system: firmware gives control to a boot sector; the boot sector loads a kernel; the kernel establishes CPU rules; interrupts let devices and faults enter the kernel; memory managers allocate safe working space; processes preserve separate execution state; system calls cross the privilege boundary; and the filesystem makes data and programs persistent. Keep the source tree open, make one small observation at a time, and let each successful experiment become a new checkpoint.

Part II

Guided source reference

Appendix A

How to read this reference

The pages that follow contain the complete, build-relevant source of lytlnyblOS, organized in the same dependency order used by the tutorial. They are included so that this book can be used away from an editor and so that every explanation can be checked against the actual implementation. This is not filler: a small operating system is learned by repeatedly moving between a subsystem's contract, its code, and its observed behaviour.

Read each group in three passes. First read the header, if one exists; it tells you the names, structures, constants, and promises that other code may use. Second read the implementation from top to bottom, annotating every persistent global variable and every hardware-facing operation. Third use the tutorial chapter named in the group heading to trace one live path through the code. Do not attempt to understand every line on the first pass.

The listings reproduce the files as they are present in this repository when this manual was generated. The operating system source is the subject of the book. Generated binaries, build logs, and the old experimental `blog_playground/` snapshots are intentionally omitted.

Appendix B

Build contract and boot path

Start here when a change does not build or when QEMU never reaches the first C message. The Makefile defines exactly which files become guest code, which program runs on the host, and how disk-image sectors are arranged. The bootloader and linker script then form a strict address agreement.

B.1 ../lytlnybl/Makefile

Study focus: Follow every target from a source file to the final image. Mark the distinct host compiler and cross compiler paths, and identify every place where a new user program must be registered.

```
CC = i686-elf-gcc
HOST_CC = gcc
LD = ld
NASM = nasm
OBJCOPY = objcopy

CFLAGS = -g -m32 -Iinclude -ffreestanding
USER_CFLAGS = $(CFLAGS) -fno-pie -fno-pic

BUILD_DIR = build

KERNEL_ENTRY_ASM = kernel/kernel_intro.asm
KERNEL_ENTRY_OBJECT = $(BUILD_DIR)/$(KERNEL_ENTRY_ASM:.asm=.asm.o)

KERNEL_C_FILES = $(shell find kernel memory filesystem tasks -name '*.c')
KERNEL_ASM_FILES = $(shell find kernel memory tasks -name '*.asm')

KERNEL_C_OBJECTS = $(KERNEL_C_FILES:%.c=$(BUILD_DIR)/%.o)
KERNEL_ASM_OBJECTS = $(KERNEL_ASM_FILES:%.asm=$(BUILD_DIR)/%.asm.o)

# USER STUFFS

USER_C_OBJECTS = $(USER_C_FILES:%.c=$(BUILD_DIR)/%.o)
USER_ASM_OBJECTS = $(USER_ASM_FILES:%.asm=$(BUILD_DIR)/%.asm.o)

#need to build an archive for nice linking and file sizes
AR = i686-elf-ar

USER_LIB = $(BUILD_DIR)/libc.a

USER_LIB_C_FILES = $(shell find user/libc -name '*.c')
```

```

USER_LIB_C_OBJECTS = $(USER_LIB_C_FILES:%.c=$(BUILD_DIR)/%.o)

USER_LIB_ASM_FILES = $(shell find user/libc -name '*.asm')
USER_LIB_ASM_OBJECTS = $(USER_LIB_ASM_FILES:%.asm=$(BUILD_DIR)/%.asm.o)

USER_C_FILES = $(shell find user -name '*.c')
USER_ASM_FILES = $(shell find user -name '*.asm')

# ADD TO THIS WHEN ADDING PROGRAMS

USER_TEST_ELF = $(BUILD_DIR)/user_test.elf
USER_TEST_BIN = $(BUILD_DIR)/user_test.bin

SHELL_ELF = $(BUILD_DIR)/shell.elf
SHELL_BIN = $(BUILD_DIR)/shell.bin

USER_TEST_ROOTFS = $(ROOTFS_BIN_DIR)/user_test
SHELL_ROOTFS = $(ROOTFS_BIN_DIR)/shell

USER_BINARY_O = $(BUILD_DIR)/programs/test_binary.o

#END OF USER STUFF

BOOTLOADER = $(BUILD_DIR)/bootloader.bin

KERNEL_ELF = $(BUILD_DIR)/kernel.elf
KERNEL_BIN = $(BUILD_DIR)/kernel.bin
KERNEL_IMG = $(BUILD_DIR)/kernel.img

USER_LINKER = user/user.ld
KERNEL_LINKER = kernel/linker.ld

ROOTFS_DIR = rootfs
ROOTFS_BIN_DIR = $(ROOTFS_DIR)/bin

MKFS = $(BUILD_DIR)/mkfs

.PHONY: build clean

# ADD TO BUILD RULES WHEN ADDING USER PROGRAMS
build: $(KERNEL_IMG) $(USER_TEST_ROOTFS) $(SHELL_ROOTFS)

$(MKFS): tools/mkfs.c
    mkdir -p $(dir $@)
    $(HOST_CC) -g -Iinclude $< -o $@

$(BUILD_DIR)/%.o: %.c
    mkdir -p $(dir $@)
    $(CC) $(CFLAGS) -c $< -o $@

$(BUILD_DIR)/%.asm.o: %.asm
    mkdir -p $(dir $@)

```

```

$(NASM) -f elf32 -g -F dwarf $< -o $@

$(BOOTLOADER): boot/bootloader.asm
mkdir -p $(dir $@)
$(NASM) -f bin $< -o $@

# -----
# MAIN USER COMPILATION AND COPYING STUFF
# ADD NEW RULES FOR NEW PROGRAMS

$(USER_LIB): $(USER_LIB_C_OBJECTS) $(USER_LIB_ASM_OBJECTS)
mkdir -p $(dir $@)
$(AR) rcs $@ $^

$(USER_TEST_ELF): $(BUILD_DIR)/user/programs/test.o $(USER_LIB)
mkdir -p $(dir $@)
$(LD) -m elf_i386 -T $(USER_LINKER) $^ -o $@

$(USER_TEST_BIN): $(USER_TEST_ELF)
$(OBJCOPY) -O binary $< $@

$(SHELL_ELF): $(BUILD_DIR)/user/programs/shell.o $(USER_LIB)
mkdir -p $(dir $@)
$(LD) -m elf_i386 -T $(USER_LINKER) $^ -o $@

$(SHELL_BIN): $(SHELL_ELF)
$(OBJCOPY) -O binary $< $@

# -----
# REMEMBER TO COPY TO ROOTFS TOO

$(USER_TEST_ROOTFS): $(USER_TEST_BIN)
mkdir -p $(dir $@)
cp $< $@

$(SHELL_ROOTFS): $(SHELL_BIN)
mkdir -p $(dir $@)
cp $< $@

# -----

$(KERNEL_ELF): $(KERNEL_C_OBJECTS) $(KERNEL_ASM_OBJECTS)
mkdir -p $(dir $@)
$(LD) -m elf_i386 -T $(KERNEL_LINKER) \
    $(KERNEL_ENTRY_OBJECT) \
    $(KERNEL_C_OBJECTS) \
    $(filter-out $(KERNEL_ENTRY_OBJECT),$(KERNEL_ASM_OBJECTS)) \
    -o $@

$(KERNEL_BIN): $(KERNEL_ELF)
$(OBJCOPY) -O binary $< $@

# ADD HERE TOO, the ROOTFS
$(KERNEL_IMG): $(BOOTLOADER) $(KERNEL_BIN) $(MKFS) $(USER_TEST_ROOTFS) $(SHELL_ROOTFS)
dd if=/dev/zero of=$@ bs=512 count=20480
dd if=$(BOOTLOADER) of=$@ conv=notrunc
dd if=$(KERNEL_BIN) of=$@ seek=1 conv=notrunc
$(MKFS) $@

```

```
run: $(KERNEL_IMG)
     qemu-system-i386 -drive format=raw,file=$(KERNEL_IMG) #-s -S

clean:
     rm -rf $(BUILD_DIR)
```

B.2 ../lytlnybl/boot/bootloader.asm

Study focus: Trace DS setup, BIOS text output, E820 collection, CHS disk read, the carry-flag error path, and the final far jump. Count the boot-signature bytes.

```
start:
    mov ax, 07C0h
    mov ds, ax

    mov si, title_string
    call print_string

    mov si, message_string
    call print_string

    call store_memory_map

    call load_kernel_from_disk
    jmp 0900h:0000 ; jumps to physical address 0x9000, where the kernel was loaded

store_memory_map:
    xor ax, ax
    mov ds, ax ; in order to get exact addresses and not relative

    xor ebx, ebx ; first call
    xor bp, bp ; will store entry count

    mov di, memory_map_buffer ; safe memory location to write the map to
next_entry:
    mov ax, 0
    mov es, ax

    mov eax, 0xE820
    mov edx, 0x534D4150 ;signature that says we are requesting E820 memory map service
    mov ecx, 24

    int 15h

    jc done ; jump if there was an error

    add di, 24
    inc bp

    test ebx, ebx
    jnz next_entry
done:
    mov [memory_map_entries], bp
    mov ax, 07C0h
    mov ds, ax
    ret

load_kernel_from_disk:
    mov ax, 0900h
    mov es, ax

    mov ah, 02h ; service number,
    mov al, 60 ; number of sectors we want to read from (only simple kernel for now, so less than 512)
```

```

    mov ch, 0h ; number of track we would like to read from, is just 0.
    mov cl, 02h ; sector number that we would like to read its content, this is the second sector

    mov dh, 0h ; the type of disk we would like to read from, 0h means we are reading from a floppy
    mov dl, 80h ; this is the hard disk we are reading from, 80h means hard disk #0, 81h would be hard disk #1

    mov bx, 0h ; memory adress that content will be loaded into
    int 13h ; 13h provides services related to hard disk

    ; if successful, carry flag will be set to 0, otherwise carry flag is 1
    jc kernel_load_error

    ret

kernel_load_error:
    mov si, load_error_string
    call print_string

    jmp $

print_string:
    mov ah, 0Eh ; bios number 0Eh, sets for teletype output function
print_char:
    lodsb ; loads byte at SI, into AL and increments SI

    cmp al, 0 ; 0 stored in al if at end of string
    je printing_finished

    int 10h ;bios interrupt 0x10, to print char stored in AL

    jmp print_char
printing_finished:
    ;print new line
    mov al, 10d ; ASCII code for new line
    int 10h

    ;read current cursor position
    mov ah, 03h ; function to read cursor position
    mov bh, 0 ; page number 0 for default page
    int 10h ; 10h now used to read cursor position

    ;move cursor to beggining
    mov ah, 02h ; function to set cursor position
    mov dl, 0 ; column number (0 for begginign of line)
    int 10h ; 0x10 to set cursor pos

    ret

title_string db 'Welcome to the lytlnybl bootloader!',0
message_string db 'Loading up the kernel for you...',0
load_error_string db 'Oh oh!, there was a problem loading the kernel',0

memory_map_entries equ 0x4FFC
memory_map_buffer equ 0x5000

times 510-($-$$) db 0 ; pads the rest of the bootloader with 510 bytes, aiming for a 512 byte bootloader
dw 0xAA55 ; specifies the end of the bootloader, recognised by bios

```

B.3 ../lytlnybl/kernel/linker.ld

Study focus: Compare the link address with the bootloader load destination. Notice the order of text, data, and BSS.

```
ENTRY(start)
```

```
SECTIONS
```

```
{  
    . = 0x9000;  
  
    _kernel_start = .;  
  
    .text :  
    {  
        *(.text)  
    }  
  
    .data :  
    {  
        *(.data)  
    }  
  
    .bss :  
    {  
        *(.bss)  
    }  
  
    _kernel_end = .;  
}
```

B.4 ../lytlnybl/kernel/kernel_intro.asm

Study focus: Read the real-mode prelude and protected-mode transition as separate programs. Identify the GDT descriptors and every selector loaded after the far jump.

```
global kernel_stack_bottom
global kernel_stack_top
global gdt_tss

global tss
global tss_end

; no org code starts at 0x0900 though
[bits 16]
start:
    mov ax, cs
    mov ds, ax

    mov si, hello_string - start
    call print_string

    jmp enter_protected

print_string:
    mov ah, 0Eh

print_char:
    lodsb ; sets al = [DS:SI++]

    cmp al, 0
    je done

    int 10h

    jmp print_char

done:
    ret

enter_protected:
    cli ;disable interrupts
    lgdt [gdt_r - start] ; load GDT register with start address of GDT
    mov eax, cr0
    or eax, 1 ;set protection enable bit in control register 0 (cr0)
    mov cr0, eax

    ; perform far jump to selector 08h (offset into GDT, pointing at a 32bit
    ; PM code segment descriptor)
    ; to load CS with proper PM32 descriptor)

    CODE_SEG equ gdt_code - gdt_start
    jmp CODE_SEG:p_mode_main
[bits 32]
p_mode_main:
    mov ax, 10h
    mov ds, ax
    mov es, ax
```

```
    mov fs, ax
    mov gs, ax
    mov ss, ax
    mov esp, kernel_stack_top

    mov byte [0xB8000], 'P'
    mov byte [0xB8001], 0x02

    ; go into C

    extern kernel_main
    call kernel_main
hang:
    hlt
    jmp hang

hello_string db 'Hello World!, i am lytlnybIOS, in real mode', 0

gdt_start:
gdt_null:
    dq 0
gdt_code:
    dw 0xFFFF ; limit
    dw 0x0000 ; base_low
    db 0x00 ;base_middle
    db 0x9A ;access
    db 0xCF ;flags + limit high 4 bits
    db 0x00 ;base_high
gdt_data:
    dw 0xFFFF
    dw 0x0000
    db 0x00
    db 0x92
    db 0xCF
    db 0x00
gdt_user_code:
    dw 0xFFFF
    dw 0x0000
    db 0x00
    db 0xFA
    db 0xCF
    db 0x00
gdt_user_data:
    dw 0xFFFF
    dw 0x0000
    db 0x00
    db 0xF2
    db 0xCF
    db 0x00
gdt_tss: ; will be populated in C later
    dw 0
    dw 0
    db 0
    db 0
    db 0
    db 0
gdt_end:
gdtr:
```

```
    dw gdt_end - gdt_start - 1
    dd gdt_start

section .bss
align 16
kernel_stack_bottom:
    resb 0x4000
kernel_stack_top:
```

B.5 ../lytlnybl/kernel/kernel_main.c

Study focus: This is the initialization dependency graph. For each call, ask what earlier component it assumes and what later component relies on it.

```
#include "kernel/drivers/vga_text.h"
#include "kernel/interrupts.h"
#include "kernel/drivers/timer.h"
#include "kernel/drivers/keyboard.h"
#include "memory/pmm.h"
#include "memory/vmm.h"
#include "memory/heap.h"
#include "tasks/procman.h"
#include "filesystem/fs_manager.h"
#include "kernel/drivers/ata.h"
#include "tasks/loader.h"
#include "filesystem/fs.h"

#include <stdint.h>

vga_text terminal;

void test_process() {
    vga_text_writeline(&terminal, "NEW KERNEL PROCESS RUNNING");
    for (;;);
}

void kernel_main(void)
{
    volatile char* vga = (volatile char*)0xB8000;

    //signal that we have reached C
    vga[0] = 'C';
    vga[1] = 0x02;

    vga_text_init(&terminal);
    vga_text_writeline(&terminal, "Welcome to the lytlnyblOS!");

    idt_init();

    init_pmm();
    init_vmm();
    init_heap();
    init_procman();

    timer_init(100);
    keyboard_init();
    init_tss();
    fs_manager_init();

    vga_text_set_color(&terminal, VGA_COLOR_GREEN, VGA_COLOR_BLACK);
    vga_text_writeline(&terminal, "Kernel multiprocessing tests:");
    process_t* test_proc= create_process(test_process, PROCESS_KERNEL);
    timer_wait_ms(10);
    vga_text_writeline(&terminal, "BACK IN MAIN PROCESS");
    vga_text_set_color(&terminal, VGA_COLOR_WHITE, VGA_COLOR_RED);
```

```
vga_text_writeline(&terminal, "Memory allocation tests:");
vga_text_set_color(&terminal, VGA_COLOR_WHITE, VGA_COLOR_BLACK);
uint32_t* numbers = (uint32_t*)kmalloc(5 * sizeof(uint32_t));

for (int i = 0; i < 5; i++) {
    numbers[i] = (i + 1) * 10; // Stores 10, 20, 30, 40, 50
}

vga_text_write(&terminal, "Values: ");
for (int i = 0; i < 5; i++) {
    vga_text_write_dec(&terminal, numbers[i]);
    vga_text_write(&terminal, " ");
}
vga_text_writeline(&terminal, "");

kfree(numbers);
vga_text_set_color(&terminal, VGA_COLOR_WHITE, VGA_COLOR_RED);

uint8_t test_write[512];
uint8_t test_read[512];

for (int i = 0; i < 512; i++)
    test_write[i] = 0xAA;

if (!ata_write_sector(50, (void*)test_write)) {
    vga_text_writeline(&terminal, "ATA WRITE FAIL");
}

if (!ata_read_sector(50, (void*)test_read)) {
    vga_text_writeline(&terminal, "ATA READ FAIL");
}

for (int i = 0; i < 512; i++) {
    if (test_read[i] != test_write[i]) {
        vga_text_write_dec(&terminal, i);
        vga_text_writeline(&terminal, " ATA DATA FAILLLLLLLLLLLLLLLLLLLLLLLLLLLLLL");
        break;
    }
}

// FILE SYSTEM TESTS

vga_text_writeline(&terminal, "Filesystem tests:");
vga_text_set_color(&terminal, VGA_COLOR_LIGHT_BLUE, VGA_COLOR_BLACK);

fs_touch("/file.txt");

fs_touch("/notes.txt");

vga_text_writeline(&terminal, "Root directory contents:");
fs_ls("/");
vga_text_set_color(&terminal, VGA_COLOR_LIGHT_BLUE, VGA_COLOR_BLACK);
vga_text_writeline(&terminal, "Bin directory contents:");
fs_ls("/bin");

int32_t fd = fs_open("/notes.txt");
```

```
    const char* msg = "Hello filesystem!";
    fs_write(fd, (void*)msg, 17);
    fs_close(fd);
    fd = fs_open("/notes.txt");

    vga_text_set_color(&terminal, VGA_COLOR_LIGHT_GREY, VGA_COLOR_BLACK);
    vga_text_writeline(&terminal, "Running shell:");
    vga_text_set_color(&terminal, VGA_COLOR_WHITE, VGA_COLOR_RED);
    load_program("/bin/shell");
    //load_program("/bin/user_test");

    for (;;)
}
```

Appendix C

Interrupt architecture and basic kernel services

This group defines how the CPU enters the kernel for faults, hardware events, and system calls. Read the C structure and NASM stack pushes together; neither is correct without the other.

C.1 `../lytlnybl/include/kernel/interrupts.h`

Study focus: Use this as the ABI document. Pay special attention to the ordering of `registers_t`, PIC constants, vectors, and system-call numbers.

```
#ifndef INTERRUPTS_H
#define INTERRUPTS_H

//PORT DEFINITIONS

#define PIC1 0x20 /* IO base address for master PIC */
#define PIC2 0xA0 /* IO base address for slave PIC */
#define PIC1_COMMAND PIC1
#define PIC1_DATA (PIC1+1)
#define PIC2_COMMAND PIC2
#define PIC2_DATA (PIC2+1)

//ICW DEFINITIONS

#define ICW1_ICW4 0x01
#define ICW1_SINGLE 0x02
#define ICW1_INTERVAL4 0x04
#define ICW1_LEVEL 0x08
#define ICW1_INIT 0x10

#define ICW4_8086 0x01
#define ICW4_AUTO 0x02
#define ICW4_BUF_SLAVE 0x08
#define ICW4_BUF_MASTER 0x0C
#define ICW4_SFNM 0x10

#define PIC_EOI 0x20 /* End-of-interrupt command code */

// SYSCALL DEFS
#define SYSCALL_EXIT 0x01
#define SYSCALL_GETPID 0x02
```

```
#define SYSCALL_YIELD 0x03
#define SYSCALL_SLEEP 0x04
#define SYSCALL_WRITE 0x05
#define SYSCALL_READ 0x06
#define SYSCALL_SBRK 0x07
#define SYSCALL_FSOPS 0x08
#define SYSCALL_OPEN 0x09
#define SYSCALL_CLOSE 0x0A

#define FS_LS 0
#define FS_MKDIR 1
#define FS_TOUCH 2
#define FS_RM 3
#define FS_RUN 4

#include <stdint.h>
typedef struct {
    uint16_t offset_low;
    uint16_t selector;
    uint8_t reserved;
    uint8_t flags;
    uint16_t offset_high;
} __attribute__((packed)) idt_entry_t;

typedef struct {
    uint16_t limit;
    uint32_t base;
} __attribute__((packed)) idtr_t;

/* registers passed from asm to C */
typedef struct {
    uint32_t ds;

    uint32_t edi;
    uint32_t esi;
    uint32_t ebp;
    uint32_t esp;
    uint32_t ebx;
    uint32_t edx;
    uint32_t ecx;
    uint32_t eax;

    uint32_t interrupt_number;
    uint32_t error_code;

    uint32_t eip;
    uint32_t cs;
    uint32_t eflags;

    uint32_t user_esp;
    uint32_t ss;
} __attribute__((packed)) registers_t;

#define IDT_ENTRIES 256

void idt_init(void);
```

```
void idt_set_gate(
    uint8_t interrupt,
    uint32_t handler_address,
    uint16_t selector,
    uint8_t flags
);

void isr_handler(registers_t* regs);
void irq_handler(registers_t* regs);
void syscall_handler(registers_t* regs);

void pic_remap(int offset1, int offset2);

extern void outb(uint16_t port, uint8_t value);
extern uint8_t inb(uint16_t port);
extern void outw(uint16_t port, uint16_t value);
extern uint16_t inw(uint16_t port);
extern void io_wait(void);

void pic_send_eoi(uint8_t irq);

extern void irq0(void);
extern void irq1(void);
extern void irq2(void);
extern void irq3(void);
extern void irq4(void);
extern void irq5(void);
extern void irq6(void);
extern void irq7(void);
extern void irq8(void);
extern void irq9(void);
extern void irq10(void);
extern void irq11(void);
extern void irq12(void);
extern void irq13(void);
extern void irq14(void);
extern void irq15(void);

extern void idt_load(idtr_t* idtr);

extern void isr0(void);
extern void isr1(void);
extern void isr2(void);
extern void isr3(void);
extern void isr4(void);
extern void isr5(void);
extern void isr6(void);
extern void isr7(void);
extern void isr8(void);
extern void isr9(void);
extern void isr10(void);
extern void isr11(void);
extern void isr12(void);
extern void isr13(void);
extern void isr14(void);
extern void isr15(void);
extern void isr16(void);
extern void isr17(void);
```

```
extern void isr18(void);
extern void isr19(void);
extern void isr20(void);
extern void isr21(void);
extern void isr22(void);
extern void isr23(void);
extern void isr24(void);
extern void isr25(void);
extern void isr26(void);
extern void isr27(void);
extern void isr28(void);
extern void isr29(void);
extern void isr30(void);
extern void isr31(void);

extern void syscall_entry(void);

#endif
```

C.2 ../lytlnybl/kernel/interrupts.asm

Study focus: Follow one ISR stub from entry through saved registers, C call, restoration, and iret. Then find the port-I/O helper routines.

[BITS 32]

```
extern isr_handler
extern irq_handler
extern syscall_handler
```

```
global idt_load
global isr0
global outb
global inb
global outw
global inw
global io_wait
global syscall_entry
```

```
idt_load:
    mov eax, [esp + 4]
    lidt [eax]
    ret
```

```
isr_common_stub:
    pusha

    mov ax, ds
    push eax

    push esp
    call isr_handler
    add esp, 4

    pop eax

    popa

    add esp, 8
    iret
```

```
global isr0
global isr1
global isr2
global isr3
global isr4
global isr5
global isr6
global isr7
global isr8
global isr9
global isr10
global isr11
global isr12
global isr13
global isr14
global isr15
```

```
global isr16
global isr17
global isr18
global isr19
global isr20
global isr21
global isr22
global isr23
global isr24
global isr25
global isr26
global isr27
global isr28
global isr29
global isr30
global isr31

%macro ISR_NOERRCODE 1
isr%1:
    push dword 0
    push dword %1
    jmp isr_common_stub
%endmacro

%macro ISR_ERRCODE 1
isr%1:
    push dword %1
    jmp isr_common_stub
%endmacro

ISR_NOERRCODE 0
ISR_NOERRCODE 1
ISR_NOERRCODE 2
ISR_NOERRCODE 3
ISR_NOERRCODE 4
ISR_NOERRCODE 5
ISR_NOERRCODE 6
ISR_NOERRCODE 7

ISR_ERRCODE 8

ISR_NOERRCODE 9

ISR_ERRCODE 10
ISR_ERRCODE 11
ISR_ERRCODE 12
ISR_ERRCODE 13
ISR_ERRCODE 14

ISR_NOERRCODE 15
ISR_NOERRCODE 16

ISR_ERRCODE 17

ISR_NOERRCODE 18
ISR_NOERRCODE 19
ISR_NOERRCODE 20
ISR_NOERRCODE 21
```

```
ISR_NOERRCODE 22
ISR_NOERRCODE 23
ISR_NOERRCODE 24
ISR_NOERRCODE 25
ISR_NOERRCODE 26
ISR_NOERRCODE 27
ISR_NOERRCODE 28
ISR_NOERRCODE 29

ISR_ERRCODE 30

ISR_NOERRCODE 31

outb:
    mov dx, [esp + 4]
    mov al, [esp + 8]
    out dx, al
    ret
inb:
    mov dx, [esp + 4]
    in al, dx
    movzx eax, al
    ret
outw:
    mov dx, [esp + 4]
    mov ax, [esp + 8]
    out dx, ax
    ret
inw:
    mov dx, [esp + 4]
    in ax, dx
    movzx eax, ax
    ret
io_wait:
    mov al, 0
    out 0x80, al
    ret

irq_common_stub:
    pusha

    mov ax, ds
    push eax

    push esp
    call irq_handler
    add esp, 4

    pop eax

    popa

    add esp, 8
    iret

global irq0
global irq1
global irq2
```

```
global irq3
global irq4
global irq5
global irq6
global irq7
global irq8
global irq9
global irq10
global irq11
global irq12
global irq13
global irq14
global irq15

%macro IRQ 2
irq%1:
    push dword 0
    push dword %2
    jmp irq_common_stub
%endmacro

IRQ 0, 32
IRQ 1, 33
IRQ 2, 34
IRQ 3, 35
IRQ 4, 36
IRQ 5, 37
IRQ 6, 38
IRQ 7, 39
IRQ 8, 40
IRQ 9, 41
IRQ 10, 42
IRQ 11, 43
IRQ 12, 44
IRQ 13, 45
IRQ 14, 46
IRQ 15, 47

syscall_entry:
    push dword 0x80
    push dword 0

    pusha

    mov ax, ds
    push eax

    push esp

    call syscall_handler
    add esp, 4

    pop eax

    popa

    add esp, 8
```

`iret`

C.3 ../lytlnybl/kernel/interrupts.c

Study focus: Trace IDT construction, PIC remapping, exception policy, IRQ dispatch, and the syscall switch statement. Compare every syscall case with its user wrapper.

```
#include "kernel/interrupts.h"
#include "kernel/drivers/vga_text.h"
#include "kernel/drivers/timer.h"
#include "kernel/drivers/keyboard.h"
#include "memory/vmm.h"
#include "memory/pmm.h"
#include "kernel/mappings.h"
#include "tasks/procman.h"
#include "tasks/context.h"
#include "tasks/scheduler.h"
#include "kernel/kernel_utils.h"
#include "filesystem/fs_manager.h"
#include "tasks/loader.h"

extern vga_text terminal;

const char* exception_messages[32] =
{
    "Divide By Zero",
    "Debug",
    "Non Maskable Interrupt",
    "Breakpoint",
    "Overflow",
    "Bound Range Exceeded",
    "Invalid Opcode",
    "Device Not Available",
    "Double Fault",
    "Coprocessor Segment Overrun",
    "Invalid TSS",
    "Segment Not Present",
    "Stack Segment Fault",
    "General Protection Fault",
    "Page Fault",
    "Reserved",
    "x87 Floating Point Exception",
    "Alignment Check",
    "Machine Check",
    "SIMD Floating Point Exception",
    "Virtualization Exception",
    "Control Protection Exception",
    "Reserved",
    "Reserved",
    "Reserved",
    "Reserved",
    "Reserved",
    "Reserved",
    "Hypervisor Injection Exception",
    "VMM Communication Exception",
    "Security Exception",
    "Reserved"
};
```

```
idt_entry_t idt[256];
idtr_t idtr;

void idt_set_gate(
    uint8_t interrupt,
    uint32_t handler_address,
    uint16_t selector,
    uint8_t flags
) {
    idt[interrupt].offset_low = handler_address & 0xFFFF;

    idt[interrupt].selector = selector;

    idt[interrupt].reserved = 0;

    idt[interrupt].flags = flags;

    idt[interrupt].offset_high = (handler_address >> 16) & 0xFFFF;
}

void isr_handler(registers_t* regs) {
    switch (regs->interrupt_number) {
        case 14:
            page_fault_handler(regs);
            break;
        default:
            vga_text_writeline(&terminal, exception_messages[regs->interrupt_number]);
            break;
    }

    if ((regs->cs & 3) == 3) {
        current_process->state = PROCESS_TERMINATED;
        context_switch(current_process, get_next_process(), regs);
        return;
    }

    for (;;)
}

void irq_handler(registers_t* regs) {
    switch (regs->interrupt_number - 32) {
        case 0:
            timer_handler(regs);
            break;
        case 1:
            keyboard_handler();
            break;
        case 2:
            break;
    }
    pic_send_eoi(regs->interrupt_number - 32);
}

void syscall_handler(registers_t* regs) {
    int fd;
    uint32_t buffer;
    size_t count;
```

```

switch (regs->eax) {
    case SYSCALL_EXIT:
        // wake up parent
        process_t* parent = find_process_by_pid(current_process->parent_pid);

        if (parent != NULL && parent->pid_waiting_for == current_process->pid) {
            parent->pid_waiting_for = 0;
            parent->state = PROCESS_READY;
        }

        //exit
        current_process->state = PROCESS_TERMINATED;
        context_switch(current_process, get_next_process(), regs);
        break;
    case SYSCALL_GETPID:
        regs->eax = current_process->pid;
        break;
    case SYSCALL_YIELD:
        context_switch(current_process, get_next_process(), regs);
        break;
    case SYSCALL_SLEEP:
        current_process->wake_tick = timer_get_ticks() + regs->ebx;
        current_process->state = PROCESS_SLEEPING;
        context_switch(current_process, get_next_process(), regs);
        break;
    case SYSCALL_WRITE:
        fd = regs->ebx;
        buffer = regs->ecx;
        count = regs->edx;

        if (fd < 1) {
            regs->eax = -1;
            break;
        }
        if (fd > 2) {
            regs->eax = fs_write(fd, (void*)buffer, count);
            break;
        }

        ((char*) buffer)[count] = '\0';
        vga_text_write(&terminal, (char*)buffer);

        regs->eax = (int)count;
        break;
    case SYSCALL_READ:
        fd = regs->ebx;
        buffer = regs->ecx;
        count = regs->edx;

        if (fd < 0 || fd == 1) {
            regs->eax = -1;
            return;
        }
        if (fd > 2) {
            regs->eax = fs_read(fd, (void*)buffer, count);
            break;
        }
}

```

```

current_process->state = PROCESS_BLOCKED;
current_process->reading_state.buffer = (void*)regs->ecx;
current_process->reading_state.size = regs->edx;
current_process->reading_state.count = 0;
context_switch(current_process, get_next_process(), regs);

//keyboard is treated as stdin, so need to block until we recieve that data
//need to block the process until we wait for input

break;
case SYSCALL_SBRK:
    current_process->user_heap_end += regs->ebx;

    //allocate more pages
    while (((current_process->user_heap_end + 4096) - USER_HEAP_START) / 4096
        > current_process->heap_pages_allocated){
        map_page(current_process->page_directory,
            USER_HEAP_START + (current_process->heap_pages_allocated++ * 4096),
            (uintptr_t)alloc_frame(),
            PAGE_PRESENT | PAGE_USER | PAGE_WRITABLE
        );
    }

    regs->eax = current_process->user_heap_end;

    break;
case SYSCALL_FSOPS:
    char* path = (char*)regs->ecx;
    switch (regs->ebx) {
        case FS_LS:
            fs_ls(path);
            break;
        case FS_MKDIR:
            fs_mkdir(path);
            break;
        case FS_TOUCH:
            fs_touch(path);
            break;
        case FS_RM:
            fs_rm(path);
            break;
        case FS_RUN:
            process_t* child = load_program(path);

            child->parent_pid = current_process->pid;

            current_process->pid_waiting_for = child->pid;
            current_process->state = PROCESS_BLOCKED;
            context_switch(current_process, get_next_process(), regs);
            break;
    }
    break;
case SYSCALL_OPEN:

```

```

        regs->eax = fs_open((char*)regs->ebx);
        break;
    case SYSCALL_CLOSE:
        fs_close(regs->ebx);
        break;
    default:
        vga_text_writeline(&terminal, "syscall not found");
        break;
}
return;
}

void idt_init(void) {
    memset(idt, 0, sizeof(idt));

    idtr.limit = sizeof(idt) - 1;

    idtr.base = (uint32_t)idt;

    idt_set_gate(0, (uint32_t)isr0, 0x08, 0x8E);
    idt_set_gate(1, (uint32_t)isr1, 0x08, 0x8E);
    idt_set_gate(2, (uint32_t)isr2, 0x08, 0x8E);
    idt_set_gate(3, (uint32_t)isr3, 0x08, 0x8E);
    idt_set_gate(4, (uint32_t)isr4, 0x08, 0x8E);
    idt_set_gate(5, (uint32_t)isr5, 0x08, 0x8E);
    idt_set_gate(6, (uint32_t)isr6, 0x08, 0x8E);
    idt_set_gate(7, (uint32_t)isr7, 0x08, 0x8E);
    idt_set_gate(8, (uint32_t)isr8, 0x08, 0x8E);
    idt_set_gate(9, (uint32_t)isr9, 0x08, 0x8E);
    idt_set_gate(10, (uint32_t)isr10, 0x08, 0x8E);
    idt_set_gate(11, (uint32_t)isr11, 0x08, 0x8E);
    idt_set_gate(12, (uint32_t)isr12, 0x08, 0x8E);
    idt_set_gate(13, (uint32_t)isr13, 0x08, 0x8E);
    idt_set_gate(14, (uint32_t)isr14, 0x08, 0x8E);
    idt_set_gate(15, (uint32_t)isr15, 0x08, 0x8E);
    idt_set_gate(16, (uint32_t)isr16, 0x08, 0x8E);
    idt_set_gate(17, (uint32_t)isr17, 0x08, 0x8E);
    idt_set_gate(18, (uint32_t)isr18, 0x08, 0x8E);
    idt_set_gate(19, (uint32_t)isr19, 0x08, 0x8E);
    idt_set_gate(20, (uint32_t)isr20, 0x08, 0x8E);
    idt_set_gate(21, (uint32_t)isr21, 0x08, 0x8E);
    idt_set_gate(22, (uint32_t)isr22, 0x08, 0x8E);
    idt_set_gate(23, (uint32_t)isr23, 0x08, 0x8E);
    idt_set_gate(24, (uint32_t)isr24, 0x08, 0x8E);
    idt_set_gate(25, (uint32_t)isr25, 0x08, 0x8E);
    idt_set_gate(26, (uint32_t)isr26, 0x08, 0x8E);
    idt_set_gate(27, (uint32_t)isr27, 0x08, 0x8E);
    idt_set_gate(28, (uint32_t)isr28, 0x08, 0x8E);
    idt_set_gate(29, (uint32_t)isr29, 0x08, 0x8E);
    idt_set_gate(30, (uint32_t)isr30, 0x08, 0x8E);
    idt_set_gate(31, (uint32_t)isr31, 0x08, 0x8E);
    idt_set_gate(32, (uint32_t)irq0, 0x08, 0x8E);
    idt_set_gate(33, (uint32_t)irq1, 0x08, 0x8E);
    idt_set_gate(34, (uint32_t)irq2, 0x08, 0x8E);
    idt_set_gate(35, (uint32_t)irq3, 0x08, 0x8E);
    idt_set_gate(36, (uint32_t)irq4, 0x08, 0x8E);
    idt_set_gate(37, (uint32_t)irq5, 0x08, 0x8E);
    idt_set_gate(38, (uint32_t)irq6, 0x08, 0x8E);

```

```
    idt_set_gate(39, (uint32_t)irq7, 0x08, 0x8E);
    idt_set_gate(40, (uint32_t)irq8, 0x08, 0x8E);
    idt_set_gate(41, (uint32_t)irq9, 0x08, 0x8E);
    idt_set_gate(42, (uint32_t)irq10, 0x08, 0x8E);
    idt_set_gate(43, (uint32_t)irq11, 0x08, 0x8E);
    idt_set_gate(44, (uint32_t)irq12, 0x08, 0x8E);
    idt_set_gate(45, (uint32_t)irq13, 0x08, 0x8E);
    idt_set_gate(46, (uint32_t)irq14, 0x08, 0x8E);
    idt_set_gate(47, (uint32_t)irq15, 0x08, 0x8E);

    //software interrupts
    idt_set_gate(0x80, (uint32_t)syscall_entry, 0x08, 0xEE);

    idt_load(&idtr);
    pic_remap(0x20, 0x28);
    asm volatile("sti");
}

void pic_send_eoi(uint8_t irq){
    if(irq >= 8)
        outb(PIC2_COMMAND, PIC_EOI);

    outb(PIC1_COMMAND, PIC_EOI);
}

void pic_remap(int offset1, int offset2) {
    //save state of enabled IRQs
    uint8_t a1 = inb(PIC1_DATA);
    uint8_t a2 = inb(PIC2_DATA);

    /* Enter initialization mode. */
    outb(PIC1_COMMAND, ICW1_INIT | ICW1_ICW4);
    io_wait();

    outb(PIC2_COMMAND, ICW1_INIT | ICW1_ICW4);
    io_wait();

    /* set up first interrupt vector used by master and slave */

    outb(PIC1_DATA, offset1);
    io_wait();

    outb(PIC2_DATA, offset2);
    io_wait();

    /* Connect master and slave through IRQ2 line */
    outb(PIC1_DATA, 4);
    io_wait();

    outb(PIC2_DATA, 2);
    io_wait();

    /* select 8086/x86 interrupt mode */
    outb(PIC1_DATA, ICW4_8086);
    io_wait();

    outb(PIC2_DATA, ICW4_8086);
}
```

```
    io_wait();

    /* restore interrupt masks */
    outb(PIC1_DATA, a1);
    outb(PIC2_DATA, a2);
}
```

C.4 ../lytlnybl/include/kernel/kernel_utils.h

Study focus: These are the small freestanding helpers that replace normal library assumptions in kernel code.

```
#ifndef KERNEL_UTILS_H
#define KERNEL_UTILS_H

#include <stddef.h>
#include <stdint.h>

void* memset(void* dest, uint8_t val, size_t len);
void* memcpy(void* dest, const void* src, size_t len);
int32_t strcmp(const char* a, const char* b);
char* strncpy(char* dest, const char* src, size_t n);

#endif
```

C.5 ../lytlnybl/kernel/kernel_utils.c

Study focus: Read implementation details such as byte copying carefully: utilities run in many contexts and mistakes here spread everywhere.

```
#include "kernel/kernel_utils.h"

void* memset(void* dest, uint8_t val, size_t len) {
    uint8_t* d = (uint8_t*)dest;

    for (size_t i = 0; i < len; i++)
        d[i] = val;

    return dest;
}

void* memcpy(void* dest, const void* src, size_t len) {
    uint8_t* d = (uint8_t*)dest;
    const uint8_t* s = (const uint8_t*)src;

    for (size_t i = 0; i < len; i++)
        d[i] = s[i];

    return dest;
}

int32_t strcmp(const char* a, const char* b) {
    size_t i = 0;

    while (a[i] != '\0' && b[i] != '\0' && a[i] == b[i])
        i++;

    return (uint8_t)a[i] - (uint8_t)b[i];
}

char* strncpy(char* dest, const char* src, size_t n) {
    size_t i = 0;

    for (; i < n && src[i] != '\0'; i++)
        dest[i] = src[i];

    for (; i < n; i++)
        dest[i] = '\0';

    return dest;
}
```

C.6 ../lytlnybl/include/kernel/mappings.h

Study focus: Keep these address constants visible while reading paging, process creation, and the loader. A constant is a design claim that must be confirmed by mappings.

```
#ifndef MAPPINGS_H
#define MAPPINGS_H

#define USER_CODE_BASE 0x00400000
#define USER_VGA 0x00B00000
#define HEAP_START 0xC0000000
#define USER_STACK_TOP 0xBFFF000
#define USER_HEAP_START 0x00A00000

#endif
```

Appendix D

Screen, timer, keyboard, and disk drivers

Drivers translate a device-specific protocol into a small kernel interface. For each one, make a table with inputs, persistent state, device ports or memory, interrupt context, and outputs visible to other modules.

D.1 `../lytlntybl/include/kernel/drivers/vga_text.h`

Study focus: Learn the terminal state structure and the color values before reading how characters reach 0xB8000.

```
#ifndef VGA_TEXT_H
#define VGA_TEXT_H

#include <stddef.h>
#include <stdint.h>

typedef enum {
    VGA_COLOR_BLACK = 0,
    VGA_COLOR_BLUE,
    VGA_COLOR_GREEN,
    VGA_COLOR_CYAN,
    VGA_COLOR_RED,
    VGA_COLOR_MAGENTA,
    VGA_COLOR_BROWN,
    VGA_COLOR_LIGHT_GREY,
    VGA_COLOR_DARK_GREY,
    VGA_COLOR_LIGHT_BLUE,
    VGA_COLOR_LIGHT_GREEN,
    VGA_COLOR_LIGHT_CYAN,
    VGA_COLOR_LIGHT_RED,
    VGA_COLOR_LIGHT_MAGENTA,
    VGA_COLOR_LIGHT_BROWN,
    VGA_COLOR_WHITE
} vga_color;

typedef struct {
    size_t row;
    size_t column;

    size_t width;
```

```
    size_t height;

    uint8_t color;

    uint16_t* buffer;
} vga_text;

// init
void vga_text_init(vga_text* terminal);

//screen ops
//
void vga_text_clear(vga_text* terminal);

/* cursor ops */
void vga_text_set_cursor(
    vga_text* terminal,
    size_t row,
    size_t column
);

/* Color ops */
void vga_text_set_color(
    vga_text* terminal,
    vga_color f,
    vga_color b
);

/* char out */
void vga_text_putchar(
    vga_text* terminal,
    char c
);

/* string out */
void vga_text_write(
    vga_text* terminal,
    const char* string
);

void vga_text_writeline(
    vga_text* terminal,
    const char* string
);

/* num out */
void vga_text_write_dec(
    vga_text* terminal,
    uint32_t value
);

void vga_text_write_hex(
    vga_text* terminal,
    uint32_t value
);

/* low level writing */
```

```
void vga_text_put_entry_at(
    vga_text* terminal,
    char character,
    uint8_t fcolor,
    uint8_t bcolor,
    size_t row,
    size_t column
);

/* operations added in making input */

void vga_text_backspace(vga_text* terminal);
char vga_text_getchar(vga_text* terminal);

#endif
```

D.2 ../lytlnybl/kernel/drivers/vga_text.c

Study focus: Trace a string write across line wrapping and scrolling. Check why VGA memory is volatile.

```
#include "kernel/drivers/vga_text.h"

void vga_text_init(vga_text* terminal) {
    terminal->row = 0;
    terminal->column = 0;

    vga_text_set_color(terminal, VGA_COLOR_WHITE, VGA_COLOR_RED);

    terminal->width = 80;
    terminal->height = 25;

    terminal->buffer = (uint16_t*)0xB8000;
    vga_text_clear(terminal);
}

void vga_text_clear(vga_text* terminal) {
    uint8_t color = terminal->color;

    uint16_t blank = ((uint16_t)color << 8) | ' ';

    for (size_t row = 0; row < terminal->height; row++) {
        for (size_t col = 0; col < terminal->width; col++) {
            size_t index = row * terminal->width + col;
            terminal->buffer[index] = blank;
        }
    }

    terminal->row = 0;
    terminal->column = 0;
}

void vga_text_set_color(vga_text* terminal, vga_color f, vga_color b) {
    terminal->color = ((uint8_t)b << 4) | (uint8_t)f;
}

void vga_text_set_cursor(vga_text* terminal, size_t row, size_t column) {
    terminal->row = row;
    terminal->column = column;
}

void vga_text_putchar(vga_text* terminal, char c) {
    uint8_t color = terminal->color;
    size_t index = terminal->row * terminal->width + terminal->column;
    uint16_t entry = ((uint16_t)color << 8) | (uint16_t)c;
    terminal->buffer[index] = entry;
}

char vga_text_getchar(vga_text* terminal) {
    size_t index = terminal->row * terminal->width + terminal->column;
    char c = (uint8_t)(terminal->buffer[index] & 0x00FF);
    return c;
}
```

```
void vga_text_write(vga_text* terminal, const char* string) {
    size_t pos = terminal->row * terminal->width + terminal->column;

    for (size_t i = 0; string[i] != '\0'; i++) {
        if (string[i] == '\n') {
            vga_text_writeline(terminal, "");
            continue;
        }
        vga_text_putchar(terminal, string[i]);
        pos++;
        terminal->row = pos / terminal->width;
        terminal->column = pos % terminal->width;
        terminal->row = terminal->row % terminal->height;
    }
}

void vga_text_writeline(vga_text* terminal, const char* string) {
    vga_text_write(terminal, string);
    terminal->column = 0;
    terminal->row++;
    terminal->row = terminal->row % terminal->height;
}

void vga_text_write_dec(vga_text* terminal, uint32_t value) {
    char buffer[10];
    size_t digits = 0;
    if (value == 0) {
        vga_text_putchar(terminal, '0');
        return;
    }

    while (value > 0) {
        buffer[digits++] = '0' + (value % 10);
        value /= 10;
    }

    size_t index = 0;
    while (index < digits) {
        vga_text_putchar(terminal, buffer[--digits]);
        terminal->column++;
    }
}

void vga_text_write_hex(vga_text* terminal, uint32_t value) {
    char buffer[8];
    size_t digits = 0;

    if (value == 0) {
        vga_text_putchar(terminal, '0');
        return;
    }

    while(value > 0) {
        uint32_t digit = value % 16;

        if (digit < 10) {
            buffer[digits++] = '0' + digit;
        }
    }
}
```

```
        } else {
            buffer[digits++] = 'A' + (digit - 10);
        }
        value /= 16;
    }

    size_t index = 0;
    while (index < digits) {
        vga_text_putchar(terminal, buffer[--digits]);
        terminal->column++;
    }
}

void vga_text_put_entry_at(
    vga_text* terminal,
    char character,
    uint8_t fcolor,
    uint8_t bcolor,
    size_t row,
    size_t column
) {
    uint8_t color = ((uint8_t)bcolor << 4) | (uint8_t)fcolor;
    size_t index = row * terminal->width + column;
    uint16_t entry = ((uint16_t)color << 8) | (uint16_t)character;
    terminal->buffer[index] = entry;
}

void vga_text_backspace(vga_text* terminal) {
    if (terminal->row == 0 && terminal->column == 0) return;
    else if (terminal->column != 0) {
        terminal->column--;
    } else {
        terminal->row--;
        terminal->column = terminal->width - 1;
        while (vga_text_getchar(terminal) == ' ') {
            terminal->column--;
        }
    }
    vga_text_putchar(terminal, ' ');
}
```

D.3 ../lytlnybl/include/kernel/drivers/timer.h

Study focus: Relate PIT constants to the divisor calculation and tick-based API.

```
#ifndef TIMER_H
#define TIMER_H

/* PIT Ports */
#define PIT_CHANNELO_DATA    0x40
#define PIT_CHANNEL1_DATA    0x41
#define PIT_CHANNEL2_DATA    0x42
#define PIT_COMMAND          0x43

/* PIT Input Clock */
#define PIT_BASE_FREQUENCY    1193182

/* Channel Selection */
#define PIT_CHANNELO          0x00
#define PIT_CHANNEL1          0x40
#define PIT_CHANNEL2          0x80

/* Access Mode */
#define PIT_LATCH              0x00
#define PIT_ACCESS_LOBYTE     0x10
#define PIT_ACCESS_HIBYTE     0x20
#define PIT_ACCESS_LOHIBYTE   0x30

/* Operating Modes */
#define PIT_MODE0              0x00
#define PIT_MODE1              0x02
#define PIT_MODE2              0x04
#define PIT_MODE3              0x06
#define PIT_MODE4              0x08
#define PIT_MODE5              0x0A

/* Counting Mode */
#define PIT_BINARY              0x00
#define PIT_BCD                 0x01

#include <stdint.h>
#include <stdbool.h>
#include "kernel/interrupts.h"

void timer_init(uint32_t frequency);

void timer_handler(registers_t* regs);

uint64_t timer_get_ticks(void);

void timer_wait_ms(uint32_t ms);

#endif
```

D.4 ../lytlnybl/kernel/drivers/timer.c

Study focus: Follow initialization, IRQ tick handling, sleepers waking, and the hand-off to the scheduler.

```
#include "kernel/drivers/timer.h"
#include "kernel/interrupts.h"
#include "kernel/drivers/vga_text.h"
#include "tasks/procman.h"
#include "tasks/scheduler.h"

volatile uint32_t ticks = 0;
static uint32_t freq;

extern vga_text terminal;

void timer_init(uint32_t frequency) {
    freq = frequency;
    uint16_t divisor = 1193182 / frequency;

    /* tell pit how we send the divisor value and the mode*/
    outb(PIT_COMMAND, PIT_ACCESS_LOHIBYTE | PIT_MODE3 | PIT_CHANNEL0 | PIT_BINARY);
    io_wait();

    /* write low and high bytes respectively */
    outb(PIT_CHANNEL0_DATA, divisor & 0xFF);
    io_wait();
    outb(PIT_CHANNEL0_DATA, divisor >> 8);
    io_wait();
}

void timer_handler(registers_t* regs) {
    ticks++;
    if ((ticks % 100) == 0) {
        //vga_text_writeline(&terminal, " 1 second ");
    }

    process_t* traversal_process = process_head;
    while (traversal_process) {
        if (traversal_process->state == PROCESS_SLEEPING && ticks >= traversal_process->wake_tick) {
            traversal_process->state = PROCESS_READY;
        }
        traversal_process = traversal_process->next;
    }

    schedule(regs);
}

uint64_t timer_get_ticks() {
    return ticks;
}

void timer_wait_ms(uint32_t ms) {
    uint32_t start = ticks;

    while ((ticks - start) < ms) {
        asm volatile ("hlt");
    }
}
```

```
}  
}
```

D.5 ../lytlnybl/include/kernel/drivers/keyboard.h

Study focus: Treat scan codes and modifier constants as the grammar of this hardware protocol.

```
#ifndef KEYBOARD_H
#define KEYBOARD_H

#define PS2_DATA 0x60
#define PS2_COMMAND 0x64

#define ENABLE_SCANNING 0xF4
#define DISABLE_SCANNING 0xF5
#define SET_LED 0xED

#define KEY_RELEASED 0x80
#define EXTENDED_SCANCODE 0xE0

#define SHIFT_PRESS 0x2A
#define L_CTRL_PRESS 0x9C

#include <stdint.h>
#include <stdbool.h>

void keyboard_handler(void);
void keyboard_init(void);
void keyboard_set_keymap(void);
void keyboard_set_shift_keymap(void);
bool keyboard_modifier_keys(uint8_t scancode);
void keyboard_extended_scancodes(void);

#endif
```

D.6 ../lytlnybl/kernel/drivers/keyboard.c

Study focus: Trace one printable key, Backspace, and Enter. Determine exactly when a blocked read becomes ready.

```
#include "kernel/drivers/keyboard.h"
#include "kernel/interrupts.h"
#include "kernel/drivers/vga_text.h"
#include "tasks/procman.h"

volatile bool shift_pressed = false;
volatile bool ctrl_pressed = false;
volatile bool alt_pressed = false;

extern vga_text terminal;

char keymap[128];
char shift_keymap[128];

void keyboard_init() {
    outb(PS2_DATA, ENABLE_SCANNING);
    keyboard_set_keymap();
    keyboard_set_shift_keymap();
}

void keyboard_set_keymap(void)
{
    /* Numbers */
    keymap[0x02] = '1';
    keymap[0x03] = '2';
    keymap[0x04] = '3';
    keymap[0x05] = '4';
    keymap[0x06] = '5';
    keymap[0x07] = '6';
    keymap[0x08] = '7';
    keymap[0x09] = '8';
    keymap[0x0A] = '9';
    keymap[0x0B] = '0';

    /* Symbols */
    keymap[0x0C] = '-';
    keymap[0x0D] = '=';
    keymap[0x1A] = '[';
    keymap[0x1B] = ']';
    keymap[0x27] = ';';
    keymap[0x28] = '\\';
    keymap[0x29] = '\'';
    keymap[0x2B] = '\\\\';
    keymap[0x33] = ',';
    keymap[0x34] = '.';
    keymap[0x35] = '/';

    /* Top row */
    keymap[0x10] = 'q';
    keymap[0x11] = 'w';
    keymap[0x12] = 'e';
    keymap[0x13] = 'r';
    keymap[0x14] = 't';
```

```

    keymap[0x15] = 'y';
    keymap[0x16] = 'u';
    keymap[0x17] = 'i';
    keymap[0x18] = 'o';
    keymap[0x19] = 'p';

    /* Home row */
    keymap[0x1E] = 'a';
    keymap[0x1F] = 's';
    keymap[0x20] = 'd';
    keymap[0x21] = 'f';
    keymap[0x22] = 'g';
    keymap[0x23] = 'h';
    keymap[0x24] = 'j';
    keymap[0x25] = 'k';
    keymap[0x26] = 'l';

    /* Bottom row */
    keymap[0x2C] = 'z';
    keymap[0x2D] = 'x';
    keymap[0x2E] = 'c';
    keymap[0x2F] = 'v';
    keymap[0x30] = 'b';
    keymap[0x31] = 'n';
    keymap[0x32] = 'm';

    /* Control characters */
    keymap[0x01] = 27;      /* Escape */
    keymap[0x0E] = '\b';    /* Backspace */
    keymap[0x0F] = '\t';    /* Tab */
    keymap[0x1C] = '\n';    /* Enter */
    keymap[0x39] = ' ';

}

void keyboard_set_shift_keymap(void)
{
    /* Numbers */
    shift_keymap[0x02] = '!';
    shift_keymap[0x03] = '@';
    shift_keymap[0x04] = '#';
    shift_keymap[0x05] = '$';
    shift_keymap[0x06] = '%';
    shift_keymap[0x07] = '^';
    shift_keymap[0x08] = '&';
    shift_keymap[0x09] = '*';
    shift_keymap[0x0A] = '(';
    shift_keymap[0x0B] = ')';

    /* Symbols */
    shift_keymap[0x0C] = '_';
    shift_keymap[0x0D] = '+';
    shift_keymap[0x1A] = '{';
    shift_keymap[0x1B] = '}';
    shift_keymap[0x27] = ':';
    shift_keymap[0x28] = '"';
    shift_keymap[0x29] = '~';
    shift_keymap[0x2B] = '|';
    shift_keymap[0x33] = '<';

```

```

    shift_keymap[0x34] = '>';
    shift_keymap[0x35] = '?';

    /* Top row */
    shift_keymap[0x10] = 'Q';
    shift_keymap[0x11] = 'W';
    shift_keymap[0x12] = 'E';
    shift_keymap[0x13] = 'R';
    shift_keymap[0x14] = 'T';
    shift_keymap[0x15] = 'Y';
    shift_keymap[0x16] = 'U';
    shift_keymap[0x17] = 'I';
    shift_keymap[0x18] = 'O';
    shift_keymap[0x19] = 'P';

    /* Home row */
    shift_keymap[0x1E] = 'A';
    shift_keymap[0x1F] = 'S';
    shift_keymap[0x20] = 'D';
    shift_keymap[0x21] = 'F';
    shift_keymap[0x22] = 'G';
    shift_keymap[0x23] = 'H';
    shift_keymap[0x24] = 'J';
    shift_keymap[0x25] = 'K';
    shift_keymap[0x26] = 'L';

    /* Bottom row */
    shift_keymap[0x2C] = 'Z';
    shift_keymap[0x2D] = 'X';
    shift_keymap[0x2E] = 'C';
    shift_keymap[0x2F] = 'V';
    shift_keymap[0x30] = 'B';
    shift_keymap[0x31] = 'N';
    shift_keymap[0x32] = 'M';

    /* Control characters */
    shift_keymap[0x01] = 27;
    shift_keymap[0x0E] = '\b';
    shift_keymap[0x0F] = '\t';
    shift_keymap[0x1C] = '\n';
    shift_keymap[0x39] = ' ';
}

void keyboard_extended_scancodes() {
    uint8_t scancode = inb(PS2_DATA);

    /* This only handles the LGUI press for now */
    if (scancode == 0x5B) {
    }
}

bool keyboard_modifier_keys(uint8_t scancode) {
    /* shift */
    if (scancode == SHIFT_PRESS) {
        shift_pressed = true;
        return true;
    } else if (scancode == (SHIFT_PRESS | KEY_RELEASED)) {
        shift_pressed = false;
    }
}

```

```

        return true;
    }

    /* control */
    if (scancode == L_CTRL_PRESS) {
        ctrl_pressed = true;
        return true;
    } else if (scancode == (L_CTRL_PRESS | KEY_RELEASED)) {
        ctrl_pressed = false;
        return true;
    }

    return false;
}

void keyboard_handler () {
    uint8_t scancode = inb(PS2_DATA);

    if (keyboard_modifier_keys(scancode)) {
        return;
    }
    if (scancode == EXTENDED_SCANCODE) {
        keyboard_extended_scancodes();
        return;
    }
    if (scancode & KEY_RELEASED) {
        return;
    }

    if (scancode >= 128) {
        return;
    }

    char c[2];
    c[0] = keymap[scancode];
    c[1] = '\0';

    switch (c[0]) {
        case 27:
            vga_text_clear(&terminal);
            break;
        case '\t':
            vga_text_write(&terminal, "    ");
            break;
        case '\b':
            vga_text_backspace(&terminal);
        case '\n':
        default:
            if (shift_pressed) {
                c[0] = shift_keymap[scancode];
            }
            if (c[0] != '\b')
                vga_text_write(&terminal, c);
            if (!c[0]) return;

            process_t* traversal_process = process_head;
            while (traversal_process) {
                if (traversal_process->state == PROCESS_BLOCKED && traversal_process->reading_state.si

```

```
void* buffer = traversal_process->reading_state.buffer;
uint32_t size = traversal_process->reading_state.size;

if (c[0] == '\\b') {
    if (traversal_process->reading_state.count)
        traversal_process->reading_state.count--;
    set_cr3((uintptr_t)traversal_process->page_directory);
    ((char*)(buffer))[traversal_process->reading_state.count] = '\\0';
    set_cr3((uintptr_t)kernel_directory);
} else {
    //quite a crude way of doing this, (really only the context switcher should
    //be changing cr3), but it works
    set_cr3((uintptr_t)traversal_process->page_directory);
    ((char*)(buffer))[traversal_process->reading_state.count++] = c[0];
    set_cr3((uintptr_t)kernel_directory);
}

if (traversal_process->reading_state.count >= size || c[0] == '\\n') {
    traversal_process->regs.eax = traversal_process->reading_state.count;
    traversal_process->reading_state.count = 0;
    traversal_process->reading_state.size = 0;
    traversal_process->state = PROCESS_READY;
}

}

traversal_process = traversal_process->next;
}
break;
}
}
```

D.7 ../lytlnybl/include/kernel/drivers/ata.h

Study focus: Use these port and status constants while tracing a PIO sector transaction.

```
#ifndef ATA_H
#define ATA_H

#include <stdint.h>
#include <stdbool.h>

#define ATA_DATA 0x1F0
#define ATA_SECTOR_COUNT 0x1F2
#define ATA_LBA_LOW 0x1F3
#define ATA_LBA_MID 0x1F4
#define ATA_LBA_HIGH 0x1F5
#define ATA_DRIVE 0x1F6
#define ATA_STATUS 0x1F7
#define ATA_COMMAND 0x1F7

#define ATA_STATUS_ERR 0x01
#define ATA_STATUS_DRQ 0x08
#define ATA_STATUS_DF 0x20
#define ATA_STATUS_BSY 0x80

#define ATA_CMD_READ_PIO 0x20
#define ATA_CMD_WRITE_PIO 0x30
#define ATA_CMD_FLUSH_CACHE 0xE7

#define ATA_DRIVE_MASTER 0xE0

#define ATA_SECTOR_SIZE 512

#define ATA_ALT_STATUS 0x3F6

void init_ata(void);

bool ata_read_sector(uint32_t lba, void* buffer);
bool ata_write_sector(uint32_t lba, const void* buffer);

#endif
```

D.8 ../lytlnybl/kernel/drivers/ata.c

Study focus: Read one sector read and one sector write as a state machine: select, command, wait, transfer, and report failure.

```
#include "kernel/drivers/ata.h"
#include "kernel/interrupts.h"

static void ata_400ns_delay() {
    inb(ATA_ALT_STATUS);
    inb(ATA_ALT_STATUS);
    inb(ATA_ALT_STATUS);
    inb(ATA_ALT_STATUS);
}

static void ata_wait_bsy() {
    uint8_t status;
    do {
        status = inb(ATA_STATUS);
    } while(status & ATA_STATUS_BSY);
}

static bool ata_wait_drq() {
    uint8_t status;

    for(;;) {
        status = inb(ATA_STATUS);
        if (status & ATA_STATUS_BSY) continue;
        if (status & ATA_STATUS_ERR) return false;
        if (status & ATA_STATUS_DF) return false;
        if (status & ATA_STATUS_DRQ) return true;
    }
}

static void ata_select_drive(uint32_t lba) {
    outb(ATA_DRIVE, ATA_DRIVE_MASTER | ((lba >> 24) & 0x0F));
    ata_400ns_delay();
    ata_wait_bsy();
}

void init_ata() {
    outb(ATA_DRIVE, ATA_DRIVE_MASTER);
    ata_400ns_delay();
}

bool ata_read_sector(uint32_t lba, void* buffer) {
    uint16_t *data = (uint16_t*)buffer;

    if (lba > 0xFFFFFFFF) return false;

    ata_select_drive(lba);

    outb(ATA_SECTOR_COUNT, 1);

    outb(ATA_LBA_LOW, lba & 0xFF);
    outb(ATA_LBA_MID, (lba >> 8) & 0xFF);
    outb(ATA_LBA_HIGH, (lba >> 16) & 0xFF);
}
```

```
    outb(ATA_COMMAND, ATA_CMD_READ_PIO);

    if (!ata_wait_drq()) return false;

    for (int i = 0; i < 256; i++) {
        data[i] = inw(ATA_DATA);
    }

    ata_400ns_delay();

    return true;
}

bool ata_write_sector(uint32_t lba, const void* buffer) {
    const uint16_t *data = (const uint16_t*)buffer;

    if (lba > 0xFFFFFFFF) return false;

    ata_select_drive (lba);

    outb(ATA_SECTOR_COUNT, 1);

    outb(ATA_LBA_LOW, lba & 0xFF);
    outb(ATA_LBA_MID, (lba >> 8) & 0xFF);
    outb(ATA_LBA_HIGH, (lba >> 16) & 0xFF);

    outb(ATA_COMMAND, ATA_CMD_WRITE_PIO);

    if (!ata_wait_drq()) return false;

    for (int i = 0; i < 256; i++) {
        outw(ATA_DATA, data[i]);
    }

    ata_400ns_delay();
    outb(ATA_COMMAND, ATA_CMD_FLUSH_CACHE);
    ata_wait_bsy();

    return true;
}
```

Appendix E

Physical frames, virtual pages, and the heap

Read this group only after completing the slow virtual-memory chapter. Keep a paper address translation beside you. The code is compact, but every pointer has a physical or virtual meaning that matters.

E.1 `../lytlntybl/include/memory/pmm.h`

Study focus: The PMM contract: E820 records enter, aligned frames leave. Identify bitmap ownership.

```
#ifndef PPM_H
#define PPM_H

#include <stdint.h>
#include <stdbool.h>

#define FRAME_SIZE 4096
#define BITMAP_BASE 0x100000

typedef struct {
    uint64_t base;
    uint64_t length;
    uint32_t type;
    uint32_t attributes;
} __attribute__((packed)) memory_map_entry_t;

void init_pmm();

void bitmap_set_frame(uint32_t frame_index);
void bitmap_clear_frame(uint32_t frame_index);
void bitmap_get_frame(uint32_t frame_index);
void print_bitmap_summary();

void* alloc_frame();
void free_frame(void* frame_address);

#endif
```

E.2 ../lytlnybl/memory/pmm.c

Study focus: Trace bitmap initialization from the BIOS memory map. Check every reserved range before trusting allocation.

```
#include "memory/pmm.h"
#include "kernel/drivers/vga_text.h"

extern vga_text terminal;

uint32_t* bitmap = (uint32_t*)BITMAP_BASE;
uint32_t total_frames = 0;
uint32_t bitmap_entries = 0;
extern char _kernel_start;
extern char _kernel_end;

void bitmap_set_frame(uint32_t frame_index) {
    bitmap[frame_index / 32] |= (1 << (frame_index % 32));
}

void bitmap_clear_frame(uint32_t frame_index) {
    bitmap[frame_index / 32] &= ~(1 << (frame_index % 32));
}

void* alloc_frame() {
    uint32_t frame_i;
    for (frame_i = 0; frame_i < total_frames; frame_i++) {
        uint32_t is_reserved = (bitmap[frame_i / 32] & (1 << (frame_i % 32)));
        if (!is_reserved) {
            bitmap_set_frame(frame_i);
            return (void *)(frame_i * FRAME_SIZE);
        }
    }
    return NULL;
}

void free_frame(void* frame_address) {
    uint32_t frame_i = (uint32_t)frame_address / FRAME_SIZE;
    bitmap_clear_frame(frame_i);
}

void init_pmm() {
    uint16_t map_entry_count = *(uint16_t*)0x4FFC;
    memory_map_entry_t* memory_map = (memory_map_entry_t*)0x5000;

    /*

    vga_text_write(&terminal, "Entries: ");
    vga_text_write_hex(&terminal, map_entry_count);
    vga_text_writeline(&terminal, "");

    for (uint16_t i = 0; i < map_entry_count; i++) {
        // Format: #0: B:0x00000000 L:0x00000000 T:0x01
        vga_text_write(&terminal, "#");
        vga_text_write_dec(&terminal, i);
        vga_text_write(&terminal, " B:");
        vga_text_write_hex(&terminal, memory_map[i].base);
        vga_text_write(&terminal, " L:");
        vga_text_write_hex(&terminal, memory_map[i].length);
    }
    */
}
```

```

        vga_text_write(&terminal, " T:");
        vga_text_write_hex(&terminal, memory_map[i].type);
        vga_text_writeline(&terminal, "");
    }

    */

    uint64_t max_usable_address = 0;

    // find highest usable physical RAM address to calc max frames
    for (uint16_t i = 0; i < map_entry_count; i++) {
        if (memory_map[i].type == 1) {
            uint64_t highest_access = memory_map[i].base + memory_map[i].length;
            if (highest_access > max_usable_address) max_usable_address = highest_access;
        }
    }

    // calculate bitmap dimensions
    total_frames = max_usable_address / FRAME_SIZE;
    bitmap_entries = (total_frames + 32 - 1) / 32; //always round up
                                                    //

    // set all regions to reserved for safety
    for (uint32_t i = 0; i < bitmap_entries; i++) {
        bitmap[i] = 0xFFFFFFFF;
    }

    //mark usable regions as free using the bitmap
    for(uint32_t i = 0; i < map_entry_count; i++) {
        if (memory_map[i].type == 1) { // Usable RAM
            uint64_t starting_frame_index = (memory_map[i].base / FRAME_SIZE);
            uint64_t frame_length = memory_map[i].length / FRAME_SIZE;
            for (uint32_t j = starting_frame_index; j < starting_frame_index + frame_length; j++) {
                bitmap_clear_frame(j);
            }
        }
    }

    //protect kernel frames, bitmap and bootloader
    bitmap_set_frame(0); //protect bios data

    //kernel
    uint32_t kernel_frame_index = (uint32_t)&_kernel_start / FRAME_SIZE;
    uint32_t kernel_frame_end = (uint32_t)&_kernel_end / FRAME_SIZE;

    for (; kernel_frame_index < kernel_frame_end; kernel_frame_index++) {
        bitmap_set_frame(kernel_frame_index);
    }

    //bitmap
    uint32_t bitmap_frame_start = BITMAP_BASE / FRAME_SIZE;
    uint32_t bitmap_size_bytes = bitmap_entries * sizeof(uint32_t);
    uint32_t bitmap_frame_count = (bitmap_size_bytes + FRAME_SIZE - 1) / FRAME_SIZE;
    for(uint32_t i = 0; i < bitmap_frame_count; i++) {
        bitmap_set_frame(bitmap_frame_start + i);
    }

    // protect bootloader

```

```

    bitmap_set_frame(0x7C00 / FRAME_SIZE);

    // protect video memory
    uint32_t video_frame_index = 0xA0000 / FRAME_SIZE;
    uint32_t video_frame_end = 0xFFFFF / FRAME_SIZE;
    for (; video_frame_index < video_frame_end; video_frame_index++) {
        bitmap_set_frame(video_frame_index);
    }

    //print_bitmap_summary();
}

void print_bitmap_summary() {
    vga_text_write(&terminal, "BITMAP: ");

    uint32_t total_free_frames = 0;
    uint32_t total_used_frames = 0;

    // Get initial state of Frame 0: 1 = reserved, 0 = free
    uint32_t current_state = (bitmap[0] & 1) ? 1 : 0;
    uint32_t current_run_start = 0;

    // 1. Scan the entire bitmap to print consecutive blocks
    for (uint32_t f = 0; f < total_frames; f++) {
        uint32_t state = (bitmap[f / 32] & (1 << (f % 32))) ? 1 : 0;

        if (state == 0) total_free_frames++;
        else total_used_frames++;

        // When the state changes, print the memory block that just ended
        if (state != current_state) {
            vga_text_write(&terminal, current_state == 1 ? "[RSVD: 0x" : "[FREE: 0x");
            vga_text_write_hex(&terminal, current_run_start * 4096);
            vga_text_write(&terminal, "-0x");
            vga_text_write_hex(&terminal, ((f - 1) * 4096) + 4095);
            vga_text_write(&terminal, "] ");

            current_state = state;
            current_run_start = f;
        }
    }

    // Print the very last block of the loop
    vga_text_write(&terminal, current_state == 1 ? "[RSVD: 0x" : "[FREE: 0x");
    vga_text_write_hex(&terminal, current_run_start * 4096);
    vga_text_write(&terminal, "-0x");
    vga_text_write_hex(&terminal, ((total_frames - 1) * 4096) + 4095);
    vga_text_writeline(&terminal, "] ");

    // 2. Print the one-line summary totals
    vga_text_write(&terminal, "TOTALS -> Free: ");
    vga_text_write_dec(&terminal, total_free_frames);
    vga_text_write(&terminal, " frames (");
    vga_text_write_dec(&terminal, (total_free_frames * 4) / 1024); // KB to MB
    vga_text_write(&terminal, "MB) | Reserved: ");
    vga_text_write_dec(&terminal, total_used_frames);

```

```
    vga_text_writeline(&terminal, " frames.");  
}
```

E.3 ../lytlnybl/include/memory/vmm.h

Study focus: This header names page bits, table shapes, control-register helpers, and the mapping interface.

```
#ifndef VMM_H
#define VMM_H

#define PAGE_PRESENT (1 << 0)
#define PAGE_WRITABLE (1 << 1)
#define PAGE_USER (1 << 2)

//page fault definitions
#define PRESENT_FAULT (1 << 0)
#define WRITE_FAULT (1 << 1)
#define USER_FAULT (1 << 2)
#define RESERVED_FAULT (1 << 3)
#define INSTRUCTION_FETCH_FAULT (1 << 4)

#define PHYS_MAP_BASE 0xC0000000

#include <stdint.h>
#include "kernel/interrupts.h"

typedef uint32_t page_directory_t[1024];
typedef uint32_t page_table_t[1024];

void init_vmm();

extern void set_cr3(uintptr_t dir_ptr);
extern void set_cr0();
extern uint32_t get_cr2();

//calc dir index, calc table index, find/create page table, insert page dir
void map_page(
    page_directory_t* directory,
    uintptr_t virtual_address,
    uintptr_t physical_address,
    uint32_t flags
);

void unmap_page(page_directory_t* directory, uintptr_t virtual_address);
uintptr_t get_physical_address(page_directory_t* directory, uintptr_t virtual_address);

extern void flush_tlb(void);
extern void flush_tlb_page(uintptr_t virtual_address);

// Page table management

page_table_t* create_page_table(page_directory_t* directory, uint32_t directory_index, uint32_t flags);

// Page fault handling
void page_fault_handler(registers_t* registers);

extern page_directory_t* kernel_directory;
extern page_directory_t* current_directory;

#endif
```

E.4 ../lytlnybl/memory/vmm.asm

Study focus: These functions are tiny but privileged. Link each C declaration to its exact instruction.

[BITS 32]

```
global set_cr3
global set_cr0
global flush_tlb
global flush_tlb_page
global get_cr2
```

```
set_cr3:
    mov eax, [esp + 4]
    mov cr3, eax
    ret
```

```
set_cr0:
    mov eax, cr0
    or eax, 0x80000000
    mov cr0, eax
    ret
```

```
get_cr2:
    mov eax, cr2
    ret
```

```
; flush entire tlb
; reload CR3 with itself
; cpu discards all cached virtual physical translations
```

```
flush_tlb:
    mov eax, cr3
    mov cr3, eax
    ret
```

```
flush_tlb_page:
    mov eax, [esp + 4]
    invlpg [eax]
    ret
```

E.5 ../lytlnybl/memory/vmm.c

Study focus: Work through `init_vmm` and `map_page` with real addresses. Then read the page-fault diagnostic as a decoder of CPU evidence.

```
#include "memory/vmm.h"
#include "kernel/drivers/vga_text.h"
#include "memory/pmm.h"
#include "kernel/kernel_utils.h"

extern vga_text terminal;

page_directory_t* kernel_directory;
page_directory_t* current_directory;

void init_vmm() {
    kernel_directory = alloc_frame();
    page_table_t* tbl_ptr = alloc_frame();

    memset(kernel_directory, 0, 4096);
    memset(tbl_ptr, 0, 4096);

    //gives every page up to the limit an entry in the table.
    for (uint32_t i = 0; i < 1024; i++) {
        (*tbl_ptr)[i] = (i * 0x1000) | PAGE_PRESENT | PAGE_WRITABLE;
    }

    (*kernel_directory)[0] = ((uintptr_t)tbl_ptr) | PAGE_PRESENT | PAGE_WRITABLE;

    current_directory = kernel_directory;
    set_cr3((uintptr_t)kernel_directory);
    set_cr0();
}

page_table_t* create_page_table(page_directory_t* directory, uint32_t directory_index, uint32_t flags) {
    page_table_t* new_table = alloc_frame();
    memset(new_table, 0, 4096);
    (*directory)[directory_index] = (uintptr_t)new_table | flags;
    return new_table;
}

//DIRECTORY MUST BE A VIRTUAL ADDRESS, identity mapped for kernel
void map_page(page_directory_t* directory, uintptr_t virtual_address, uintptr_t physical_address, uint32_t flags) {
    uint16_t dir_index = (virtual_address >> 22);
    uint16_t table_index = (virtual_address >> 12 & 0x3FF);

    uint32_t dir_entry = (*directory)[dir_index];
    page_table_t* selected_table;
    if (!(dir_entry & PAGE_PRESENT)) {
        selected_table = create_page_table(directory, dir_index, flags);
    } else {
        selected_table = (page_table_t*)(dir_entry & 0xFFFFF000);
    }

    (*selected_table)[table_index] = physical_address | flags;
    if (directory == current_directory) {
        flush_tlb_page(virtual_address);
    }
}
```

```
}

void unmap_page(page_directory_t* directory, uintptr_t virtual_address) {
    uint16_t dir_index = (virtual_address >> 22);
    uint16_t table_index = (virtual_address >> 12 & 0x3FF);

    uint32_t dir_entry = (*directory)[dir_index];
    page_table_t* selected_table = (page_table_t*)(dir_entry & 0xFFFFF000);
    (*selected_table)[table_index] &= ~(PAGE_PRESENT);
    if (directory == current_directory) {
        flush_tlb_page(virtual_address);
    }
}

uintptr_t get_physical_address(page_directory_t* directory, uintptr_t virtual_address) {
    uint16_t dir_index = (virtual_address >> 22);
    uint16_t table_index = (virtual_address >> 12 & 0x3FF);

    uint32_t dir_entry = (*directory)[dir_index];
    page_table_t* selected_table = (page_table_t*)(dir_entry & 0xFFFFF000);
    return (*selected_table)[table_index] & 0xFFFFF000;
}

void page_fault_handler(registers_t* registers) {

    uintptr_t address = get_cr2();
    uint32_t error_code = registers->error_code;

    vga_text_writeline(&terminal, "PAGE FAULT");

    vga_text_write(&terminal, "Address: ");
    vga_text_write_hex(&terminal, address);
    vga_text_writeline(&terminal, "");

    vga_text_write(&terminal, "Error code: ");
    vga_text_write_hex(&terminal, error_code);
    vga_text_writeline(&terminal, "");

    if (error_code & PRESENT_FAULT) {
        vga_text_writeline(&terminal, "Reason: Protection violation");
    }
    else {
        vga_text_writeline(&terminal, "Reason: Page not present");
    }

    if (error_code & WRITE_FAULT) {
        vga_text_writeline(&terminal, "Access: Write");
    }
    else {
        vga_text_writeline(&terminal, "Access: Read");
    }

    if (error_code & USER_FAULT) {
```

```
        vga_text_writeline(&terminal, "Mode: User");
    }
    else {
        vga_text_writeline(&terminal, "Mode: Kernel");
    }

    if (error_code & RESERVED_FAULT) {
        vga_text_writeline(&terminal, "Reserved bit violation");
    }

    if (error_code & INSTRUCTION_FETCH_FAULT) {
        vga_text_writeline(&terminal, "Instruction fetch fault");
    }

    vga_text_write(&terminal, "Instruction address: ");
    vga_text_write_hex(&terminal, registers->eip);
    vga_text_writeline(&terminal, "");

}
```

E.6 ../lytlnybl/include/memory/heap.h

Study focus: The header describes variable-size blocks living inside pages that VMM has already mapped.

```
#ifndef HEAP_H
#define HEAP_H

#include <stddef.h>
#include <stdint.h>
#include <stdbool.h>

typedef struct heap_header{
    size_t size;
    bool free;
    struct heap_header* next;
} heap_header_t;

void init_heap(void);

void* kmalloc(size_t requested_size);
void kfree(void* ptr);

void expand_heap(size_t required_size);

// helpers

heap_header_t* find_free_block(size_t requested_size);

void split_block(
    heap_header_t* block,
    size_t size
);

void merge_blocks(
    heap_header_t* block
);

heap_header_t* get_header(void* ptr);

#endif
```

E.7 ../lytlnybl/memory/heap.c

Study focus: Draw the linked blocks after every call to split, expand, free, and merge. Distinguish allocator metadata from payload bytes.

```
#include "memory/heap.h"
#include "memory/vmm.h"
#include "memory/pmm.h"
#include "kernel/mappings.h"

heap_header_t* heap_start_head;
uintptr_t heap_end;

void init_heap() {
    void* physical_start = alloc_frame();
    map_page(kernel_directory, HEAP_START, (uintptr_t)physical_start, PAGE_PRESENT | PAGE_WRITABLE);
    heap_header_t* first_header = (heap_header_t*)HEAP_START;

    first_header->size = 4096 - sizeof(heap_header_t); //4096 is page size
    first_header->free = true;
    first_header->next = NULL;

    heap_start_head = first_header;
    heap_end = HEAP_START + 0x1000;
}

heap_header_t* find_free_block(size_t requested_size) {
    heap_header_t* current_head = heap_start_head;
    while (current_head) {
        if (current_head->free && current_head->size >= requested_size) {
            return current_head;
        }
        current_head = current_head->next;
    }
    return NULL;
}

void split_block(heap_header_t* block, size_t requested_size) {
    size_t space_remaining = (block->size - requested_size);
    if (space_remaining < sizeof(heap_header_t) + sizeof(uint8_t)) {
        return;
        //don't make block, not big enough
    };

    uintptr_t origin_block_end = (uintptr_t)(block) + block->size + sizeof(heap_header_t);
    uintptr_t new_block_start = (uintptr_t)(block) + requested_size + sizeof(heap_header_t);
    heap_header_t* new_block = (heap_header_t*)(new_block_start);

    new_block->size = origin_block_end - new_block_start - sizeof(heap_header_t);
    new_block->free = true;

    heap_header_t* temp_next = block->next;
    new_block->next = temp_next;
    block->next = new_block;

    block->size = requested_size;
}
```

```

void expand_heap(size_t required_size) {
    uint32_t required_pages = (required_size + 4096 - 1) / 4096;
    for (size_t i = 0; i < required_pages; i++) {
        void* new_page_physical_start = alloc_frame();
        map_page(kernel_directory, heap_end, (uintptr_t)new_page_physical_start, PAGE_PRESENT | PAGE_

        heap_header_t* traversal_head = heap_start_head;
        while (traversal_head) {
            if (!traversal_head->next && !traversal_head->free) {
                heap_header_t* new_frame_header = (heap_header_t*)heap_end;

                new_frame_header->free = true;
                new_frame_header->next = NULL;
                new_frame_header->size = 4096 - sizeof(heap_header_t);

                traversal_head->next = new_frame_header;
                break;
            } else if (!traversal_head->next && traversal_head->free) {
                traversal_head->size += 4096;
                break; //will just be empty space for the previous free head
            }
            traversal_head = traversal_head->next;
        }
        heap_end += 0x1000;
    }
}

void* kmalloc(size_t requested_size) {
    if (!requested_size) return NULL;

    heap_header_t* block = find_free_block(requested_size);
    if (block) {
        split_block(block, requested_size);
        block->free = false;
        return (void*)((uintptr_t)block + sizeof(heap_header_t)); //return free memory not the head
    } else {
        expand_heap(requested_size);
        return kmalloc(requested_size);
    }
}

heap_header_t* get_header(void* ptr) {
    return (heap_header_t*)((uintptr_t)ptr - sizeof(heap_header_t));
}

void merge_blocks(heap_header_t* block) {
    if (!block->next || !block->next->free) return;
    heap_header_t* block_to_merge = block->next;

    block->size += block_to_merge->size + sizeof(heap_header_t);
    block->next = block_to_merge->next;
    return merge_blocks(block); //do to next
}

void kfree(void* ptr) {
    if (!ptr) return;
    heap_header_t* block_to_free = get_header(ptr);
    block_to_free->free = true;
}

```

```
    merge_blocks(block_to_free);  
}
```

Appendix F

Processes, context, scheduler, and loading

This group turns interrupt frames and page directories into separate executions. A useful exercise is to color every field in `process_t` according to whether it belongs to identity, scheduling, registers, memory, input, or parent/child control.

F.1 `../lytlnybl/include/tasks/procman.h`

Study focus: This is the process contract. Read the state and type enums before looking at context switching.

```
#ifndef PROCMAN_H
#define PROCMAN_H

#include <stdint.h>
#include <stddef.h>

#define INITIAL_PID 1
#define KERNEL_STACK_SIZE 0x4000

extern uint8_t kernel_stack_bottom;

#include "memory/vmm.h"

typedef enum {
    PROCESS_RUNNING = 0,
    PROCESS_READY = 1,
    PROCESS_BLOCKED = 2,
    PROCESS_SLEEPING = 3,
    PROCESS_TERMINATED = 4
} process_states_t;

typedef struct {
    uint32_t ss;
    uint32_t eip;
    uint32_t cs;
    uint32_t eflags;
    uint32_t ds;

    uint32_t edi;
    uint32_t esi;
```

```
    uint32_t ebp;
    uint32_t esp;
    uint32_t ebx;
    uint32_t edx;
    uint32_t ecx;
    uint32_t eax;

} __attribute__((packed)) process_registers_t;

typedef enum {
    PROCESS_KERNEL,
    PROCESS_USER
} process_type_t;

typedef struct {
    void* buffer;
    uint32_t count;
    uint32_t size;
} process_reading_state_t;

typedef struct process {
    uint32_t pid_waiting_for;
    uint32_t parent_pid;

    uintptr_t user_heap_end;
    uint32_t heap_pages_allocated;

    uint32_t wake_tick;
    process_reading_state_t reading_state;

    uint32_t pid;

    process_registers_t regs;

    process_states_t state;
    process_type_t type;

    struct process* next;

    page_directory_t* page_directory;

    void* kstack;
    void* ustack;
} __attribute__((packed)) process_t;

void init_procman();

process_t* create_process(void* task_address, process_type_t type);
void create_kprocess(process_t* new_process);
void create_uprocess(process_t* new_process);

void destroy_process(process_t* proc);

process_t* find_process_by_pid(uint32_t pid);

extern process_t* current_process;
extern process_t* process_head;
```

```
typedef struct {
    uint32_t prev_tss;

    uint32_t esp0;
    uint32_t ss0;

    uint32_t esp1;
    uint32_t ss1;

    uint32_t esp2;
    uint32_t ss2;

    uint32_t cr3;
    uint32_t eip;
    uint32_t eflags;

    uint32_t eax;
    uint32_t ecx;
    uint32_t edx;
    uint32_t ebx;

    uint32_t esp;
    uint32_t ebp;
    uint32_t esi;
    uint32_t edi;

    uint32_t es;
    uint32_t cs;
    uint32_t ss;
    uint32_t ds;
    uint32_t fs;
    uint32_t gs;

    uint32_t ldt;

    uint16_t trap;
    uint16_t iomap_base;
} __attribute__((packed)) tss_t;

extern tss_t tss;
extern uint8_t gdt_tss[];

void init_tss(void);
void set_kernel_stack(uintptr_t stack);
extern void load_tss(void);

#endif
```

F.2 ../lytlnybl/tasks/procman.asm

Study focus: The TSS load is short; understand why its selector is 0x28 from the GDT layout.

[BITS 32]

global load_tss

```
load_tss:
    mov ax, 0x28
    ltr ax
    ret
```

F.3 ../lytlnybl/tasks/procman.c

Study focus: Follow kernel-process and user-process construction separately. For every allocated item, locate its cleanup path.

```
#include "tasks/procman.h"
#include "memory/heap.h"
#include "memory/pmm.h"
#include "kernel/mappings.h"
#include "kernel/kernel_utils.h"

process_t* process_head;
process_t* current_process;
uint32_t next_pid;

void init_procman() {
    process_head = NULL;
    next_pid = INITIAL_PID;

    process_t* kernel_process = kmalloc(sizeof(process_t));
    kernel_process->pid = next_pid++;
    kernel_process->state = PROCESS_RUNNING;
    kernel_process->page_directory = current_directory;

    process_registers_t kernel_regs = {0};

    kernel_process->regs = kernel_regs;
    kernel_process->next = NULL;

    kernel_process->kstack = (void *)&kernel_stack_bottom;

    process_head = kernel_process;
    current_process = kernel_process;
}

process_t* create_process(void* task_address, process_type_t type) {
    process_t* new_process = kmalloc(sizeof(process_t));

    new_process->wake_tick = 0;
    new_process->type = type;
    new_process->pid = next_pid++;
    new_process->state = PROCESS_READY;
    new_process->kstack = kmalloc(KERNEL_STACK_SIZE);
    new_process->next = NULL;

    new_process->regs.eax = 0;
    new_process->regs.ebx = 0;
    new_process->regs.ecx = 0;
    new_process->regs.edx = 0;
    new_process->regs.esi = 0;
    new_process->regs.edi = 0;
    new_process->regs.ebp = 0;

    new_process->regs.eip = (uintptr_t)task_address;
    //esp defined based on type

    new_process->regs.eflags = 0x202;
```

```

    if (type == PROCESS_KERNEL) {
        create_kprocess(new_process);
    } else {
        create_uprocess(new_process);
    }

    process_t* traversal_process = process_head;
    while (traversal_process) {
        if (!traversal_process->next) {
            traversal_process->next = new_process;
            break;
        }
        traversal_process = traversal_process->next;
    }

    return new_process;
}

void create_kprocess(process_t* new_process) {
    new_process->regs.esp = (uint32_t)(new_process->kstack) + KERNEL_STACK_SIZE;
    new_process->regs.cs = 0x08;
    new_process->regs.ds = 0x10;
    new_process->regs.ss = 0x10;
    new_process->regs.eflags = 0x202;

    new_process->page_directory = kernel_directory;
}

void create_uprocess(process_t* new_process) {
    new_process->regs.esp = USER_STACK_TOP;

    new_process->regs.cs = 0x18 | 3;
    new_process->regs.ds = 0x20 | 3;
    new_process->regs.ss = 0x20 | 3;

    new_process->regs.eflags = 0x202;

    new_process->page_directory = (page_directory_t*)alloc_frame();
    memset((page_directory_t*)new_process->page_directory, 0, 4096);

    for (uint32_t i = 0; i < 1024; i++) {
        (*new_process->page_directory)[i] = (*kernel_directory)[i];
    }

    uintptr_t ustack_frame = (uintptr_t)alloc_frame();

    map_page(new_process->page_directory,
        USER_STACK_TOP - 4096,
        ustack_frame,
        PAGE_PRESENT | PAGE_WRITABLE | PAGE_USER
    );

    map_page(new_process->page_directory,
        USER_HEAP_START,
        (uintptr_t)alloc_frame(),
        PAGE_PRESENT | PAGE_WRITABLE | PAGE_USER
    );
}

```

```

/* if you ever try to create more mappings and get a general protection fault
 * it is likely that you have overwritten an existing kernel mapping, as we copy
 * the kernel mappings to the user process for when we use our interrupts, just
 * be careful of this */

new_process->user_heap_end = USER_HEAP_START + 4096;
new_process->heap_pages_allocated = 1;

new_process->ustack = (void*)(USER_STACK_TOP - 4096);
}

void destroy_process(process_t* proc) {
    if (!proc || proc->state == PROCESS_RUNNING) return;

    // removing it from list
    process_t* traversal_process = process_head;
    while (traversal_process) {
        if (traversal_process->pid == process_head->pid && !traversal_process->next && traversal_pro
            process_head = traversal_process->next;
            break;
        }

        if (!traversal_process->next) {
            traversal_process = NULL;
            break;
        }

        if (traversal_process->next->pid == proc->pid) {
            traversal_process->next = traversal_process->next->next;
            break;
        }
        traversal_process = traversal_process->next;
    }

    if (!traversal_process) return;

    kfree(proc->kstack);
    if (proc->type == PROCESS_USER) {
        unmap_page(proc->page_directory, USER_STACK_TOP - 4096);
        unmap_page(proc->page_directory, USER_CODE_BASE);
        unmap_page(proc->page_directory, USER_VGA);
        unmap_page(kernel_directory, (uintptr_t)(proc->page_directory));
    }
    kfree(proc);
}

process_t* find_process_by_pid(uint32_t pid) {
    process_t* traversal_process = process_head;
    while (traversal_process) {
        if (traversal_process->pid == pid) {
            return traversal_process;
        }
        traversal_process = traversal_process->next;
    }
    return NULL;
}

```

```
tss_t tss;

void init_tss(void) {
    uintptr_t base = (uintptr_t)&tss;
    uint32_t limit = sizeof(tss_t) - 1;
    gdt_tss[0] = limit & 0xFF;
    gdt_tss[1] = (limit >> 8) & 0xFF;

    gdt_tss[2] = base & 0xFF;
    gdt_tss[3] = (base >> 8) & 0xFF;

    gdt_tss[4] = (base >> 16) & 0xFF;
    gdt_tss[5] = 0x89;

    gdt_tss[6] = (limit >> 16) & 0x0F;
    gdt_tss[7] = (limit >> 24) & 0xFF;

    memset(&tss, 0, sizeof(tss_t));

    tss.ss0 = 0x10;
    tss.esp0 = 0;

    tss.iomap_base = sizeof(tss_t);
    load_tss();
}
```

F.4 ../lytlnybl/include/tasks/context.h

Study focus: Compare its function signatures with the saved interrupt register layout.

```
#ifndef CONTEXT_H
#define CONTEXT_H

#include "tasks/procman.h"
#include <stdint.h>
#include <stdbool.h>

void context_switch(process_t* old_process,
                   process_t* new_process,
                   registers_t* regs);

void save_context(process_t* process, registers_t* regs);

void load_context(process_t* process, registers_t* regs);

extern volatile bool return_to_user;

#endif
```

F.5 ../lytlnybl/tasks/context.c

Study focus: Trace save, choose, CR3 switch, TSS update, destroy, and restore in order.

```
#include "tasks/context.h"
#include <stdbool.h>
volatile bool return_to_user;

void save_context(process_t* process, registers_t* regs) {
    process->regs.eip = regs->eip;
    process->regs.cs = regs->cs;
    process->regs.eflags = regs->eflags;
    process->regs.ds = regs->ds;

    process->regs.edi = regs->edi;
    process->regs.esi = regs->esi;
    process->regs.ebp = regs->ebp;
    process->regs.ebx = regs->ebx;
    process->regs.edx = regs->edx;
    process->regs.ecx = regs->ecx;
    process->regs.eax = regs->eax;

    if (process->type == PROCESS_KERNEL) {
        process->regs.esp = regs->esp;
    } else {
        process->regs.esp = regs->user_esp;
        process->regs.ss = regs->ss;
    }
}

void load_context(process_t* process, registers_t* regs) {
    regs->eip = process->regs.eip;
    regs->cs = process->regs.cs;
    regs->eflags = process->regs.eflags;
    regs->ds = process->regs.ds;

    regs->edi = process->regs.edi;
    regs->esi = process->regs.esi;
    regs->ebp = process->regs.ebp;
    regs->ebx = process->regs.ebx;
    regs->edx = process->regs.edx;
    regs->ecx = process->regs.ecx;
    regs->eax = process->regs.eax;

    if (process->type == PROCESS_KERNEL) {
        regs->esp = process->regs.esp;
    } else {
        regs->user_esp = process->regs.esp;
        regs->ss = process->regs.ss;
    }
}

void context_switch(process_t* old_process, process_t* new_process, registers_t* regs) {
    current_process = new_process;
    if (old_process->state == PROCESS_RUNNING) {
        old_process->state = PROCESS_READY;
    }
}
```

```
    save_context(old_process, regs);
    new_process->state = PROCESS_RUNNING;

    return_to_user = (new_process->type == PROCESS_USER);

    if (new_process->type == PROCESS_USER) {
        tss.esp0 = (uintptr_t)(new_process->kstack) + KERNEL_STACK_SIZE;
        set_cr3((uintptr_t)new_process->page_directory);
    } else if (new_process->type == PROCESS_KERNEL) {
        set_cr3((uintptr_t)kernel_directory);
    }

    if (old_process->state == PROCESS_TERMINATED) {
        destroy_process(old_process);
    }

    load_context(new_process, regs);
}
```

F.6 ../lytlnybl/include/tasks/scheduler.h

Study focus: The scheduler interface says when a timer tick becomes a scheduling decision.

```
#ifndef SCHEDULER_H
#define SCHEDULER_H

#include "tasks/procman.h"
#include "kernel/interrupts.h"

#define DEFAULT_TIME_SLICE 10

process_t* get_next_process();

void schedule(registers_t* regs);

#endif
```

F.7 ../lytlnybl/tasks/scheduler.c

Study focus: Simulate a circular ready list on paper. Check each state test in `get_next_process`.

```
#include "tasks/scheduler.h"
#include "tasks/context.h"

volatile uint32_t scheduler_tick_count = 0;
uint32_t time_slice = DEFAULT_TIME_SLICE;

process_t* get_next_process() {
    process_t* traversal_process = current_process;
    do {
        if (traversal_process->state == PROCESS_READY) {
            return traversal_process;
        }

        if (!traversal_process->next) {
            traversal_process = process_head;
        } else {
            traversal_process = traversal_process->next;
        }
    } while (!(traversal_process->state == PROCESS_RUNNING));

    return current_process;
}

void schedule(registers_t* regs) {
    scheduler_tick_count++;
    if (scheduler_tick_count >= time_slice) {
        scheduler_tick_count = 0;
        process_t* next_process = get_next_process();
        if (next_process != current_process) {
            context_switch(current_process, next_process, regs);
        }
    }
    return;
}
```

F.8 ../lytlnybl/include/tasks/loader.h

Study focus: The loader exposes a simple promise: a path either yields a process or fails.

```
#ifndef LOADER_H
#define LOADER_H

#include "tasks/procman.h"

process_t *load_program(const char* path);

#endif
```

F.9 ../lytlnybl/tasks/loader.c

Study focus: Compare the file size, page count, physical frames, virtual code address, and user linker script. Identify the large-file limitation.

```
#include "tasks/loader.h"
#include "filesystem/fs_manager.h"
#include "filesystem/fs.h"
#include "tasks/procman.h"
#include "memory/pmm.h"
#include "kernel/mappings.h"
#include "kernel/kernel_utils.h"

process_t* load_program(const char* path) {
    int32_t fd = fs_open(path);
    uint32_t inode_num = fs_fd_to_inode(fd);

    fs_inode_t file_inode;

    if (!fs_read_inode(inode_num, &file_inode))
        return NULL;

    size_t file_size = file_inode.size;

    uint8_t buffer[file_size];
    if (fs_read(fd, buffer, file_size) == -1)
        return NULL;

    process_t* user_proc = create_process((void*)USER_CODE_BASE, PROCESS_USER);

    for(size_t i = 0; i < ((file_size + 4095) / 4096); ++i) {
        void* code_frame = alloc_frame();

        //write to frame
        size_t offset = i * 4096;
        size_t bytes = file_size - offset;
        if (bytes > 4096)
            bytes = 4096;
        memcpy(code_frame, buffer + offset, bytes);

        map_page(
            user_proc->page_directory,
            USER_CODE_BASE + (4096 * i),
            (uintptr_t)code_frame,
            PAGE_PRESENT | PAGE_WRITABLE | PAGE_USER
        );
    }

    fs_close(fd);

    return user_proc;
}
```

Appendix G

Filesystem layout, core, and manager

This is a complete small storage stack: PIO sectors at the bottom, fixed blocks, metadata, paths, descriptors, and shell commands at the top. Do not modify layout constants without following their use in both the kernel and host formatter.

G.1 ../lytlnybl/include/filesystem/fs_layout.h

Study focus: Calculate the size of each packed on-disk structure. Mark superblock, bitmap, inode, and directory-entry fields.

```
#ifndef FS_LAYOUT_H
#define FS_LAYOUT_H

#include <stdint.h>

#define FS_BLOCK_SIZE 512

#define FS_START_BLOCK 70
#define FS_SUPERBLOCK 0 // 1 block
#define FS_BITMAP_BLOCK 1 //1 block
#define FS_INODE_START 2 //block count calculated when formatting

#define FS_FILENAME_LENGTH 32

#define FS_TYPE_FREE 0
#define FS_TYPE_FILE 1
#define FS_TYPE_DIRECTORY 2

#define FS_INODE_MAX_BLOCKS 10
#define FS_TOTAL_BLOCKS 1000
#define FS_TOTAL_INODES 128

#define FS_ROOT_INODE 0

#define FS_MAGIC 0xDEADBABE

typedef struct {
    uint32_t magic;

    uint32_t block_size;
    uint32_t total_blocks;

    uint32_t bitmap_start;
```

```
uint32_t inode_start;
uint32_t inode_count;
uint32_t inode_blocks;
uint32_t root_inode;

uint32_t data_start;

uint32_t free_blocks;
uint32_t free_inodes;
} fs_superblock_t;

typedef struct {
    uint32_t size;

    uint16_t type;

    uint32_t blocks[FS_INODE_MAX_BLOCKS];
} fs_inode_t;

typedef struct {
    uint32_t inode;
    char name[FS_FILENAME_LENGTH];
} fs_directory_entry_t;

#endif
```

G.2 ../lytlnybl/include/filesystem/fs.h

Study focus: Read this as the low-level filesystem API. Separate allocation, metadata, directory, and file-byte routines.

```
#ifndef FS_H
#define FS_H

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>
#include "filesystem/fs_layout.h"

bool fs_read_block(uint32_t block_num, void* buffer);
bool fs_write_block (uint32_t block_num, const void* buffer);

bool fs_write_superblock(const fs_superblock_t* superblock);
bool fs_read_superblock(fs_superblock_t* superblock);

bool fs_format(void);

int32_t fs_alloc_block(void);
bool fs_free_block(uint32_t block_num);

bool fs_read_inode(uint32_t inode_num, fs_inode_t* inode);
bool fs_write_inode(uint32_t inode_num, const fs_inode_t* inode);

int32_t fs_alloc_inode(uint8_t type);
bool fs_free_inode(uint32_t inode_num);

int32_t fs_find_directory_entry(uint32_t directory_inode_num, const char* name);
bool fs_add_directory_entry(uint32_t directory_inode_num, uint32_t inode_number, const char* name);
bool fs_remove_directory_entry(uint32_t directory_inode_num, const char* name);

int32_t fs_create_file(uint32_t directory_inode_num, const char* name);
int32_t fs_create_directory(uint32_t parent_inode_num, const char* name);
int32_t fs_read_file(uint32_t file_inode_num, void* read_buffer, uint32_t size, uint32_t offset);
int32_t fs_write_file(uint32_t inode_num, const void* write_buffer, uint32_t size, uint32_t offset);
bool fs_delete_file(uint32_t directory_inode_num, const char* name);

#endif
```

G.3 ../lytlnybl/filesystem/fs.c

Study focus: Follow formatting first, then block allocation, inode access, directory insertion, and byte-range read/write. At each multi-write operation, consider failure consistency.

```
#include "filesystem/fs.h"
#include "kernel/drivers/ata.h"
#include "kernel/kernel_utils.h"

bool fs_read_block(uint32_t block_num, void* buffer) {
    uint32_t sector = FS_START_BLOCK + block_num;
    return ata_read_sector(sector, buffer);
}

bool fs_write_block(uint32_t block_num, const void* buffer) {
    uint32_t sector = FS_START_BLOCK + block_num;
    return ata_write_sector(sector, buffer);
}

bool fs_write_superblock(const fs_superblock_t* superblock) {
    uint8_t buffer[FS_BLOCK_SIZE];

    memset(buffer, 0, FS_BLOCK_SIZE);

    memcpy(buffer, superblock, sizeof(fs_superblock_t));

    return fs_write_block(FS_SUPERBLOCK, buffer);
}

bool fs_read_superblock(fs_superblock_t* superblock) {
    uint8_t buffer[FS_BLOCK_SIZE];

    if (!fs_read_block(FS_SUPERBLOCK, buffer))
        return false;

    memcpy(superblock, buffer, sizeof(fs_superblock_t));

    if (superblock->magic != FS_MAGIC)
        return false;

    return true;
}

bool fs_format(void) {
    fs_superblock_t superblock;
    fs_inode_t root_inode;
    uint8_t buffer[FS_BLOCK_SIZE];

    superblock.magic = FS_MAGIC;
    superblock.block_size = FS_BLOCK_SIZE;

    superblock.total_blocks = FS_TOTAL_BLOCKS;
    superblock.bitmap_start = FS_BITMAP_BLOCK;

    superblock.inode_start = FS_INODE_START;
    superblock.inode_count = FS_TOTAL_INODES;
    superblock.inode_blocks = (superblock.inode_count * sizeof(fs_inode_t) + FS_BLOCK_SIZE - 1) / FS_BLOCK_SIZE;
    superblock.root_inode = FS_ROOT_INODE;
}
```

```
superblock.data_start = superblock.inode_start + superblock.inode_blocks;

// -1 for root dir remember
superblock.free_blocks = superblock.total_blocks - superblock.data_start - 1;
superblock.free_inodes = superblock.inode_count - 1;

if (!fs_write_superblock(&superblock))
    return false;

memset(buffer, 0, FS_BLOCK_SIZE);

//mark everything other than the data as used (in the bitmap)
for (uint32_t block = 0; block < superblock.data_start; block++) {
    uint32_t byte = block / 8;
    uint32_t bit = block % 8;
    buffer[byte] |= (1 << bit);
}

//mark root as used in bitmap
uint32_t root_block = superblock.data_start;
uint32_t root_byte = root_block / 8;
uint32_t root_bit = root_block % 8;
buffer[root_byte] |= (1 << root_bit);

if (!fs_write_block(superblock.bitmap_start, buffer))
    return false;

//clear inode table blocks
memset(buffer, 0, FS_BLOCK_SIZE);
for(
    uint32_t block = 0;
    block < superblock.inode_blocks;
    block++) {
    if(!fs_write_block(superblock.inode_start + block, buffer))
        return false;
}

//set up root inode
memset(&root_inode, 0, sizeof(fs_inode_t));

root_inode.type = FS_TYPE_DIRECTORY;
root_inode.size = 0;
root_inode.blocks[0] = root_block;

if (!fs_write_inode(superblock.root_inode, &root_inode))
    return false;

//empty root dir block
memset(buffer, 0, FS_BLOCK_SIZE);
if (!fs_write_block(root_block, buffer))
    return false;

return true;
}
```

```

int32_t fs_alloc_block(void) {
    uint8_t buffer[FS_BLOCK_SIZE];
    fs_superblock_t superblock;

    if (!fs_read_superblock(&superblock))
        return -1;
    if (!fs_read_block(superblock.bitmap_start, buffer))
        return -1;

    size_t i;
    for (i = 0; i < superblock.total_blocks && i < (FS_BLOCK_SIZE * 8); i++) {
        uint32_t is_reserved = (buffer[i / 8] & (1 << (i % 8)));
        if (!is_reserved) {
            buffer[i / 8] |= (1 << (i % 8));
            break;
        }
    }

    if (i == superblock.total_blocks)
        return -1;

    if (!fs_write_block(superblock.bitmap_start, buffer))
        return -1;
    superblock.free_blocks--;
    if (!fs_write_superblock(&superblock))
        return -1;
    return i;
}

bool fs_free_block(uint32_t block) {
    uint8_t buffer[FS_BLOCK_SIZE];
    fs_superblock_t superblock;

    if (!fs_read_superblock(&superblock))
        return false;

    if (block >= superblock.total_blocks)
        return false;

    if (!fs_read_block(superblock.bitmap_start, buffer))
        return false;

    uint32_t byte = block / 8;
    uint32_t bit = block % 8;

    buffer[byte] &= ~(1 << bit);

    superblock.free_blocks++;

    if (!fs_write_block(superblock.bitmap_start, buffer))
        return false;

    if (!fs_write_superblock(&superblock))
        return false;

    return true;
}

```

```
bool fs_read_inode(uint32_t inode_num, fs_inode_t* inode) {
    uint8_t buffer[FS_BLOCK_SIZE];
    fs_superblock_t superblock;
    if (!fs_read_superblock(&superblock))
        return false;

    if (inode_num >= superblock.inode_count)
        return false;

    uint32_t inodes_per_block = FS_BLOCK_SIZE / sizeof(fs_inode_t);
    uint32_t block_offset = inode_num / inodes_per_block;
    uint32_t inode_offset = inode_num % inodes_per_block;

    uint32_t block = superblock.inode_start + block_offset;

    if (!fs_read_block(block, buffer))
        return false;

    fs_inode_t* inodes = (fs_inode_t*)buffer;
    *inode = inodes[inode_offset];

    return true;
}

bool fs_write_inode(uint32_t inode_num, const fs_inode_t* inode) {
    fs_superblock_t superblock;
    uint8_t buffer[FS_BLOCK_SIZE];

    if (!fs_read_superblock(&superblock))
        return false;

    if (inode_num >= superblock.inode_count)
        return false;

    uint32_t inodes_per_block = FS_BLOCK_SIZE / sizeof(fs_inode_t);
    uint32_t block_offset = inode_num / inodes_per_block;
    uint32_t inode_offset = inode_num % inodes_per_block;

    uint32_t block = superblock.inode_start + block_offset;

    if (!fs_read_block(block, buffer))
        return false;

    fs_inode_t* inodes = (fs_inode_t*)buffer;
    inodes[inode_offset] = *inode;

    if (!fs_write_block(block, buffer))
        return false;

    return true;
}

int32_t fs_alloc_inode(uint8_t type) {
    fs_superblock_t superblock;
    fs_inode_t inode;

    if (!fs_read_superblock(&superblock))
```

```

    return -1;

for (size_t i = 0; i < superblock.inode_count; i++) {
    if (!fs_read_inode(i, &inode))
        return -1;

    if (inode.type == FS_TYPE_FREE) {
        inode.type = type;
        inode.size = 0;

        memset(inode.blocks, 0, sizeof(inode.blocks));

        if (!fs_write_inode(i, &inode))
            return -1;

        superblock.free_inodes--;

        if(!fs_write_superblock(&superblock))
            return -1;

        return i;
    }
}
return -1;
}

bool fs_free_inode(uint32_t inode_num) {
    fs_superblock_t superblock;
    fs_inode_t inode;
    bool failure = false;

    if (!fs_read_inode(inode_num, &inode))
        return false;

    for (size_t i = 0; i < FS_INODE_MAX_BLOCKS; i++) {
        if (inode.blocks[i] != 0) {
            //need a variable to represent failure, this is becuae we don't wanna return halfway though w
            //on a failure because this will corrupt the filesystem.
            if (!fs_free_block(inode.blocks[i]))
                failure = true;

            inode.blocks[i] = 0;
        }
    }

    if (failure)
        return false;

    inode.type = FS_TYPE_FREE;
    inode.size = 0;

    if (!fs_write_inode(inode_num, &inode))
        return false;

    if (!fs_read_superblock(&superblock))
        return false;

    superblock.free_inodes++;

```

```
    if (!fs_write_superblock(&superblock))
        return false;

    return true;
}

int32_t fs_find_directory_entry(uint32_t directory_inode_num, const char* name) {
    fs_inode_t directory;
    uint8_t buffer[FS_BLOCK_SIZE];

    if (!fs_read_inode(directory_inode_num, &directory))
        return -1;

    uint32_t entries_per_block = FS_BLOCK_SIZE / sizeof(fs_directory_entry_t);

    for (size_t i = 0; i < FS_INODE_MAX_BLOCKS; i++) {
        if (directory.blocks[i] == 0)
            break;

        if (!fs_read_block(directory.blocks[i], buffer))
            return -1;

        fs_directory_entry_t* entries = (fs_directory_entry_t*)buffer;

        for (uint32_t j = 0; j < entries_per_block; j++) {
            if (entries[j].inode == 0)
                continue;

            if (strcmp(name, entries[j].name) == 0)
                return entries[j].inode;
        }
    }

    return -1;
}

bool fs_add_directory_entry(uint32_t directory_inode_num, uint32_t inode_num, const char* name) {
    fs_inode_t directory;
    uint8_t buffer[FS_BLOCK_SIZE];

    if (!fs_read_inode(directory_inode_num, &directory))
        return false;

    uint32_t entries_per_block = FS_BLOCK_SIZE / sizeof(fs_directory_entry_t);

    for (size_t i = 0; i < FS_INODE_MAX_BLOCKS; i++) {
        //no block is assigned so allocate a block
        if (directory.blocks[i] == 0) {
            int32_t block_num = fs_alloc_block();

            if (block_num < 0)
                return false;

            directory.blocks[i] = block_num;

            memset(buffer, 0, FS_BLOCK_SIZE);
```

```

        if (!fs_write_block(block_num, buffer)) {
            fs_free_block(block_num);
            return false;
        }
    }

    if (!fs_read_block(directory.blocks[i], buffer))
        return false;

    fs_directory_entry_t* entries = (fs_directory_entry_t*)buffer;

    //search for free entry indicated by the .inode, if not found, will go to next iteration of
    //block loop, if free, allocate it accordingly and then return true
    for (size_t j = 0; j < entries_per_block; j++) {
        if (entries[j].inode == 0) {
            entries[j].inode = inode_num;

            memset(entries[j].name, 0, FS_FILENAME_LENGTH);

            strncpy(entries[j].name, name, FS_FILENAME_LENGTH - 1);

            if (!fs_write_block(directory.blocks[i], buffer))
                return false;

            directory.size += sizeof(fs_directory_entry_t);

            if (!fs_write_inode(directory_inode_num, &directory))
                return false;

            return true;
        }
    }
}
return false;
}

bool fs_remove_directory_entry(uint32_t directory_inode_num, const char* name) {
    fs_inode_t directory;
    uint8_t buffer[FS_BLOCK_SIZE];

    if(!fs_read_inode(directory_inode_num, &directory))
        return false;

    uint32_t entries_per_block = FS_BLOCK_SIZE / sizeof(fs_directory_entry_t);

    for (size_t i = 0; i < FS_INODE_MAX_BLOCKS; i++) {
        if (directory.blocks[i] == 0)
            break;

        if (!fs_read_block(directory.blocks[i], buffer))
            return false;

        fs_directory_entry_t* entries = (fs_directory_entry_t*)buffer;

        for (uint32_t j = 0; j < entries_per_block; j++) {

            if (entries[j].inode == 0)
                continue;

```

```
        if (strcmp(name, entries[j].name) == 0) {
            //entry to remove found
            entries[j].inode = 0;

            if (!fs_write_block(directory.blocks[i], buffer))
                return false;

            directory.size -= sizeof(fs_directory_entry_t);

            if (!fs_write_inode(directory_inode_num, &directory))
                return false;

            return true;
        }
    }
    return false;
}

int32_t fs_create_file(uint32_t directory_inode_num, const char* name) {
    if (fs_find_directory_entry(directory_inode_num, name) >= 0)
        return -1;

    int32_t file_inode = fs_alloc_inode(FS_TYPE_FILE);

    if (file_inode < 0)
        return -1;

    if (!fs_add_directory_entry(directory_inode_num, file_inode, name)) {
        fs_free_inode(file_inode);
        return -1;
    }

    return file_inode;
}

int32_t fs_create_directory(uint32_t parent_inode_num, const char* name) {
    if (fs_find_directory_entry(parent_inode_num, name) >= 0)
        return -1;

    int32_t dir_inode_num = fs_alloc_inode(FS_TYPE_DIRECTORY);

    if (dir_inode_num < 0)
        return -1;

    int32_t block_num = fs_alloc_block();

    if (block_num < 0) {
        fs_free_inode(dir_inode_num);
        return -1;
    }

    fs_inode_t dir_inode;

    if (!fs_read_inode(dir_inode_num, &dir_inode))
        goto fail;
```

```
memset(dir_inode.blocks, 0, sizeof(dir_inode.blocks));

dir_inode.blocks[0] = block_num;
dir_inode.size = 0;

uint8_t buffer[FS_BLOCK_SIZE];

memset(buffer, 0, FS_BLOCK_SIZE);

if (!fs_write_block(block_num, buffer))
    goto fail;

if (!fs_write_inode(dir_inode_num, &dir_inode))
    goto fail;

if (!fs_add_directory_entry(parent_inode_num, dir_inode_num, name))
    goto fail;

return dir_inode_num;

fail:
    fs_free_block(block_num);
    fs_free_inode(dir_inode_num);
    return -1;
}

int32_t fs_read_file(uint32_t file_inode_num, void* read_buffer, uint32_t size, uint32_t offset) {
    fs_inode_t file_inode;
    uint8_t buffer[FS_BLOCK_SIZE];

    if (!fs_read_inode(file_inode_num, &file_inode))
        return -1;

    if (offset >= file_inode.size)
        return 0; //EOF should return 0 bytes instead of an error

    if (offset + size > file_inode.size)
        size = file_inode.size - offset;

    uint32_t bytes_read = 0;

    uint8_t* destination = (uint8_t*)read_buffer;

    while (bytes_read < size) {
        uint32_t position = offset + bytes_read;

        uint32_t block_index = position / FS_BLOCK_SIZE;
        uint32_t block_offset = position % FS_BLOCK_SIZE;

        if (block_index >= FS_INODE_MAX_BLOCKS)
            break;

        if (!fs_read_block(file_inode.blocks[block_index], buffer))
            return -1;

        uint32_t bytes = FS_BLOCK_SIZE - block_offset;

        if (bytes > size - bytes_read)
```

```
        bytes = size - bytes_read;

        memcpy(destination + bytes_read, buffer + block_offset, bytes);

        bytes_read += bytes;
    }

    return bytes_read;
}

int32_t fs_write_file(uint32_t inode_num, const void* write_buffer, uint32_t size, uint32_t offset) {
    fs_inode_t file_inode;
    uint8_t buffer[FS_BLOCK_SIZE];

    if (!fs_read_inode(inode_num, &file_inode))
        return -1;

    const uint8_t* source = (const uint8_t*)write_buffer;
    uint32_t bytes_written = 0;

    while (bytes_written < size) {
        uint32_t position = offset + bytes_written;

        uint32_t block_index = position / FS_BLOCK_SIZE;
        uint32_t block_offset = position % FS_BLOCK_SIZE;

        if (block_index >= FS_INODE_MAX_BLOCKS)
            break;

        //block not allocated, so allocate one and make it free, if else just read block.
        if (file_inode.blocks[block_index] == 0) {
            int32_t block = fs_alloc_block();

            if (block < 0)
                return -1;

            file_inode.blocks[block_index] = block;

            memset(buffer, 0, FS_BLOCK_SIZE);
        } else {
            if (!fs_read_block(file_inode.blocks[block_index], buffer))
                return -1;
        }

        uint32_t bytes = FS_BLOCK_SIZE - block_offset;

        if (bytes > size - bytes_written)
            bytes = size - bytes_written;

        memcpy(buffer + block_offset, source + bytes_written, bytes);

        if (!fs_write_block(file_inode.blocks[block_index], buffer))
            return -1;

        bytes_written += bytes;
    }

    if (offset + bytes_written > file_inode.size)
```

```
    file_inode.size = offset + bytes_written;

    if (!fs_write_inode(inode_num, &file_inode))
        return -1;

    return bytes_written;
}

bool fs_delete_file(uint32_t directory_inode, const char* name) {
    int32_t inode_num = fs_find_directory_entry(directory_inode, name);

    if (inode_num < 0)
        return false;

    if (!fs_remove_directory_entry(directory_inode, name))
        return false;

    if (!fs_free_inode(inode_num))
        return false;

    return true;
}
```

G.4 ../lytlnybl/include/filesystem/fs_manager.h

Study focus: This header sits above raw inodes and defines paths, open records, and user-facing operations.

```
#ifndef FS_MANAGER_H
#define FS_MANAGER_H

#include <stdint.h>
#include <stdbool.h>

#define FS_MAX_OPEN_FILES 16
#define FS_FD_OFFSET 3

typedef struct {
    uint32_t inode;
    uint32_t offset;
    bool used;
} fs_file_t;

void fs_manager_init(void);

int32_t fs_resolve_path(const char* path);
uint32_t fs_fd_to_inode(int32_t fd);

int32_t fs_open(const char* path);
int32_t fs_close(int fd);
int32_t fs_read(int fd, void* buffer, uint32_t size);
int32_t fs_write(int fd, void* buffer, uint32_t size);

bool fs_mkdir(const char* path);
void fs_ls(const char* path);
bool fs_touch(const char* path);
bool fs_rm(const char* path);

#endif
```

G.5 ../lytlnybl/filesystem/fs_manager.c

Study focus: Trace `/bin/shell` through path resolution. Then trace a descriptor's offset across open, read/write, and close.

```
#include "filesystem/fs_manager.h"
#include "filesystem/fs.h"
#include "kernel/drivers/ata.h"
#include "kernel/drivers/vga_text.h"

extern vga_text terminal;
static fs_file_t open_files[FS_MAX_OPEN_FILES];

void fs_manager_init() {
    init_ata();

    for (int i = 0; i < FS_MAX_OPEN_FILES; i++) {
        open_files[i].used = false;
    }
}

int32_t fs_resolve_path(const char* path) {
    if (!path || path[0] != '/')
        return -1;

    uint32_t current_inode = FS_ROOT_INODE;
    uint32_t i = 1;

    while (path[i] != '\0') {
        char name[FS_FILENAME_LENGTH];
        uint32_t name_length = 0;

        while (path[i] == '/')
            i++;

        if (path[i] == '\0')
            break;

        while (path[i] != '/' && path[i] != '\0') {
            if (name_length >= FS_FILENAME_LENGTH - 1)
                return -1;

            name[name_length] = path[i];
            name_length++;
            i++;
        }

        name[name_length] = '\0';

        int32_t inode = fs_find_directory_entry(current_inode, name);

        if (inode < 0)
            return -1;

        current_inode = inode;
    }
    return current_inode;
}
```

```
int32_t fs_open(const char* path) {
    int32_t inode = fs_resolve_path(path);

    if (inode < 0)
        return -1;

    for (int i = 0; i < FS_MAX_OPEN_FILES; i++) {
        if (!open_files[i].used) {
            open_files[i].used = true;
            open_files[i].inode = inode;
            open_files[i].offset = 0;

            return i + FS_FD_OFFSET;
        }
    }
    return -1;
}

int32_t fs_close(int fd) {
    fd -= FS_FD_OFFSET;
    if (fd < 0 || fd >= FS_MAX_OPEN_FILES)
        return -1;

    if (!open_files[fd].used)
        return -1;

    open_files[fd].used = false;
    open_files[fd].inode = 0;
    open_files[fd].offset = 0;

    return 0;
}

int32_t fs_read(int fd, void* buffer, uint32_t size) {
    fd -= FS_FD_OFFSET;
    if (fd < 0 || fd >= FS_MAX_OPEN_FILES)
        return -1;

    if (!open_files[fd].used)
        return -1;

    int32_t bytes_read = fs_read_file(open_files[fd].inode,
        buffer,
        size,
        open_files[fd].offset
    );

    if (bytes_read < 0)
        return -1;

    open_files[fd].offset += bytes_read;

    return bytes_read;
}

int32_t fs_write(int fd, void* buffer, uint32_t size) {
    fd -= FS_FD_OFFSET;
```

```
if (fd < 0 || fd >= FS_MAX_OPEN_FILES)
    return -1;

if (!open_files[fd].used)
    return -1;

int32_t bytes_written = fs_write_file(open_files[fd].inode,
    buffer,
    size,
    open_files[fd].offset
);

if (bytes_written < 0)
    return -1;

open_files[fd].offset += bytes_written;

return bytes_written;
}

static bool split_path(const char* path, char* parent, char* name) {
    int length = 0;

    while (path[length] != '\0')
        length++;

    if (length == 0)
        return false;

    int last_slash = -1;
    for (int i = 0; i < length; i++) {
        if (path[i] == '/')
            last_slash = i;
    }

    int name_length = length - last_slash - 1;

    if (name_length <= 0 || name_length >= FS_FILENAME_LENGTH)
        return false;

    for (int i = 0; i < name_length; i++)
        name[i] = path[last_slash + 1 + i];

    name[name_length] = '\0';

    if (last_slash == 0) {
        parent[0] = '/';
        parent[1] = '\0';
    } else {
        for (int i = 0; i < last_slash; i++)
            parent[i] = path[i];

        parent[last_slash] = '\0';
    }

    return true;
}
```

```
bool fs_mkdir(const char* path) {
    char parent[256];
    char name[FS_FILENAME_LENGTH];

    if (!split_path(path, parent, name))
        return false;

    int32_t parent_inode = fs_resolve_path(parent);

    if (parent_inode < 0)
        return false;

    fs_inode_t inode;

    if (!fs_read_inode(parent_inode, &inode))
        return false;

    if (inode.type != FS_TYPE_DIRECTORY)
        return false;

    return fs_create_directory(parent_inode, name) >= 0;
}
```

```
bool fs_touch(const char* path) {
    char parent[256];
    char name[FS_FILENAME_LENGTH];

    if (!split_path(path, parent, name))
        return false;

    int32_t parent_inode = fs_resolve_path(parent);

    if (parent_inode < 0)
        return false;

    fs_inode_t inode;

    if (!fs_read_inode(parent_inode, &inode))
        return false;

    if (inode.type != FS_TYPE_DIRECTORY)
        return false;

    return fs_create_file(parent_inode, name) >= 0;
}
```

```
bool fs_rm(const char* path) {
    char parent[256];
    char name[FS_FILENAME_LENGTH];

    if (!split_path(path, parent, name))
        return false;

    int32_t parent_inode = fs_resolve_path(parent);

    if (parent_inode < 0)
        return false;
```

```
fs_inode_t inode;

if (!fs_read_inode(parent_inode, &inode))
    return false;

if (inode.type != FS_TYPE_DIRECTORY)
    return false;

return fs_delete_file(parent_inode, name);
}

void fs_ls(const char* path) {
    int32_t directory_inode = fs_resolve_path(path);

    if (directory_inode < 0)
        return;

    fs_inode_t directory;

    if (!fs_read_inode(directory_inode, &directory))
        return;

    if (directory.type != FS_TYPE_DIRECTORY)
        return;

    uint8_t buffer[FS_BLOCK_SIZE];

    uint32_t entries_per_block = FS_BLOCK_SIZE / sizeof(fs_directory_entry_t);

    for (int i = 0; i < FS_INODE_MAX_BLOCKS; i++) {
        if (directory.blocks[i] == 0)
            break;

        if (!fs_read_block(directory.blocks[i], buffer))
            return;

        fs_directory_entry_t* entries = (fs_directory_entry_t*)buffer;

        for (uint32_t j = 0; j < entries_per_block; j++) {
            if (entries[j].inode == 0)
                continue;

            fs_inode_t entry_inode_obj;

            if (!fs_read_inode(entries[j].inode, &entry_inode_obj))
                return;

            if (entry_inode_obj.type == FS_TYPE_DIRECTORY) {
                vga_text_set_color(&terminal, VGA_COLOR_LIGHT_MAGENTA, VGA_COLOR_RED);
            } else {
                vga_text_set_color(&terminal, VGA_COLOR_LIGHT_CYAN, VGA_COLOR_RED);
            }
            vga_text_write(&terminal, entries[j].name);
            vga_text_write(&terminal, " ");
            vga_text_set_color(&terminal, VGA_COLOR_WHITE, VGA_COLOR_RED);
        }
    }
}
```

```
    vga_text_writeline(&terminal, "");  
}  
  
uint32_t fs_fd_to_inode(int32_t fd) {  
    fd -= FS_FD_OFFSET;  
    return open_files[fd].inode;  
}
```

G.6 ../lytlnybl/tools/mkfs.c

Study focus: This host program must produce the exact layout the kernel expects. Compare its layout structures and allocation rules with the kernel filesystem.

```
#include "filesystem/fs_layout.h"

#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>
#include <string.h>

#include <dirent.h>
#include <sys/stat.h>

static bool write_block(FILE *image, uint32_t block_num, const void *buffer) {
    uint32_t sector = FS_START_BLOCK + block_num;

    if (fseek(image, sector * FS_BLOCK_SIZE, SEEK_SET) != 0)
        return false;
    return fwrite(buffer, 1, FS_BLOCK_SIZE, image) == FS_BLOCK_SIZE;
}

static bool read_block(FILE *image, uint32_t block_num, void *buffer) {
    uint32_t sector = FS_START_BLOCK + block_num;

    if (fseek(image, sector * FS_BLOCK_SIZE, SEEK_SET) != 0)
        return false;
    return fread(buffer, 1, FS_BLOCK_SIZE, image) == FS_BLOCK_SIZE;
}

static fs_superblock_t make_superblock(void) {
    fs_superblock_t superblock = {0};

    superblock.magic = FS_MAGIC;
    superblock.block_size = FS_BLOCK_SIZE;

    superblock.total_blocks = FS_TOTAL_BLOCKS;
    superblock.bitmap_start = FS_BITMAP_BLOCK;

    superblock.inode_start = FS_INODE_START;
    superblock.inode_count = FS_TOTAL_INODES;
    superblock.inode_blocks = (superblock.inode_count * sizeof(fs_inode_t) + FS_BLOCK_SIZE - 1) / FS_BLOCK_SIZE;
    superblock.root_inode = FS_ROOT_INODE;

    superblock.data_start = superblock.inode_start + superblock.inode_blocks;

    // -1 for root dir remember
    superblock.free_blocks = superblock.total_blocks - superblock.data_start - 1;
    superblock.free_inodes = superblock.inode_count - 1;

    return superblock;
}

static bool write_superblock(FILE *image, const fs_superblock_t *superblock) {
    uint8_t buffer[FS_BLOCK_SIZE] = {0};
    memcpy(buffer, superblock, sizeof(fs_superblock_t));
    return write_block(image, FS_SUPERBLOCK, buffer);
}
```

```

}

static bool write_bitmap(FILE *image, const fs_superblock_t *superblock) {
    uint8_t bitmap[FS_BLOCK_SIZE] = {0};

    for (uint32_t block = 0; block < superblock->data_start; block++) {
        bitmap[block / 8] |= (1 << (block % 8));
    }

    uint32_t root_block = superblock->data_start;

    bitmap[root_block / 8] |= (1 << (root_block % 8));

    return write_block(image, superblock->bitmap_start, bitmap);
}

static bool clear_inode_table(FILE *image, const fs_superblock_t* superblock) {
    uint8_t buffer[FS_BLOCK_SIZE] = {0};

    for (size_t i = 0; i < superblock->inode_blocks; i++) {
        if (!write_block(image, superblock->inode_start + i, buffer)) {
            return false;
        }
    }

    return true;
}

static bool write_root_inode(FILE* image, const fs_superblock_t* superblock) {
    fs_inode_t root = {0};

    root.type = FS_TYPE_DIRECTORY;
    root.size = 0;
    root.blocks[0] = superblock->data_start;

    //pointless calcs for now as root is the first inode, but if that changes this will be useful
    uint32_t inodes_per_block = FS_BLOCK_SIZE / sizeof(fs_inode_t);
    uint32_t block_offset = FS_ROOT_INODE / inodes_per_block;
    uint32_t inode_offset = FS_ROOT_INODE % inodes_per_block;
    uint32_t block = superblock->inode_start + block_offset;

    uint8_t buffer[FS_BLOCK_SIZE];
    if (fseek(image, (block) * FS_BLOCK_SIZE, SEEK_SET) != 0){
        return false;
    }

    if (fread(buffer, 1, FS_BLOCK_SIZE, image) != FS_BLOCK_SIZE) {
        return false;
    }

    fs_inode_t *inodes = (fs_inode_t *)buffer;
    inodes[inode_offset] = root;

    return write_block(image, block, buffer);
}

static bool format_filesystem(FILE *image) {

```

```

fs_superblock_t superblock = make_superblock();

if (!write_superblock(image, &superblock)) {
    fprintf(stderr, "mkfs: failed to write superblock\n");
    return false;
}

if (!write_bitmap(image, &superblock)) {
    fprintf(stderr, "mkfs: failed to write bitmap\n");
    return false;
}

if (!clear_inode_table(image, &superblock)) {
    fprintf(stderr, "mkfs: failed to clear inode table\n");
    return false;
}

if (!write_root_inode(image, &superblock)) {
    fprintf(stderr, "mkfs: failed to write root inode\n");
    return false;
}

return true;
}

/* -----
 *      REMAKE OF FS (needed) FUNCTIONS
 * -----
 */

int32_t alloc_block(FILE *image, fs_superblock_t* superblock) {
    uint8_t buffer[FS_BLOCK_SIZE];

    //read superblock
    if (!read_block(image, FS_SUPERBLOCK, buffer))
        return -1;
    memcpy(superblock, buffer, sizeof(fs_superblock_t));

    memset(buffer, 0, FS_BLOCK_SIZE);

    //read buffer
    if (!read_block(image, superblock->bitmap_start, buffer))
        return -1;

    size_t i;
    for (i = 0; i < superblock->total_blocks && i < (FS_BLOCK_SIZE * 8); i++) {
        uint32_t is_reserved = (buffer[i / 8] & (1 << (i % 8)));
        if (!is_reserved) {
            buffer[i / 8] |= (1 << (i % 8));
            break;
        }
    }

    if (i == superblock->total_blocks)
        return -1;

    if (!write_block(image, superblock->bitmap_start, buffer))
        return -1;

```

```

    superblock->free_blocks--;
    if (!write_superblock(image, superblock))
        return -1;
    return i;
}

bool write_inode(FILE* image, fs_superblock_t* superblock, uint32_t inode_num, const fs_inode_t* inode,
    uint8_t buffer[FS_BLOCK_SIZE]);

    if (!read_block(image, FS_SUPERBLOCK, buffer))
        return false;

    memcpy(superblock, buffer, sizeof(fs_superblock_t));

    if (inode_num >= superblock->inode_count)
        return false;

    uint32_t inodes_per_block = FS_BLOCK_SIZE / sizeof(fs_inode_t);
    uint32_t block_offset = inode_num / inodes_per_block;
    uint32_t inode_offset = inode_num % inodes_per_block;

    uint32_t block = superblock->inode_start + block_offset;

    if (!read_block(image, block, buffer))
        return false;

    fs_inode_t* inodes = (fs_inode_t*)buffer;
    inodes[inode_offset] = *inode;

    if (!write_block(image, block, buffer))
        return false;

    return true;
}

bool read_inode(FILE* image, fs_superblock_t* superblock, uint32_t inode_num, fs_inode_t* inode) {
    uint8_t buffer[FS_BLOCK_SIZE];

    if (!read_block(image, FS_SUPERBLOCK, buffer))
        return false;

    memcpy(superblock, buffer, sizeof(fs_superblock_t));
    memset(buffer, 0, FS_BLOCK_SIZE);

    if (inode_num >= superblock->inode_count)
        return false;

    uint32_t inodes_per_block = FS_BLOCK_SIZE / sizeof(fs_inode_t);
    uint32_t block_offset = inode_num / inodes_per_block;
    uint32_t inode_offset = inode_num % inodes_per_block;

    uint32_t block = superblock->inode_start + block_offset;

    if (!read_block(image, block, buffer))
        return false;

    fs_inode_t* inodes = (fs_inode_t*)buffer;
    *inode = inodes[inode_offset];

```

```

    return true;
}

bool add_directory_entry(FILE* image, fs_superblock_t* superblock, uint32_t directory_inode_num, uint32_t
    const char* name) {
    fs_inode_t directory;
    uint8_t buffer[FS_BLOCK_SIZE];

    if (!read_inode(image, superblock, directory_inode_num, &directory))
        return false;

    uint32_t entries_per_block = FS_BLOCK_SIZE / sizeof(fs_directory_entry_t);

    for (size_t i = 0; i < FS_INODE_MAX_BLOCKS; i++) {
        //no block is assigned so allocate a block
        if (directory.blocks[i] == 0) {
            int32_t block_num = alloc_block(image, superblock);

            if (block_num < 0)
                return false;

            directory.blocks[i] = block_num;

            memset(buffer, 0, FS_BLOCK_SIZE);

            if (!write_block(image, block_num, buffer)) {
                return false;
            }
        }

        if (!read_block(image, directory.blocks[i], buffer))
            return false;

        fs_directory_entry_t* entries = (fs_directory_entry_t*)buffer;

        //search for free entry indicated by the .inode, if not found, will go to next iteration of
        //block loop, if free, allocate it accordingly and then return true
        for (size_t j = 0; j < entries_per_block; j++) {
            if (entries[j].inode == 0) {
                entries[j].inode = inode_num;

                memset(entries[j].name, 0, FS_FILENAME_LENGTH);

                strncpy(entries[j].name, name, FS_FILENAME_LENGTH - 1);

                if (!write_block(image, directory.blocks[i], buffer))
                    return false;

                directory.size += sizeof(fs_directory_entry_t);

                if (!write_inode(image, superblock, directory_inode_num, &directory))
                    return false;

                return true;
            }
        }
    }
}

```

```

    }
    return false;
}

/* -----
 *          DIRECTORY PARSING
 * -----
 */

uint32_t next_free_inode_num = 1;

static bool install_file(FILE* image, fs_superblock_t *sb, uint32_t parent_inode_num, const char* name,
    const char* host_path) {

    FILE *file = fopen(host_path, "rb");

    if (next_free_inode_num >= sb->inode_count) {
        fprintf(stderr, "mkfs: no free inode for %s\n", host_path);
        return false;
    }

    uint32_t inode_num = next_free_inode_num++;

    if (!file) {
        perror(host_path);
        return false;
    }

    if (fseek(file, 0, SEEK_END) != 0) {
        fclose(file);
        return false;
    }

    size_t file_size = ftell(file);

    if (file_size < 0) {
        fclose(file);
        return false;
    }

    rewind(file);

    fs_inode_t file_inode = {0};

    file_inode.type = FS_TYPE_FILE;
    file_inode.size = (uint32_t)file_size;
    uint8_t buffer [FS_BLOCK_SIZE];

    uint32_t remaining = file_inode.size;

    //write data to blocks
    for (size_t i = 0; remaining > 0 && i < FS_INODE_MAX_BLOCKS; i++) {
        int32_t block_num = alloc_block(image, sb);

```

```
    if (block_num < 0) {
        fclose(file);
        return false;
    }

    file_inode.blocks[i] = block_num;

    memset(buffer, 0, FS_BLOCK_SIZE);

    uint32_t bytes = remaining > FS_BLOCK_SIZE ? FS_BLOCK_SIZE : remaining;

    if (fread(buffer, 1, bytes, file) != bytes) {
        fclose(file);
        return false;
    }

    if (!write_block(image, block_num, buffer)) {
        fclose(file);
        return false;
    }

    remaining -= bytes;
}

fclose(file);

if (remaining != 0)
    return false;

if (!write_inode(image, sb, inode_num, &file_inode))
    return false;

if (!add_directory_entry(image, sb, parent_inode_num, inode_num, name))
    return false;

return true;
}

static bool install_directory(FILE* image, fs_superblock_t* sb, uint32_t parent_inode_num, const char*
    const char* host_path) {

    if (next_free_inode_num >= sb->inode_count) {
        fprintf(stderr, "mkfs: no free inode for the directory %s\n", host_path);
        return false;
    }

    uint32_t inode_num = next_free_inode_num++;
    fs_inode_t inode = {0};

    if (inode_num >= sb->inode_count) {
        fprintf(stderr, "mkfs: no free inode for the directory %s\n", host_path);
        return false;
    }
}
```

```

int32_t block = alloc_block(image, sb);

if (block < 0) {
    fprintf(stderr, "mkfs: no free block for directory %s\n", host_path);
    return false;
}

inode.type = FS_TYPE_DIRECTORY;
inode.size = 0;
inode.blocks[0] = block;

if (!write_inode(image, sb, inode_num, &inode))
    return false;

//write to it's parent and stuff
fs_inode_t parent;

if (!read_inode(image, sb, parent_inode_num, &parent))
    return false;

if (!add_directory_entry(image, sb, parent_inode_num, inode_num, name))
    return false;

//now install everything inside of directory same logic as rootfs install

DIR *dir = opendir(host_path);
if (!dir) {
    perror(host_path);
    return false;
}

struct dirent* entry;

while ((entry = readdir(dir)) != NULL) {
    if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)
        continue;

    char path [512];

    snprintf(path, sizeof(path), "%s/%s", host_path, entry->d_name);

    struct stat st;

    if (stat(path, &st) != 0) {
        perror(path);
        closedir(dir);
        return false;
    }

    if (S_ISREG(st.st_mode)) {
        if (!install_file(image, sb, inode_num, entry->d_name, path)) {
            closedir(dir);
            return false;
        }
    }

    } else if (S_ISDIR(st.st_mode)) {
        if (!install_directory(image, sb, inode_num, entry->d_name, path)) {

```

```
        closedir(dir);
        return false;
    }
}
closedir(dir);
return true;
}

static bool install_rootfs(FILE* image, fs_superblock_t *sb) {
    DIR *dir = opendir("rootfs");

    if (!dir) {
        perror("rootfs");
        return false;
    }

    struct dirent *entry;

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)
            continue;

        char path [512];

        snprintf(path, sizeof(path), "rootfs/%s", entry->d_name);

        struct stat st;

        if (stat(path, &st) != 0) {
            perror(path);
            closedir(dir);
            return false;
        }

        if (S_ISREG(st.st_mode)) {
            if (!install_file(image, sb, 0, entry->d_name, path)) {
                closedir(dir);
                return false;
            }
        } else if (S_ISDIR(st.st_mode)) {
            if (!install_directory(image, sb, 0, entry->d_name, path)) {
                closedir(dir);
                return false;
            }
        }
    }
    closedir(dir);
    return true;
}

int main(int argc, char **argv)
{
```

```
if (argc != 2) {
    fprintf(stderr, "usage: mkfs <image>\n");
    return 1;
}

FILE *image = fopen(argv[1], "r+b");

if (!image) {
    perror("mkfs: fopen");
    return 1;
}

printf("formatting %s.....\n", argv[1]);

if (!format_filesystem(image)) {
    fprintf(stderr, "mkfs: formatting failed\n");
    fclose(image);
    return 1;
}

fs_superblock_t sb_copy;

uint8_t buffer[FS_BLOCK_SIZE];

if (!read_block(image, FS_SUPERBLOCK, buffer)) {
    fprintf(stderr, "mkfs: failed to read superblock\n");
    fclose(image);
    return 1;
}

memcpy(&sb_copy, buffer, sizeof(fs_superblock_t));

if (!install_rootfs(image, &sb_copy)){
    fprintf(stderr, "mkfs: failed to install rootfs\n");
    fclose(image);
    return 1;
}

fclose(image);

printf("mkfs success\n");

return 0;
}
```

Appendix H

User library, binaries, and shell

The guest user program has no host C library. It gets a deliberately small library whose functions ultimately cross the `int 0x80` boundary. Read these files after the syscall dispatcher, not before it.

H.1 `../lytlnybl/user/user.ld`

Study focus: Confirm that this base address is the one the loader maps.

```
ENTRY(_start)

SECTIONS
{
    . = 0x00400000;

    .text :
    {
        *(.text)
    }

    .rodata :
    {
        *(.rodata)
    }

    .data :
    {
        *(.data)
    }

    .bss :
    {
        *(.bss)
    }
}
```

H.2 ../lytlnybl/include/user/libc/syscalls.h

Study focus: Compare every number and operation constant against the kernel interrupt header.

```
#ifndef SYSCALLS_H
#define SYSCALLS_H

#include <stdint.h>
#include <stddef.h>

#define SYSCALL_EXIT 0x01
#define SYSCALL_GETPID 0x02
#define SYSCALL_YIELD 0x03
#define SYSCALL_SLEEP 0x04
#define SYSCALL_WRITE 0x05
#define SYSCALL_READ 0x06
#define SYSCALL_SBRK 0x07
#define SYSCALL_FSOPS 0x08
#define SYSCALL_OPEN 0x09
#define SYSCALL_CLOSE 0x0A

#define FS_LS 0
#define FS_MKDIR 1
#define FS_TOUCH 2
#define FS_RM 3
#define FS_RUN 4

extern uint32_t syscall(uint32_t CODE, uint32_t a, uint32_t b, uint32_t c);

void exit(void);
uint32_t get_pid(void);
void yield();
void sleep(uint32_t ticks);
int write(int fd, void *buff, size_t count);
int read(int fd, void* buff, size_t count);
void* sbrk (intptr_t increment);
void fs_ops(uint8_t operation, char* path);
int open(const char* path);
void close(int fd);

#endif
```

H.3 ../lytlnybl/user/libc/syscalls.asm

Study focus: This is the narrowest possible gateway from normal C calls to the kernel.

```
global syscall
```

```
syscall:
```

```
    mov eax, [esp + 4]  
    mov ebx, [esp + 8]  
    mov ecx, [esp + 12]  
    mov edx, [esp + 16]
```

```
    int 0x80  
    ret
```

H.4 ../lytlnybl/user/libc/syscalls.c

Study focus: Trace arguments and result values for write, read, heap growth, and filesystem calls.

```
#include "user/libc/syscalls.h"

void exit() {
    syscall(SYSCALL_EXIT, 0, 0, 0);
} //get warning here due to function name maybe, ignore it

uint32_t get_pid() {
    return syscall(SYSCALL_GETPID, 0, 0, 0);
}

void yield() {
    syscall(SYSCALL_YIELD, 0, 0, 0);
}

void sleep(uint32_t ticks) {
    syscall(SYSCALL_SLEEP, ticks, 0, 0);
}

int write(int fd, void *buff, size_t count) {
    return syscall(SYSCALL_WRITE, fd, (uint32_t)buff, count);
}

int read(int fd, void* buff, size_t count) {
    return syscall(SYSCALL_READ, fd, (uint32_t)buff, count);
}

void* sbrk (intptr_t increment) {
    return (void*)syscall(SYSCALL_SBRK, increment, 0, 0);
}

void fs_ops(uint8_t operation, char* path) {
    syscall(SYSCALL_FSOPS, operation, (uint32_t)path, 0);
}

int open(const char* path) {
    return syscall(SYSCALL_OPEN, (uint32_t)path, 0, 0);
}

void close(int fd) {
    syscall(SYSCALL_CLOSE, fd, 0, 0);
}
```

H.5 ../lytlnybl/include/user/libc/memory.h

Study focus: This header describes a second allocator, distinct from the kernel heap.

```
#ifndef MEMORY_H
#define MEMORY_H

#include <stddef.h>
#include <stdint.h>
#include <stdbool.h>
#include "../../kernel/mappings.h"
#include "syscalls.h"

typedef struct heap_header{
    size_t size;
    bool free;
    struct heap_header* next;
} heap_header_t;

void* malloc(size_t bytes);
void free(void* ptr);
int memcmp(void* ptr1, void* ptr2, size_t n);
void* memcpy(void* dest, const void* src, size_t n);
void* memmove(void* dest, const void* src, size_t bytes);
void* memset(void* ptr, uint8_t c, size_t n);
void* calloc(size_t n, size_t size);
void* realloc(void* ptr, size_t new_size);

#endif
```

H.6 ../lytlnybl/user/libc/memory.c

Study focus: Follow `sbrk` when the user heap grows. Compare its block list with the kernel heap's list.

```
#include "user/libc/memory.h"

heap_header_t* heap_start_head;
uintptr_t heap_end;

void init_heap() {
    heap_end = USER_HEAP_START + 4096;
    heap_start_head = (void*)USER_HEAP_START;
    heap_start_head->size = 4096 - sizeof(heap_header_t);
    heap_start_head->free = true;
    heap_start_head->next = NULL;
}

void split_block(heap_header_t* block, size_t requested_size) {

    size_t space_remaining = block->size - requested_size;

    if (space_remaining < sizeof(heap_header_t) + sizeof(uint8_t)) {
        return;
    }

    heap_header_t* new = (void*)((void*)block + requested_size + sizeof(heap_header_t));
    new->size = (block->size)-requested_size-sizeof(heap_header_t);
    new->free = true;
    new->next = block->next;

    block->size = requested_size;
    block->free = false;
    block->next = new;
}

heap_header_t* find_free_block(size_t requested_size) {
    heap_header_t* current_head = heap_start_head;
    while (current_head) {
        if (current_head->free && current_head->size >= requested_size) {
            return current_head;
        }
        current_head = current_head->next;
    }
    return NULL;
}

void expand_heap(size_t requested_size) {
    uintptr_t original_end = heap_end;
    heap_end = (uintptr_t)sbrk(requested_size);

    heap_header_t* traversal_head = heap_start_head;
    while (traversal_head) {
        if (!traversal_head->next && !traversal_head->free) {
            heap_header_t* new_header = (heap_header_t*)original_end;

            new_header->free = true;
            new_header->next = NULL;
        }
    }
}
```

```

        new_header->size = requested_size - sizeof(heap_header_t);

        traversal_head->next = new_header;
        break;
    } else if(!traversal_head->next && traversal_head->free) {
        traversal_head->size += requested_size;
        break;
    }
    traversal_head = traversal_head->next;
}
}

void *malloc(size_t bytes) {
    if (bytes == 0)
        return NULL;
    void *result;
    heap_header_t *curr;

    if (!heap_start_head) init_heap();

    heap_header_t* block = find_free_block(bytes);
    if (block) {
        if (block->size != bytes) split_block(block, bytes);
        block->free = false;
        result = (void*)((uintptr_t)block + sizeof(heap_header_t));
    } else {
        expand_heap(bytes);
        return malloc(bytes);
    }
    return result;
}

void merge_blocks(heap_header_t* block) {
    if (!block->next || !block->next->free) return;
    heap_header_t* block_to_merge = block->next;

    block->size += block_to_merge->size + sizeof(heap_header_t);
    block->next = block_to_merge->next;
    return merge_blocks(block);
}

heap_header_t* get_header(void* ptr) {
    return (heap_header_t*)((uintptr_t)ptr - sizeof(heap_header_t));
}

void free(void* ptr) {
    if (!ptr) return;
    heap_header_t* block_to_free = get_header(ptr);
    block_to_free->free = true;
    merge_blocks(block_to_free);
}

void* memcpy(void* dest, const void* src, size_t n) {
    uint8_t* d = (uint8_t*)dest;
    const uint8_t* s = (const uint8_t*)src;

    for (size_t i = 0; i < n; i++) {
        d[i] = s[i];
    }
}

```

```
}

return dest;
}

void* memmove(void* dest, const void* src, size_t n) {
    uint8_t* d = (uint8_t*)dest;
    const uint8_t* s = (const uint8_t*)src;

    if (d == s || n == 0) return dest;

    if (d < s) {
        //safe to copy won't overwrite the source
        for(size_t i = 0; i < n; i++) {
            d[i] = s[i];
        }
    } else {
        // copy backwards so source isn't overwritten
        for (size_t i = n; i > 0; i--) {
            d[i - 1] = s[i - 1];
        }
    }
    return dest;
}

void* memset(void* ptr, uint8_t c, size_t n) {
    uint8_t* p = ptr;
    while (n--) {
        *p++ = (c);
    }
    return ptr;
}

int memcmp(void* ptr1, void* ptr2, size_t n) {
    size_t i;
    uint8_t* p1 = (uint8_t*)ptr1;
    uint8_t* p2 = (uint8_t*)ptr2;
    int compare_status = 0;

    if (ptr1 == ptr2) return compare_status;

    while (n > 0) {
        if (*p1 != *p2) {
            compare_status = (*p1 > *p2) ? 1 : -1;
            break;
        }
        n--;
        p1++;
        p2++;
    }
    return compare_status;
}

void* calloc(size_t n, size_t size) {
    size_t total = n * size;
    void *ptr = malloc(total);

    if (ptr) memset(ptr,0,total);
}
```

```
    return ptr;
}

void* realloc(void* ptr, size_t new_size) {
    if (!ptr) return malloc(new_size);

    if (new_size == 0) {
        free(ptr);
        return NULL;
    }

    heap_header_t* old_header = get_header(ptr);

    void* new_ptr = malloc(new_size);

    if (new_ptr == NULL) return NULL;

    size_t copy_size = old_header->size;

    if (copy_size > new_size) copy_size = new_size;

    memcpy(new_ptr, ptr, copy_size);
    free(ptr);
    return new_ptr;
}
```

H.7 ../lytlnybl/include/user/libc/string.h

Study focus: These are the minimal string contracts used by the shell and filesystem-facing code.

```
#ifndef STRING_H
#define STRING_H

#include <stddef.h>

size_t strlen(const char *str);
int strcmp(const char *str1, const char *str2);
int strncmp(const char *str1, const char *str2, size_t n);
char* strcpy(char* dest, const char *src);
char* strcat(char* dest, const char *src);
char* strchr(const char *str, int c);
char* strstr(const char *text, const char *search);

#endif
```

H.8 ../lytlnybl/user/libc/string.c

Study focus: Practice tracing null-terminated strings one byte at a time.

```
#include "user/libc/string.h"

size_t strlen(const char *str) {
    size_t len = 0;

    while (str[len] != '\0') len++;

    return len;
}

int strcmp(const char *str1, const char *str2) {
    while (*str1 && *str1 == *str2) {
        str1++;
        str2++;
    }
    return *str1 - *str2;
}

int strncmp(const char *str1, const char *str2, size_t n) {
    size_t i = 0;

    while (i < n) {
        char c1 = str1[i];
        char c2 = str2[i];

        if (c1 != c2) return c1 - c2;

        if (c1 == '\0') return 0;

        i++;
    }

    return 0;
}

char* strcpy(char *dest, const char *src) {
    while ((*dest++ = *src++) != '\0');
    return dest;
}

char* strcat(char* dest, const char* src) {
    while (*dest)dest++;
    while ((*dest++ = *src++) != '\0');
    return dest;
}

char* strchr(const char *str, int c) {
    while (*str) {
        if (*str == c) return (char *)str;
        str++;
    }

    if (c == '\0') return (char *)str;
}
```

```
    return NULL;
}

char* strstr(const char *text, const char *search) {
    size_t search_length;
    size_t i;
    size_t j;

    search_length = strlen(search);

    for (i = 0; text[i] != '\0'; i++) {
        for (j = 0; j < search_length; j++) {
            if (text[i + j] == '\0') break;

            if (text[i + j] != search[j]) break;
        }

        if (j == search_length) return (char*)&text[i];
    }

    return NULL;
}
```

H.9 ../lytlnybl/include/user/libc/chars.h

Study focus: This header defines classification helpers used by parsing.

```
#ifndef CHARS_H
#define CHARS_H

int isalpha(int c);
int isdigit(int c);
int isalnum(int c);
int isspace(int c);

int islower(int c);
int isupper(int c);

int tolower(int c);
int toupper(int c);

#endif
```

H.10 ../lytlnybl/user/libc/chars.c

Study focus: Check which declared functions exist and which character ranges are recognized.

```
#include "user/libc/chars.h"

int isalpha(int c) {
    return ((c >= 'a' && c <= 'z') ||
            (c >= 'A' && c <= 'Z'));
}

int isdigit(int c) {
    return (c >= '0' && c <= '9');
}

int isspace(int c) {
    return (c == ' ' ||
            c == '\t' ||
            c == '\n' ||
            c == '\v' ||
            c == '\f' ||
            c == '\r');
}

int islower(int c) {
    return (c >= 'a' && c <= 'z');
}

int isupper(int c) {
    return (c >= 'A' && c <= 'Z');
}

int tolower(int c) {
    if (isupper(c)) return c + ('a' - 'A');
    return c;
}

int toupper(int c) {
    if (islower(c)) return c - ('a' - 'A');
    return c;
}
```

H.11 `../lytlnybl/include/user/libc/output.h`

Study focus: The output API is intentionally smaller than standard C.

```
#ifndef OUTPUT_H
#define OUTPUT_H

#include <stdarg.h>
#include "syscalls.h"

void printf(char* format, ...);
int putchar(int c);
int puts(char* str);

#endif
```

H.12 ../lytlnybl/user/libc/output.c

Study focus: Follow `printf` into `putchar`, then into `write` and the syscall boundary.

```
#include "user/libc/output.h"

int putchar(int c) {
    return write(1, &c, 1) ? 1 : -1;
}

int puts(char* str) {
    while(*str != '\0') {
        if (!putchar(*str)) return -1;
        str++;
    }
    return 1;
}

void print_num(int num) {
    if (num == 0) {
        putchar('0');
        return;
    }

    if (num < 0) {
        putchar('-');
        num = -num;
    }

    char buffer[10];
    int i = 0;

    while (num > 0) {
        buffer[i++] = (num % 10) + '0';
        num /= 10;
    }

    while (i--) {
        putchar(buffer[i]);
    }
}

void printf(char* format, ...) {
    char *traverse;
    unsigned int i;
    char *s;

    va_list args;
    va_start(args, format);

    while (*format) {
        if (*format == '%') {
            format++;
            if (*format == 'c') {
                char c = va_arg(args, int);
                putchar(c);
            } else if (*format == 's') {
```

```
        char *str = va_arg(args, char*);
        puts(str);
    } else if (*format == 'd') {
        int num = va_arg(args, int);
        print_num(num);
    } else if (*format == '%') {
        putchar('%');
    } else {
        //unknown format, so print the raw
        putchar('%');
        putchar(*format);
    }
} else {
    putchar(*format);
}
format++;
}
va_end(args);
}
```

H.13 ../lytlnybl/include/user/programs/shell.h

Study focus: Read the parser and command declarations before the implementation.

```
#ifndef SHELL_H
#define SHELL_H

#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

char** tokenize_line(const char* line, size_t *count_out);
bool execute_command(char** words, size_t word_count);

#endif
```

H.14 ../lytlnybl/user/programs/shell.c

Study focus: Trace the REPL from prompt to input, tokens, command selection, syscall, and next prompt. Note the present two-word command restriction.

```
#include "user/programs/shell.h"

#include "user/libc/syscalls.h"
#include "user/libc/output.h"
#include "user/libc/chars.h"
#include "user/libc/memory.h"
#include "user/libc/string.h"

void start(void) {
    //REPL
    for (;;) {
        printf("> ");
        //Read
        char input[50];
        read(0, input, 50);
        input[strlen(input) - 1] = '\0';

        //Eval
        size_t word_count;
        char** words = tokenize_line(input, &word_count);
        //printf("we typed: %s, %d, %d, %s\n", input, strlen(input), word_count, words[0]);

        //Print
        if (!execute_command(words, word_count)) {
            printf("command failed\n");
        }

        //CLEANUP
        memset(input, 0, strlen(input));
        word_count = 0;
        free(words);
    }
}

bool execute_command(char** words, size_t word_count) {
    if (word_count != 2) {
        return false;
    }

    int fd;

    if (strcmp(words[0], "ls") == 0) {
        fs_ops(FS_LS, words[1]);
    }
    else if (strcmp(words[0], "mkdir") == 0) {
        fs_ops(FS_MKDIR, words[1]);
    }
    else if (strcmp(words[0], "touch") == 0) {
        fs_ops(FS_TOUCH, words[1]);
    }
}
```

```
}
else if (strcmp(words[0], "rm") == 0) {
    fs_ops(FS_RM, words[1]);
}
else if (strcmp(words[0], "run") == 0) {
    fs_ops(FS_RUN, words[1]);
}
else if (strcmp(words[0], "cat") == 0) {
    fd = open(words[1]);

    if (fd < 0) {
        return false;
    }

    char buffer[128];
    int n;

    while ((n = read(fd, buffer, sizeof(buffer))) > 0) {
        write(1, buffer, n);
    }
    printf("\n");

    close(fd);
}
else if (strcmp(words[0], "write") == 0) {
    fd = open(words[1]);

    if (fd < 0) {
        return false;
    }

    size_t capacity = 128;
    size_t length = 0;

    char* buffer = malloc(capacity);

    if (buffer == NULL) {
        close(fd);
        return false;
    }

    while (true) {
        char input[128];

        int n = read(0, input, sizeof(input));

        if (n < 0) {
            free(buffer);
            close(fd);
            return false;
        }

        if (n == 0) {
            break;
        }

        if (length + n > capacity) {
            while (length + n > capacity) {
```

```
        capacity *= 2;
    }

    char *new_buffer = realloc(buffer, capacity);

    if (new_buffer == NULL) {
        free(buffer);
        close(fd);
        return false;
    }

    buffer = new_buffer;
}

memcpy(buffer + length, input, n);
length += n;

if (input[n-1] == '\n') {
    break;
}
}

if (length > 0 && buffer[length - 1] == '\n') {
    length--;
}

if (length > 0) {
    if (write(fd, buffer, length) != (int)length) {
        free(buffer);
        close(fd);
        return false;
    }
}

free(buffer);
close(fd);
}
else {
    //command not recognised
    return false;
}

return true;
}

char** tokenize_line(const char* line, size_t *count_out) {
    const char* p = line;
    char **words;
    size_t words_i = 0;
    size_t capacity = 8;

    // limit of words in a line is 8
    words = calloc(capacity + 1, sizeof(char *));

    while (*p != '\0') {
        (*count_out)++;
        const char* start;
```

```
while (isspace((unsigned char) *p)) {
    p++;
}

if (*p == '\0') {
    break;
}

start = p;

while (*p != '\0' && !isspace((unsigned char) *p)) {
    p++;
}
size_t len = p - start;
char *word = malloc(len + 1);
if (word == NULL) {
    return NULL;
}

memcpy(word, start, len);
word[len] = '\0';
if (word == NULL) {
    return NULL;
}
words[words_i++] = word;
}
return words;
}
```

H.15 ../lytlnybl/user/programs/test.c

Study focus: Use this small program as a template for a new user binary and Makefile rule.

```
#include <stdint.h>
#include "user/libc/output.h"
#include "user/libc/memory.h"

#include "user/libc/syscalls.h"

void _start(void)
{
    char input[5];

    char text[10] = "hellohello";
    write(1, (void*)input, 5);
    putchar('a');
    putchar('b');
    putchar('\n');
    printf("i have a number it is: %d \n", 314);

    printf("malloc: ");

    char *ptr = malloc(10);
    ptr[0] = 'h';
    ptr[1] = 'i';
    ptr[2] = '\0';

    puts(ptr);

    free(ptr);

    printf("\ncalloc: ");

    char *ptr2 = calloc(5, 1);
    ptr2[0] = 'a';
    ptr2[1] = 'b';
    ptr2[2] = 'c';
    ptr2[3] = '\0';

    puts(ptr2);

    free(ptr2);

    printf("\nrealloc: ");

    char *ptr3 = malloc(5);

    ptr3[0] = 'h';
    ptr3[1] = 'e';
    ptr3[2] = 'l';
    ptr3[3] = 'l';
    ptr3[4] = '\0';

    ptr3 = realloc(ptr3, 10);
```

```
    ptr3[4] = 'o';
    ptr3[5] = 'o';
    ptr3[6] = 'o';

    puts(ptr3);
    printf("\n");

    free(ptr3);

    exit();
}
```

Appendix I

Reference study projects

After reading a group, choose one small project and write its interface, address-space assumptions, and test before coding. Suggested projects: print an address-translation trace; add a safe page-table inspector; add a shell help command; give each process its own file descriptor array; add a serial logger; or convert the flat loader into a minimal ELF parser. The tutorial part of this book explains the concepts; this reference part gives you the exact code surface where those concepts become real.

Quick reference

Term	Meaning in this project
BIOS	Firmware interface that loads the boot sector in QEMU.
GDT / IDT	Tables that describe protected-mode segments and event handlers.
IRQ	Hardware interrupt request, such as timer IRQ 0 or keyboard IRQ 1.
PMM frame	A 4096-byte physical memory unit tracked by a bitmap.
Page mapping	A virtual-to-physical relation stored in page tables.
CR3	CPU register holding the active page-directory address.
TSS	Structure providing a kernel stack for privilege transitions.
PID	Small numeric identifier for a process.
System call	Controlled request from user mode to kernel mode through <code>int 0x80</code> .
Inode	Metadata record for a file or directory.
File descriptor	Small integer naming an entry in the open-file table.
MKFS	Host-side image formatter and <code>rootfs</code> importer.